Laboratorio 1 – Implementación de monolito con patrón MVC y DAO

Universidad: Pontificia Universidad Javeriana\ **Facultad:** Ingeniería de Sistemas\ **Materia:** Arquitectura de Software\ **Profesor:** Andrés Sánchez\ **Integrantes:** José Eduardo Valeriano Castiblanco, Jorge Andrés Fortich Ordosgoitia y Camila Beltrán Carreño\ **Fecha:** 29 de octubre de 2026\ **Ciudad:** Bogotá D.C.

Repositorio: https://github.com/joseval29/personapi-dotnet\ **Entrega (tag v1.0.0):** https://github.com/joseval29/personapi-dotnet/tree/v1.0.0

---

1. Marco conceptual

Arquitectura monolítica. Un monolito es una aplicación en la que todas las funciones (interfaz, lógica de negocio y acceso a datos) se compilan y despliegan como una sola unidad. Su ventaja es la simplicidad: un solo proyecto, un solo despliegue y transacciones locales. Su desventaja es que cualquier cambio exige recompilar y desplegar todo, y que las partes crecen acopladas si no se separan con cuidado. En este laboratorio el monolito se organiza en capas para reducir ese acoplamiento.

Patrón MVC (Modelo–Vista–Controlador). Separa la aplicación en tres responsabilidades: el *modelo* (datos y reglas), la *vista* (presentación) y el *controlador* (recibe la petición, coordina y devuelve la respuesta). En ASP.NET Core MVC, los controladores son clases que heredan de `Controller`, las vistas son archivos Razor (`.cshtml`) en `Views` y los modelos son las clases de `Models`. En este proyecto `PersonasController`, `ProfesionesController`, `EstudiosController` y `TelefonosController` atienden las pantallas de administración, y `HomeController` muestra la página de inicio.

Patrón DAO / Repository. El DAO (*Data Access Object*), en su variante moderna de repositorio, aísla el acceso a datos detrás de una interfaz. Los controladores dependen de `IPersonaRepository` y no de EF Core directamente, así que la implementación (`PersonaRepository`) puede cambiar sin tocar la lógica de presentación. En `Program.cs`, `AddScoped<IPersonaRepository, PersonaRepository>()` hace la inyección de dependencias.

.NET y ASP.NET Core. .NET es la plataforma de desarrollo de Microsoft. Su runtime, el **CLR** (*Common Language Runtime*), ejecuta el código compilado en lenguaje intermedio (IL) y gestiona memoria y excepciones. **.NET Framework** es la implementación original, ligada a Windows. **.NET Core** y su sucesor unificado, **.NET 5 en adelante**, es multiplataforma (Windows, macOS y Linux) y de código abierto, lo que permitió desarrollar el laboratorio en macOS con Apple Silicon. El proyecto usa **.NET 7.0.410**, versión que la guía pedía y que ya no tiene soporte de Microsoft.

Entity Framework Core y ORM. EF Core es un ORM (*Object–Relational Mapper*): traduce consultas LINQ a SQL y convierte las filas en objetos. Existen dos enfoques. En **Code First** se escriben las clases y EF genera la base con migraciones. En **Database First** la base ya existe y EF genera las clases a partir de ella. El laboratorio usa Database First: `dotnet ef dbcontext scaffold` lee `persona_db` y genera `Persona`, `Profesion`, `Estudio`, `Telefono` y `PersonaDbContext` en `Models/Entities`.

REST. REST modela la API como *recursos* (`/api/personas`, `/api/estudios/{idProf}/{ccPer}`) manipulados con verbos HTTP: `GET` para consultar, `POST` para crear, `PUT` para reemplazar y `DELETE` para

eliminar. Los códigos de estado indican el resultado: **200** (OK con cuerpo), **201** (creado), **204** (exitoso sin cuerpo), **400** (datos inválidos) y **404** (no encontrado).

Swagger / OpenAPI 3. OpenAPI es un estándar para describir APIs REST en un documento legible por máquinas. Swashbuckle (`Swashbuckle.AspNetCore` 6.5.0) genera ese documento y la interfaz Swagger UI, en `/swagger`, desde donde se prueban los endpoints.

SQL Server 2022 y Docker. SQL Server 2022 Express es la edición gratuita del motor de Microsoft, con límites de tamaño y memoria. Como el laboratorio se hizo en un Mac, el motor corre en un contenedor Docker con la imagen `mcr.microsoft.com/mssql/server:2022-latest`. Esa imagen es `linux/amd64`, así que en Apple Silicon se usa la emulación de Rosetta de Docker Desktop.

DTO (Data Transfer Object). Un DTO es un objeto que transporta datos entre capas y hacia el cliente, distinto de la entidad de base de datos. Sirve para no exponer la estructura interna de la base, para validar con atributos (`[Required]`, `[StringLength]`, `[Range]`) y para incluir campos de solo lectura como `NombrePersona` o `NombreProfesion`, que no existen en la tabla.

---

2. Diseño

2.1 Modelo de datos

La base `persona_db` tiene cuatro tablas:

Tabla	Clave primaria	Campos	Relaciones
`persona`	`cc`	nombre, apellido, genero (`M/F`), edad	1:N con `telefono` y con `estudios`
`profesion`	`id`	nom, des	1:N con `estudios`
`estudios`	(`id_prof`, `cc_per`)	fecha, univer	N:M entre `persona` y `profesion`
`telefono`	`num`	oper, duenio	N:1 con `persona` mediante `duenio`

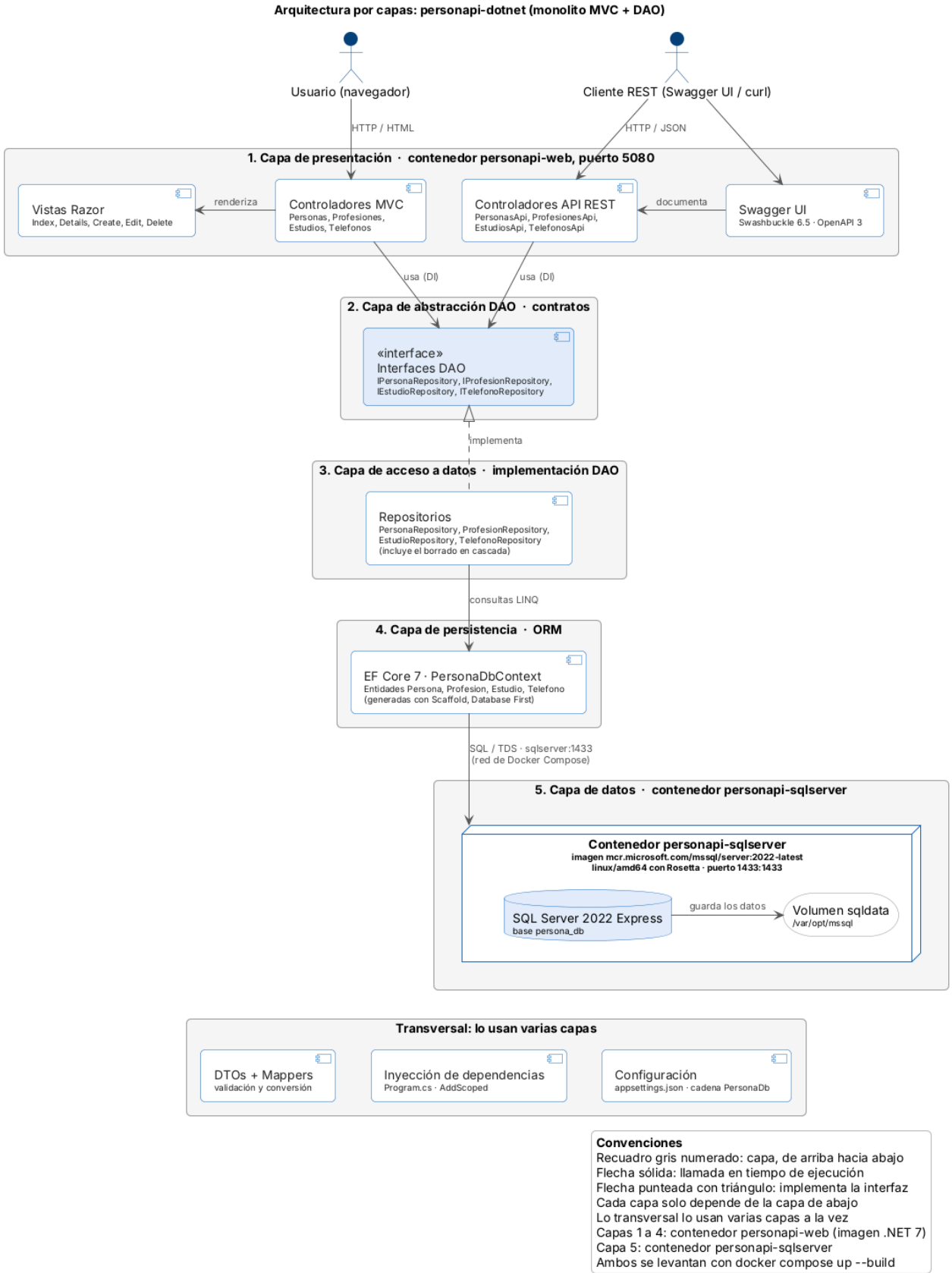
- **Persona–teléfono (1:N):** una persona puede tener varios teléfonos, y cada teléfono tiene un solo dueño (`telefono.duenio` → `persona.cc`).
- **Persona–profesión (N:M):** una persona puede estudiar varias profesiones, y una profesión la estudian varias personas. La tabla `estudios` resuelve la relación con una llave compuesta (`id_prof`, `cc_per`), que impide registrar dos veces la misma profesión para la misma persona.

Las restricciones están en `scripts/01_ddl.sql`.

2.2 Arquitectura por capas

Los controladores nunca usan `PersonaDbContext` directamente: dependen de las interfaces, y solo los repositorios hablan con EF Core.

Este diagrama es para ver las cinco capas de arriba hacia abajo, con la regla de dependencia: cada capa solo llama a la de abajo. Se muestra la presentación (controladores MVC y API REST, Vistas Razor y Swagger UI), la abstracción DAO con sus interfaces, la implementación con repositorios, la persistencia con EF Core y la capa de datos en SQL Server dentro de Docker. Los recuadros transversales (DTOs y mappers, inyección de dependencias y configuración) los usan varias capas a la vez.



Fuente del diagrama: `diagramas/08_arquitectura_capas.puml`.

2.3 Clases principales

Clase	Responsabilidad
`Persona`, `Profesion`, `Estudio`, `Telefono`	Entidades generadas por Scaffold. Mapean las tablas.
`PersonaDbContext`	Contexto de EF Core, con un `DbSet` por entidad.
`PersonaDto`, `ProfesionDto`, `EstudioDto`, `TelefonoDto`	DTOs con validación y campos de solo lectura.
`Mappers`	Convierte entre entidades y DTOs.
`I*Repository`	Contrato de acceso a datos.
`*Repository`	Implementación con EF Core.
`*Controller` / `*ApiController`	Controladores MVC (vistas) y REST (JSON).

2.4 Estructura de carpetas

```
personapi-dotnet/                  (raíz del repositorio, git)
├── docker-compose.yml             SQL Server 2022 Express en Docker
├── scripts/                       01_ddl.sql y 02_dml.sql
├── docs/                          este documento y capturas/
└── personapi-dotnet/              proyecto ASP.NET Core MVC (net7.0)
    ├── Controllers/               MVC y Controllers/Api (REST)
    ├── Interfaces/                contratos DAO
    ├── Repositories/              implementaciones DAO
    ├── Models/Dtos/               DTOs y mappers
    ├── Models/Entities/           entidades y PersonaDbContext (Scaffold)
    ├── Views/                     vistas Razor
    ├── wwwroot/                   CSS y JS estáticos
    ├── Program.cs                 registro de servicios, EF Core y Swagger
    └── appsettings.json           cadena de conexión PersonaDb
```

2.5 Endpoints REST

Recurso	GET lista	GET por id	POST	PUT	DELETE
`/api/personas`	200	`/{cc}`: 200 / 404	201	`/{cc}`: 204	`/{cc}`: 204
`/api/profesiones`	200	`/{id}`: 200 / 404	201	`/{id}`: 204	`/{id}`: 204
`/api/estudios`	200	`/{idProf}/{ccPer}`: 200 / 404	201	`/{idProf}/{ccPer}`: 204	`/{idProf}/{ccPer}`: 204
`/api/telefonos`	200	`/{num}`: 200 / 404	201	`/{num}`: 204	`/{num}`: 204

2.6 Decisiones de diseño

- **DTOs en lugar de entidades.** Las entidades de Scaffold tienen propiedades de navegación (`Estudios`, `Telefonos`) que, serializadas, crean ciclos y exponen más datos de los necesarios. Los

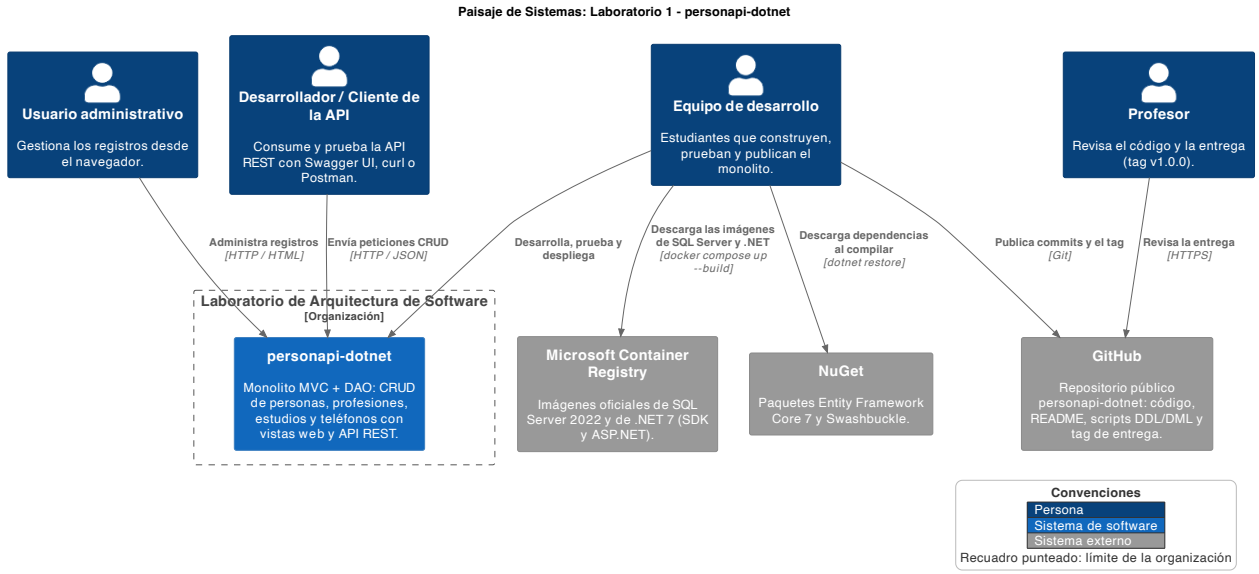
DTOs muestran solo lo que la vista o la API necesitan y llevan las reglas de validación.

- **Borrado en cascada manejado por el repositorio.** Las claves foráneas se crearon sin `ON DELETE CASCADE`. `PersonaRepository.DeleteAsync` quita primero los estudios y teléfonos de la persona, y `ProfesionRepository.DeleteAsync` quita los estudios de la profesión antes de eliminarla. La regla de negocio queda en la capa DAO, y la vista de confirmación lo advierte al usuario.
- **Adaptación del modelo de MySQL a SQL Server.** Los `ENUM` se cambiaron por restricciones `CHECK` (por ejemplo, `CHECK (genero IN ('M', 'F'))`), y `TEXT` por `VARCHAR(MAX)`, porque `TEXT` está obsoleto en SQL Server.
- **Llave compuesta en `estudios`.** En el contexto se usa `HasKey(e => new { e.IdProf, e.CcPer })`, la forma natural de representar una relación N:M.
- **Ajustes posteriores al Scaffold.** No hizo falta cambiar nombres de propiedades ni tipos: las entidades generadas coinciden con las de la referencia.

2.7 Diagramas C4

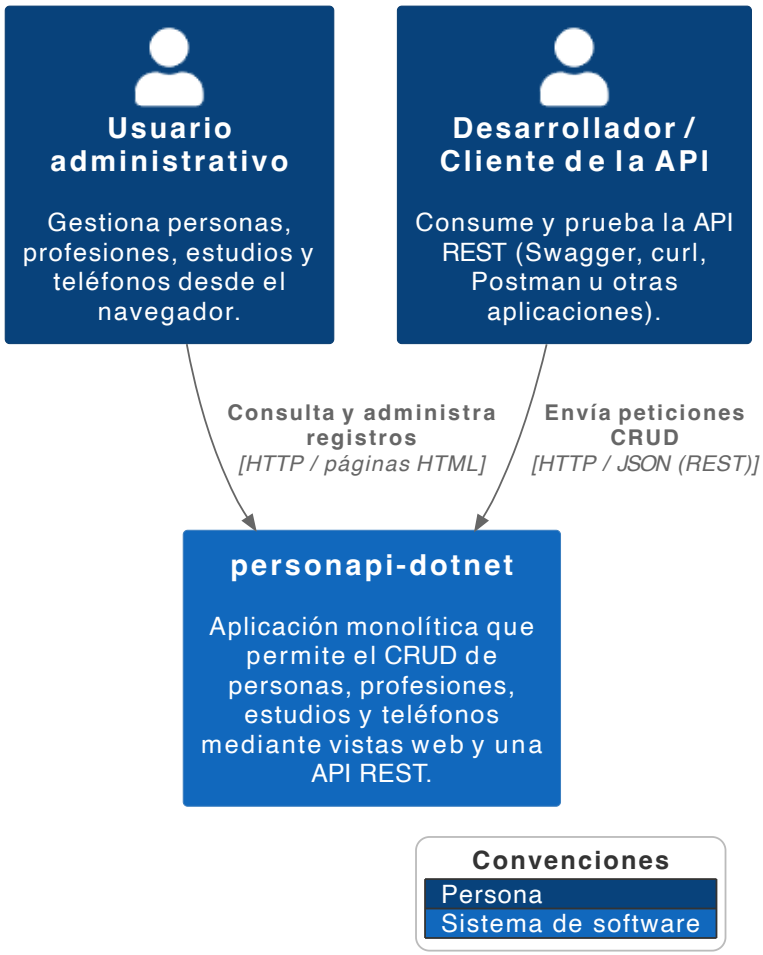
Los diagramas siguen el modelo C4 de Simon Brown, que describe un sistema en niveles de zoom: de lo más general (contexto) a lo más detallado (código). Los dos diagramas dinámicos muestran cómo colaboran las partes en un caso de uso concreto, y el de despliegue muestra dónde corre todo. Cada uno responde una pregunta distinta, así que conviene leerlos en orden.

Paisaje del sistema. Este diagrama es la vista de conjunto del laboratorio. Muestra el sistema `personapi-dotnet` junto con quién lo usa, sin entrar en detalles técnicos, para ubicar el problema antes de descomponerlo.



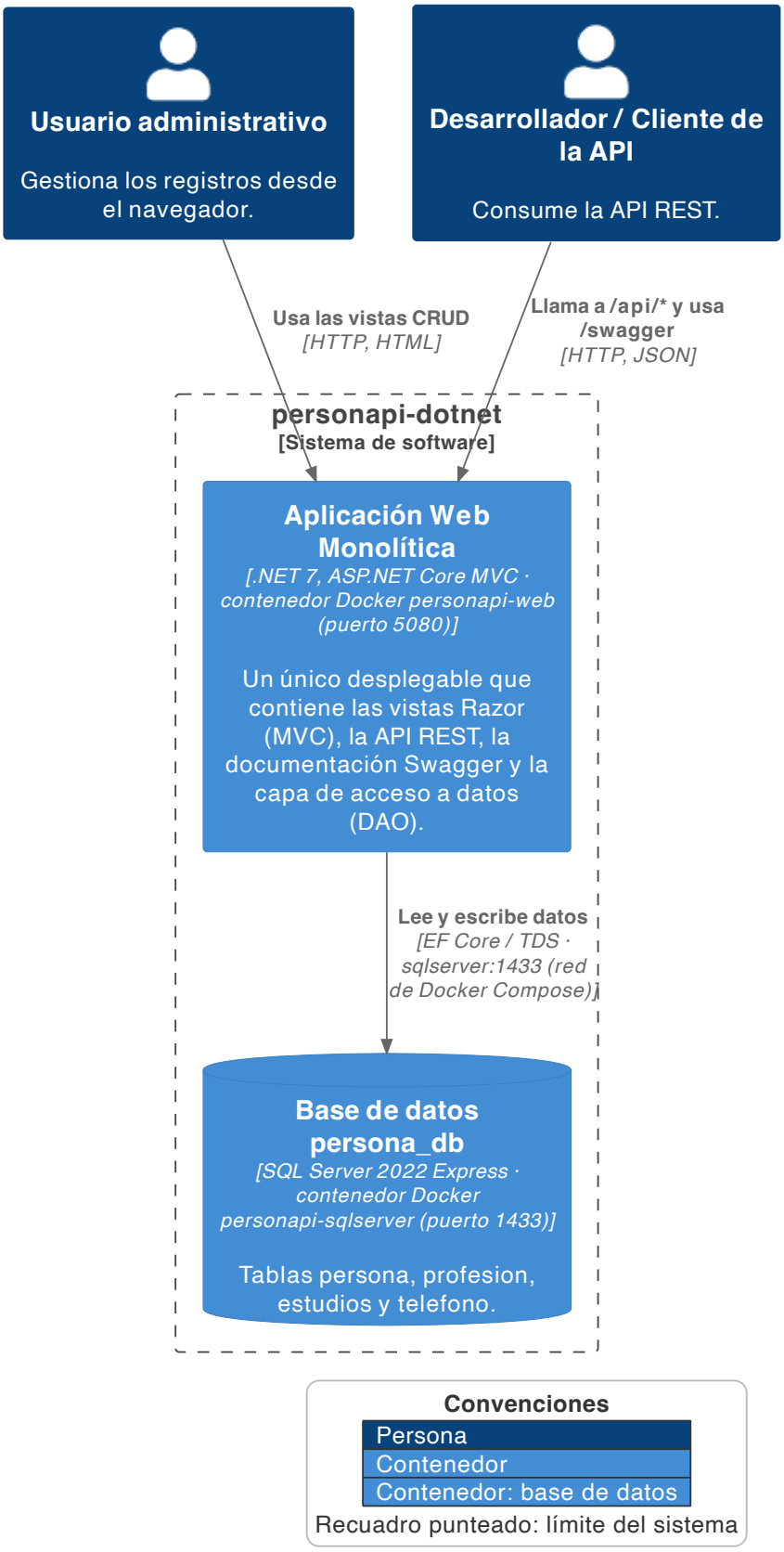
Nivel 1, contexto. Este diagrama es para entender el alcance: qué personas y sistemas externos interactúan con el monolito. Se muestra un usuario administrativo que gestiona personas, profesiones, estudios y teléfonos desde el navegador, y un desarrollador o cliente de la API que la consume con Swagger, `curl` o Postman. El sistema aparece como una sola caja, sin preguntar cómo está construido por dentro.

Nivel 1 - Diagrama de Contexto del Sistema: personapi-dotnet



Nivel 2, contenedores. Este diagrama es para entender las partes desplegables. Dentro del sistema aparece la aplicación web monolítica (.NET 7, ASP.NET Core MVC, Kestrel en el puerto 5080), que es un único ejecutable con las vistas Razor, la API REST y la documentación Swagger. Aquí se ve que el monolito se despliega como una sola unidad, y que la base de datos SQL Server está fuera de él.

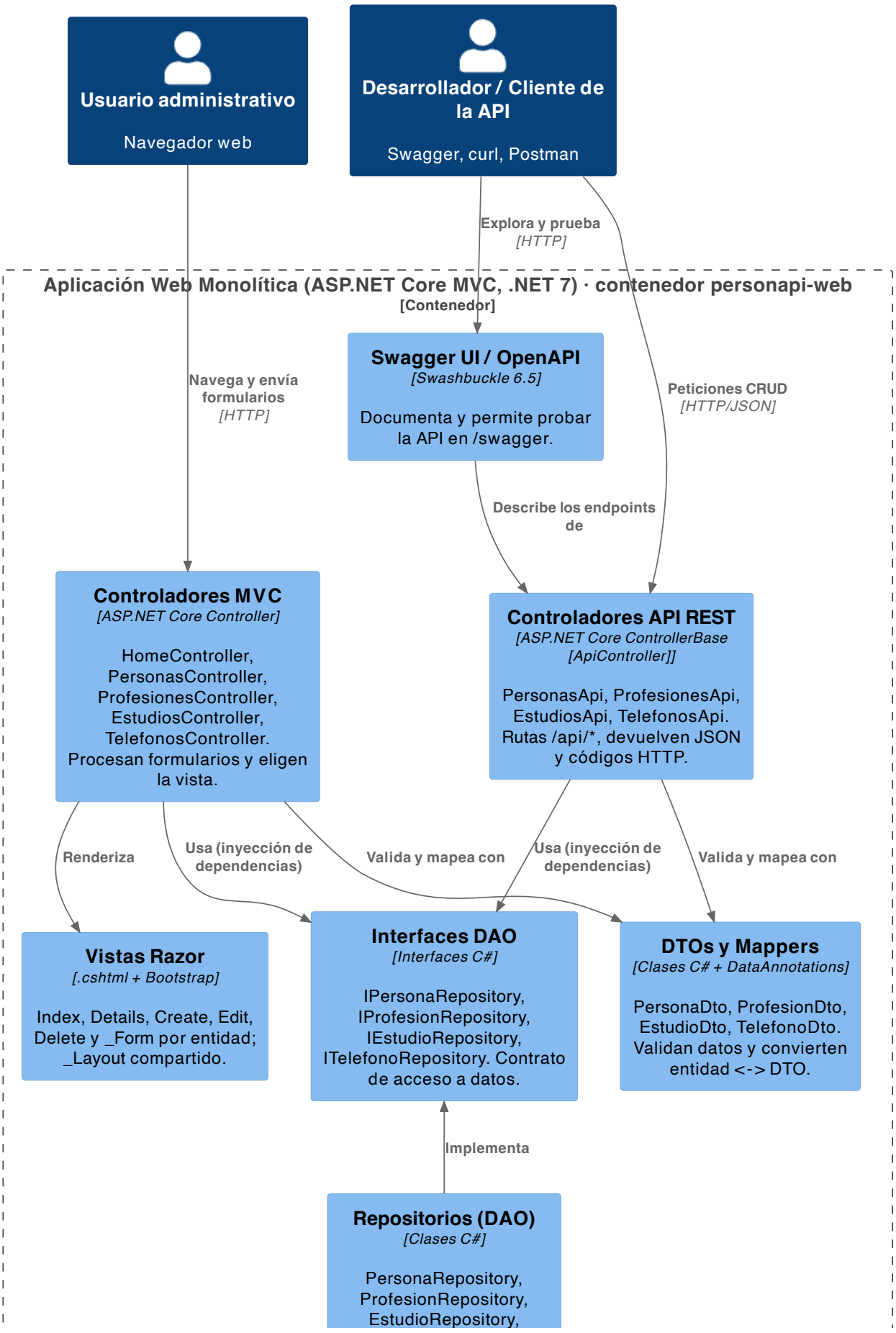
Nivel 2 - Diagrama de Contenedores: personapi-dotnet

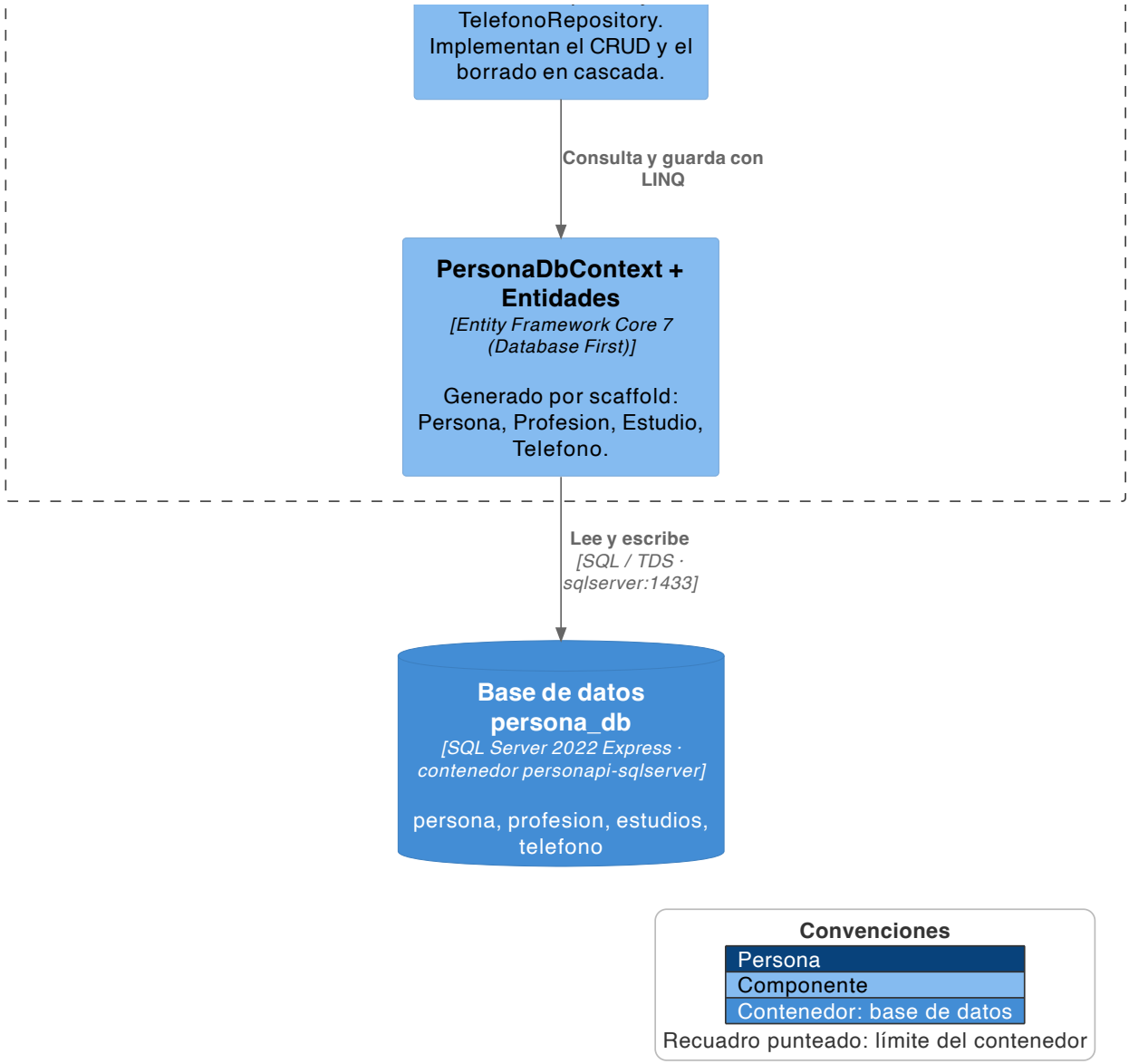


Nivel 3, componentes. Este diagrama es para entender la organización interna del monolito. Se muestran los controladores MVC (`HomeController`, `PersonasController`, `ProfesionesController`,

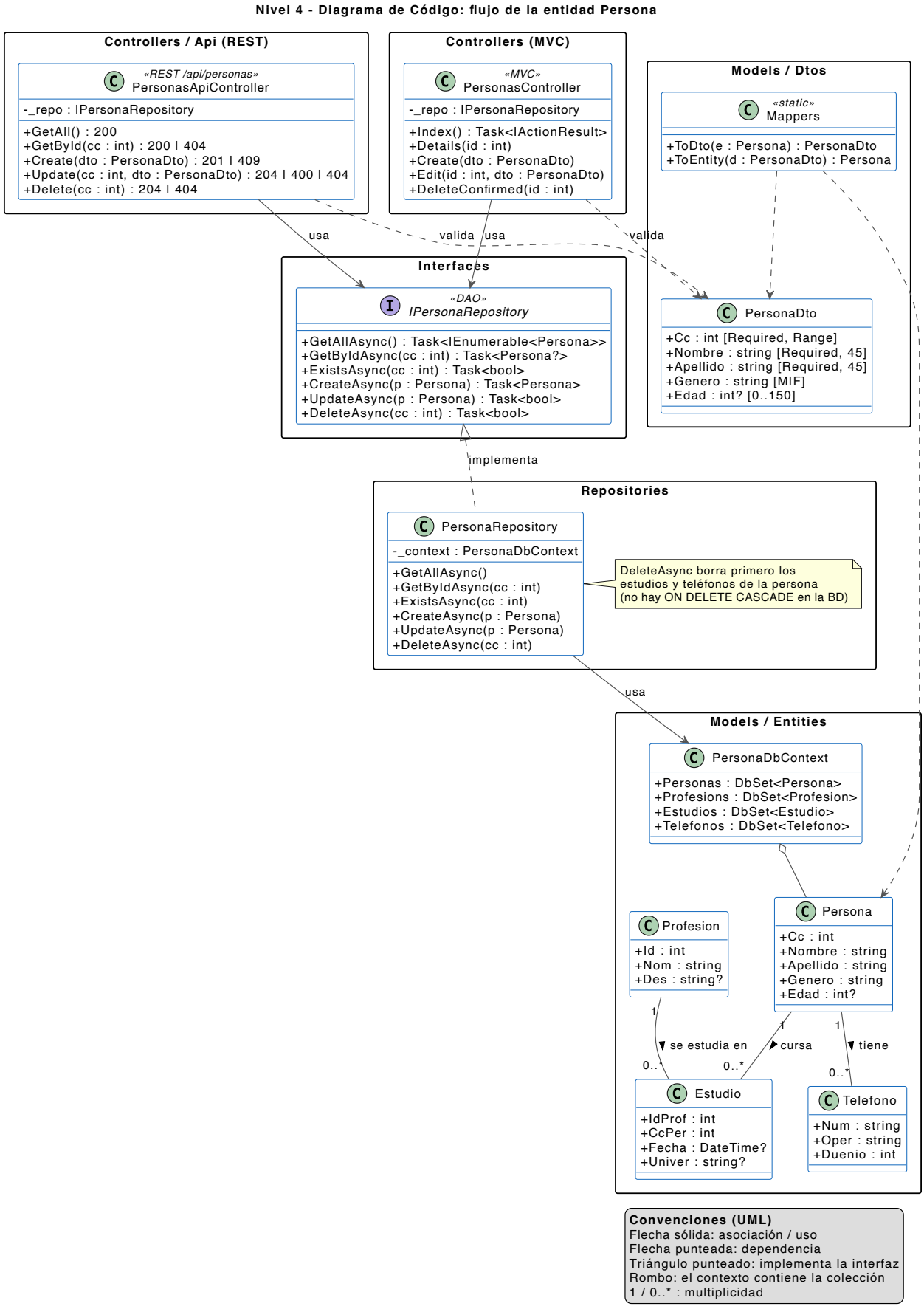
`EstudiosController` y `TelefonosController`), las vistas Razor, y la capa DAO con sus interfaces y repositorios. Es la vista que justifica la separación en capas: los controladores solo conocen interfaces, no EF Core.

Nivel 3 - Diagrama de Componentes: Aplicación Web Monolítica





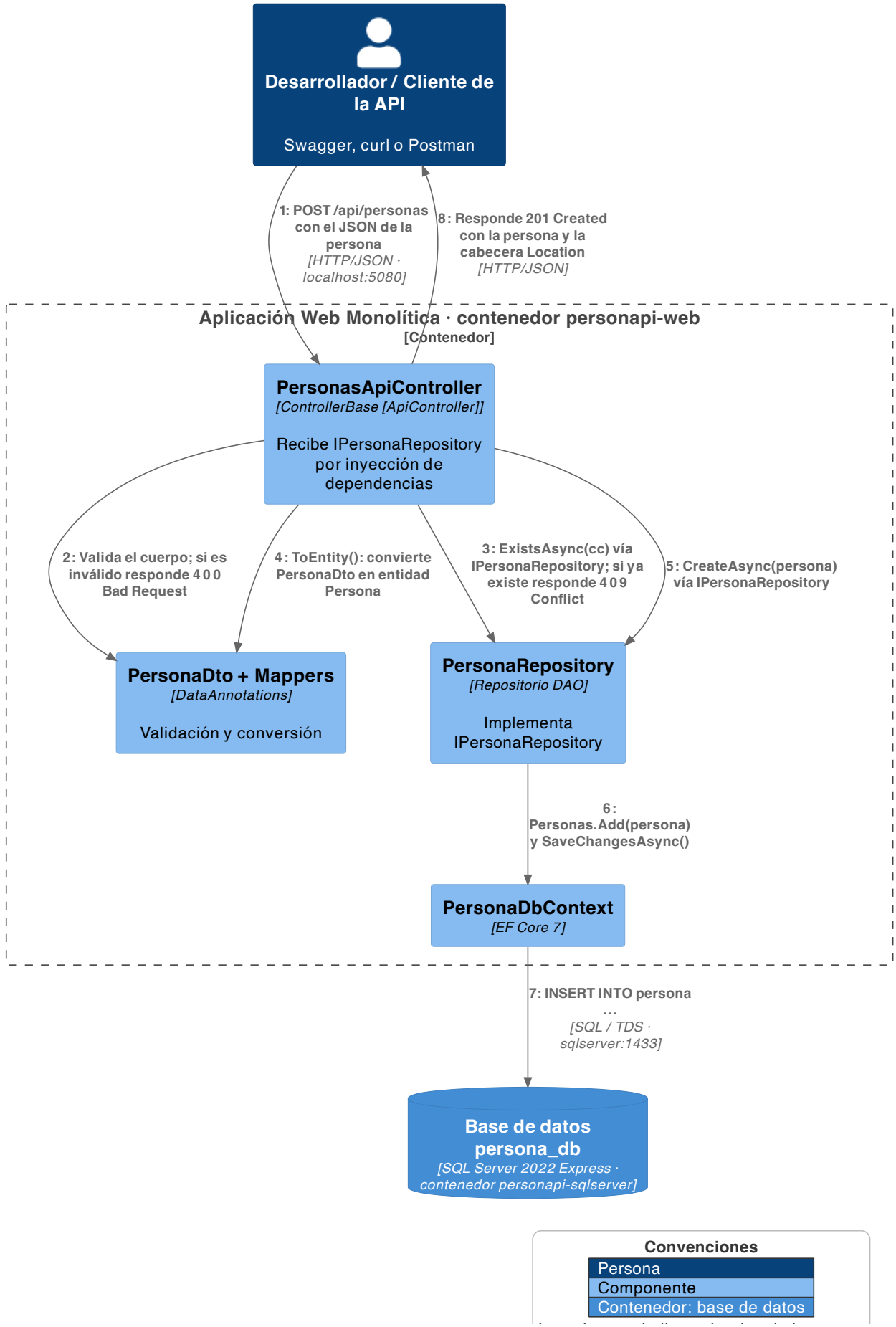
Nivel 4, código. Este diagrama es para mostrar cómo viaja la entidad `Persona` por el código. Aparecen las clases principales (`PersonasController`, `PersonasApiController`, `IPersonaRepository`, `PersonaDto` y las entidades) con sus métodos y los códigos de respuesta de la API (200, 201, 204, 400, 404 y 409). Sirve de referencia para revisar la implementación sin abrir todos los archivos.



Dinámico: crear persona por la API. Este diagrama es para seguir una petición real. Muestra el recorrido de un `POST /api/personas`: el `PersonasApiController` recibe el `PersonaDto`, lo valida con

DataAnnotations y lo convierte, llama a `IPersonaRepository`, y el repositorio usa `PersonaDbContext` de EF Core para guardar la fila. Esta es la vista para entender qué pasa entre el cliente y la base de datos.

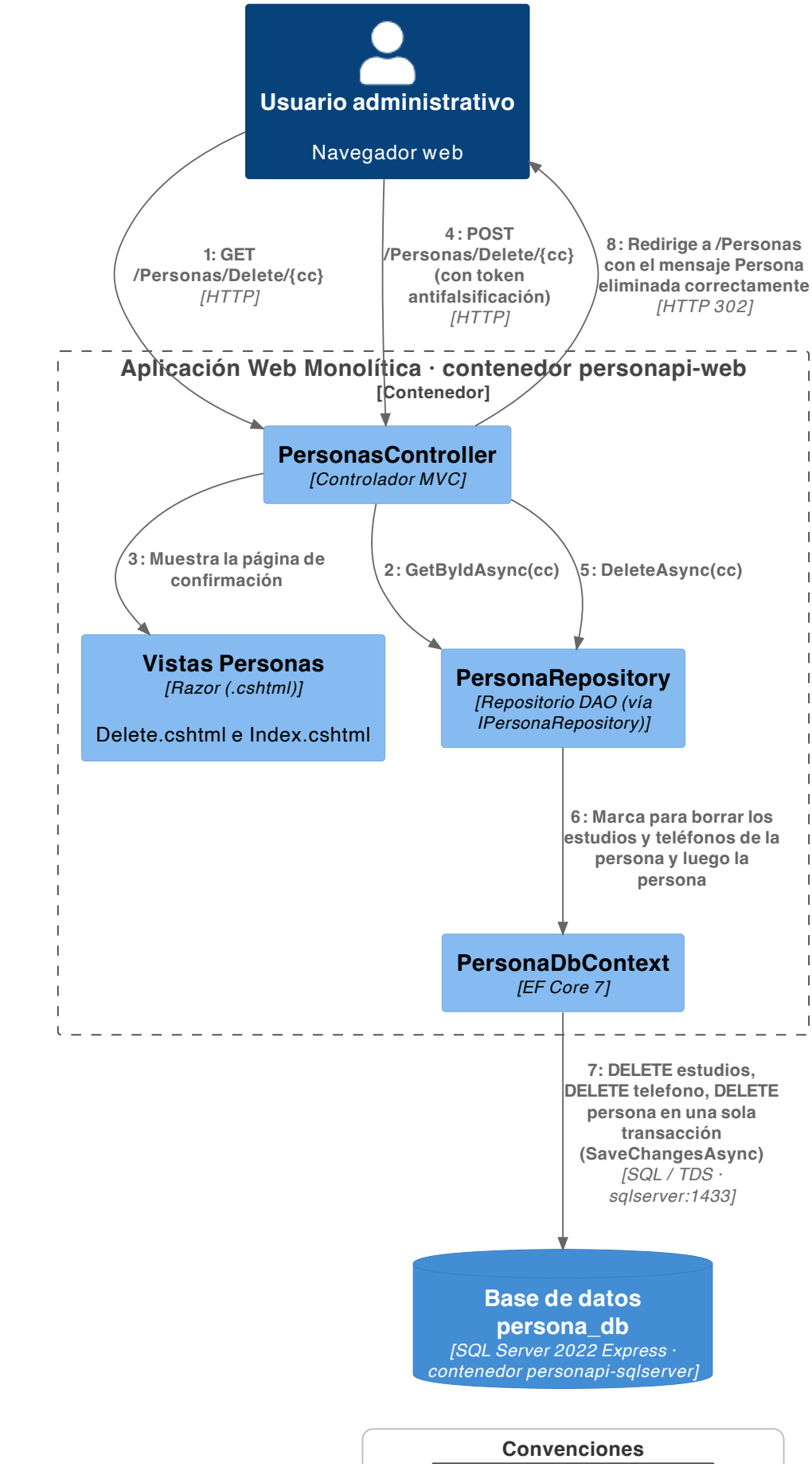
Diagrama Dinámico: crear una persona por la API REST (POST /api/personas)

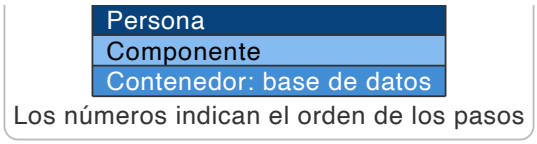


Los números indican el orden de los pasos

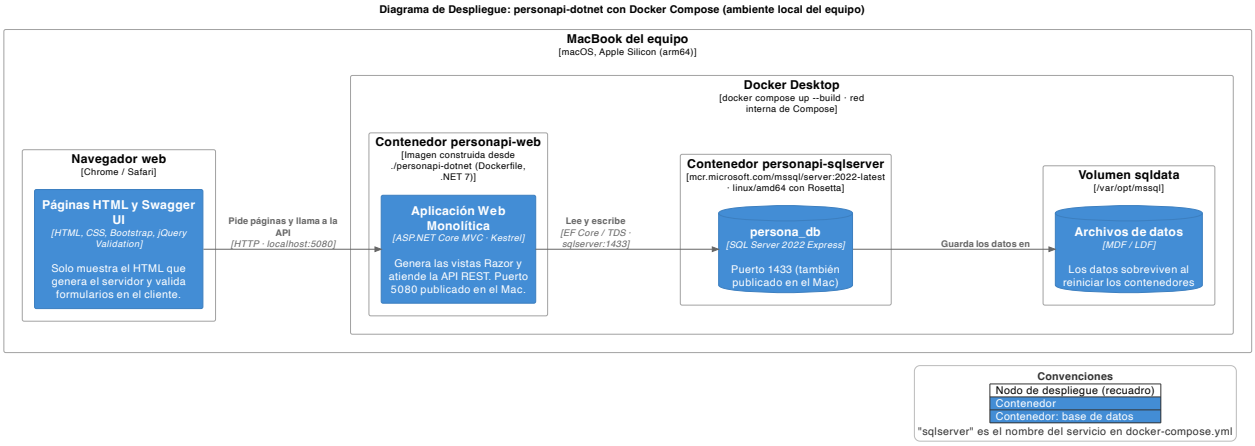
Dinámico: eliminar persona desde MVC (borrado en cascada). Este diagrama es para explicar la regla de negocio del borrado. Muestra cómo el `PersonasController` de las vistas MVC elimina una persona, y cómo el repositorio borra antes sus estudios y teléfonos, porque las claves foráneas no tienen `ON DELETE CASCADE`. Es la parte del diseño que más conviene explicar en la sustentación.

Diagrama Dinámico: eliminar una persona desde las vistas MVC (borrado en cascada)





Despliegue. Este diagrama es para mostrar el ambiente real del laboratorio. Todo corre en un MacBook con Apple Silicon: el navegador, el runtime de .NET 7 con Kestrel y Docker Desktop con Rosetta, que ejecuta el contenedor `mcr.microsoft.com/mssql/server:2022-latest` con el volumen `sqldata`. Explica por qué la imagen de SQL Server corre con emulación.



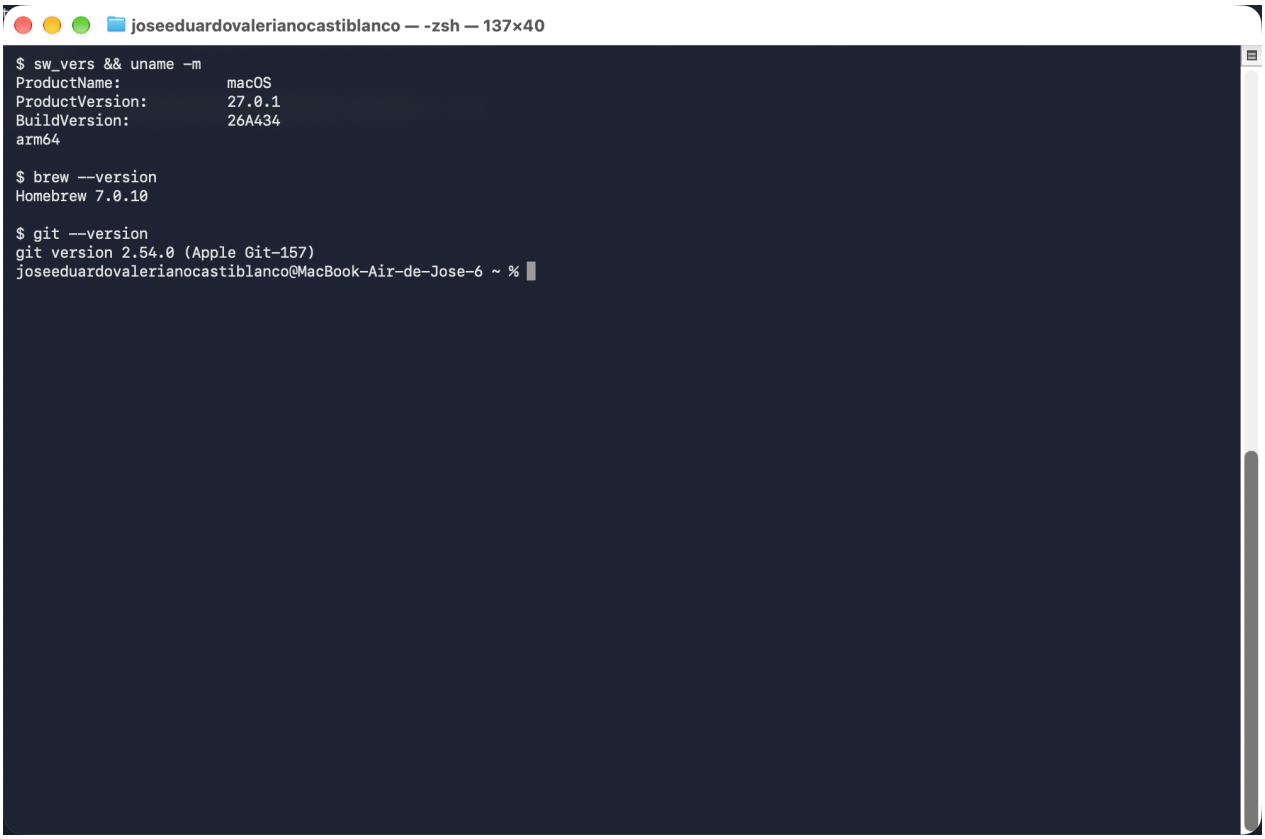
---

3. Procedimiento

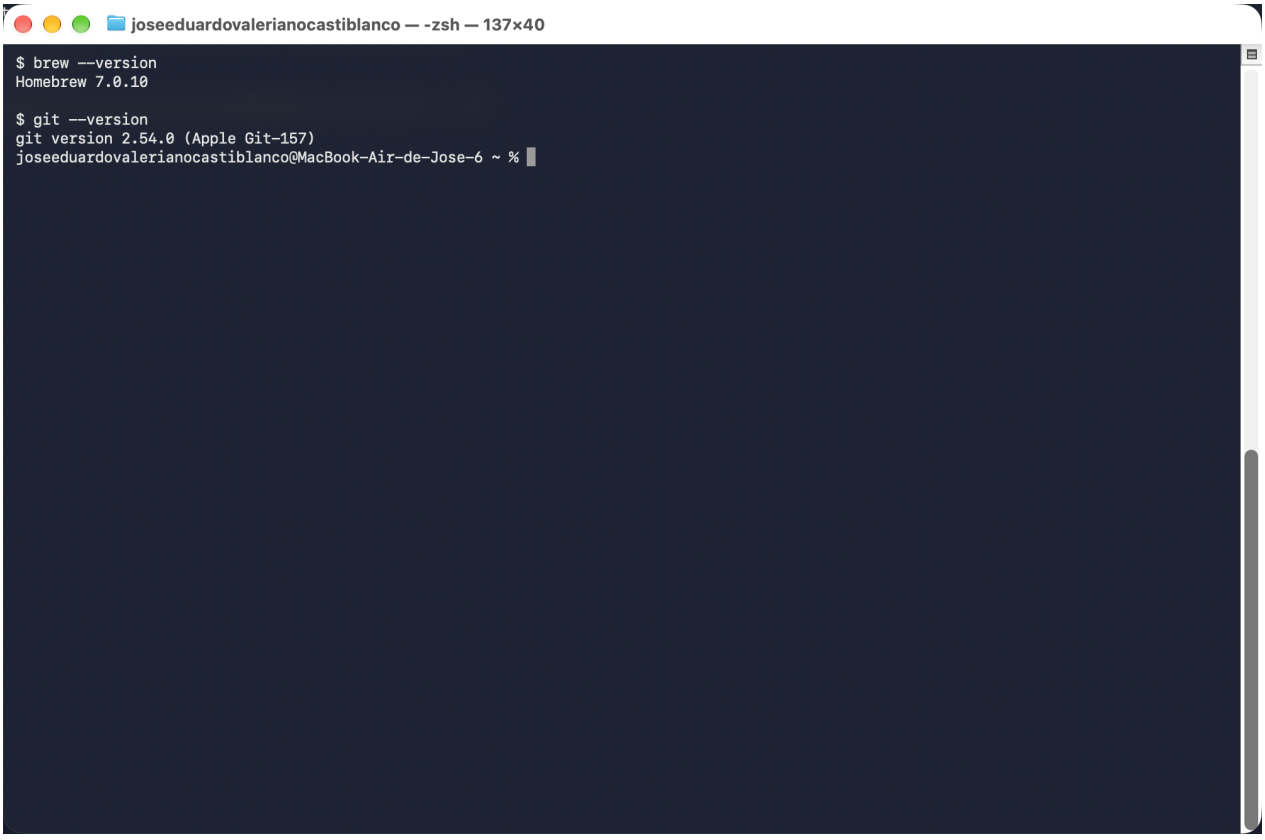
Trabajamos en macOS con chip Apple Silicon. La guía del profesor está pensada para Windows, así que cada herramienta se reemplazó por su equivalente en Mac: Docker Desktop en lugar de SQL Server Express instalado, VS Code con las extensiones C# Dev Kit y SQL Server (mssql) en lugar de Visual Studio y SSMS, y `dotnet` y `dotnet ef` en Terminal en lugar de las plantillas y la Consola del Administrador de paquetes.

3.1 Ambiente

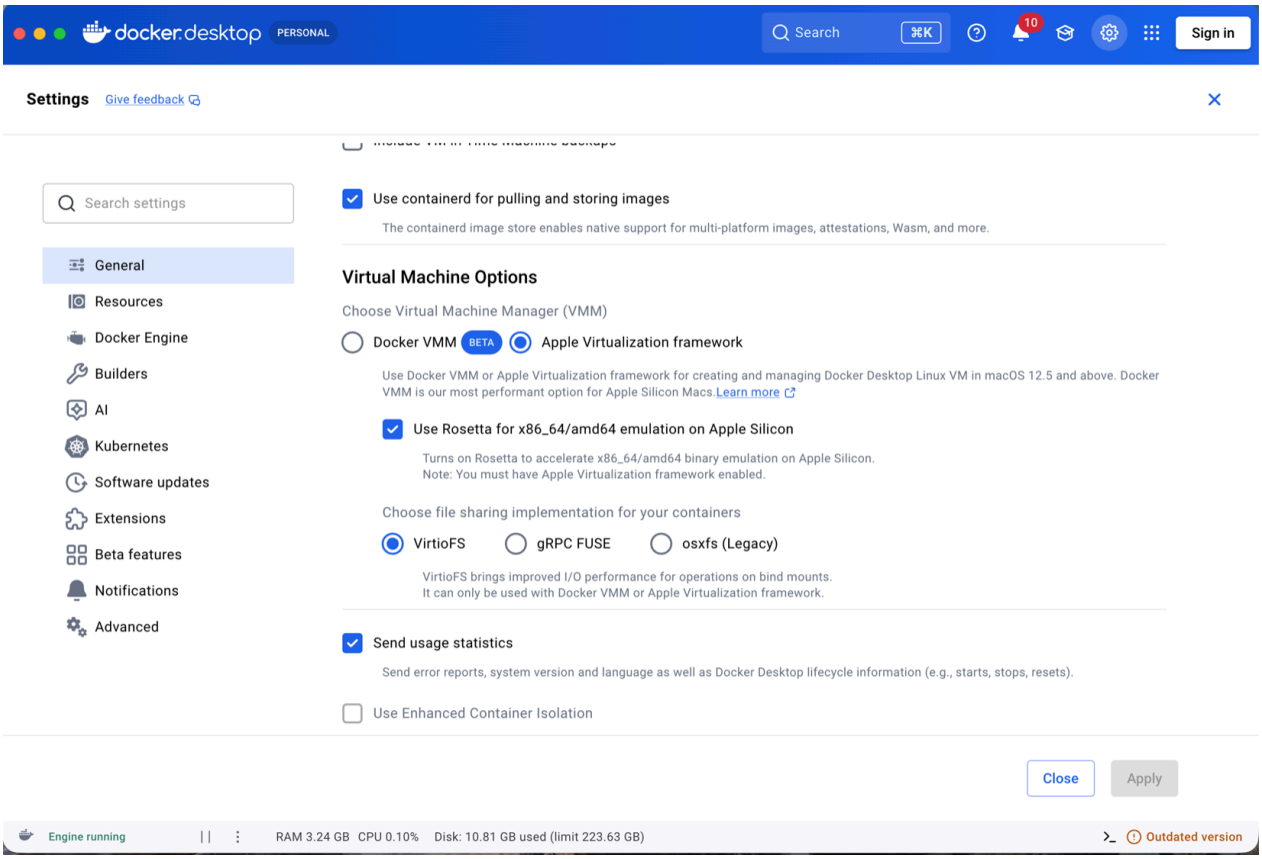
Paso 1. Sistema. macOS 27.0.1 y arquitectura `arm64`.



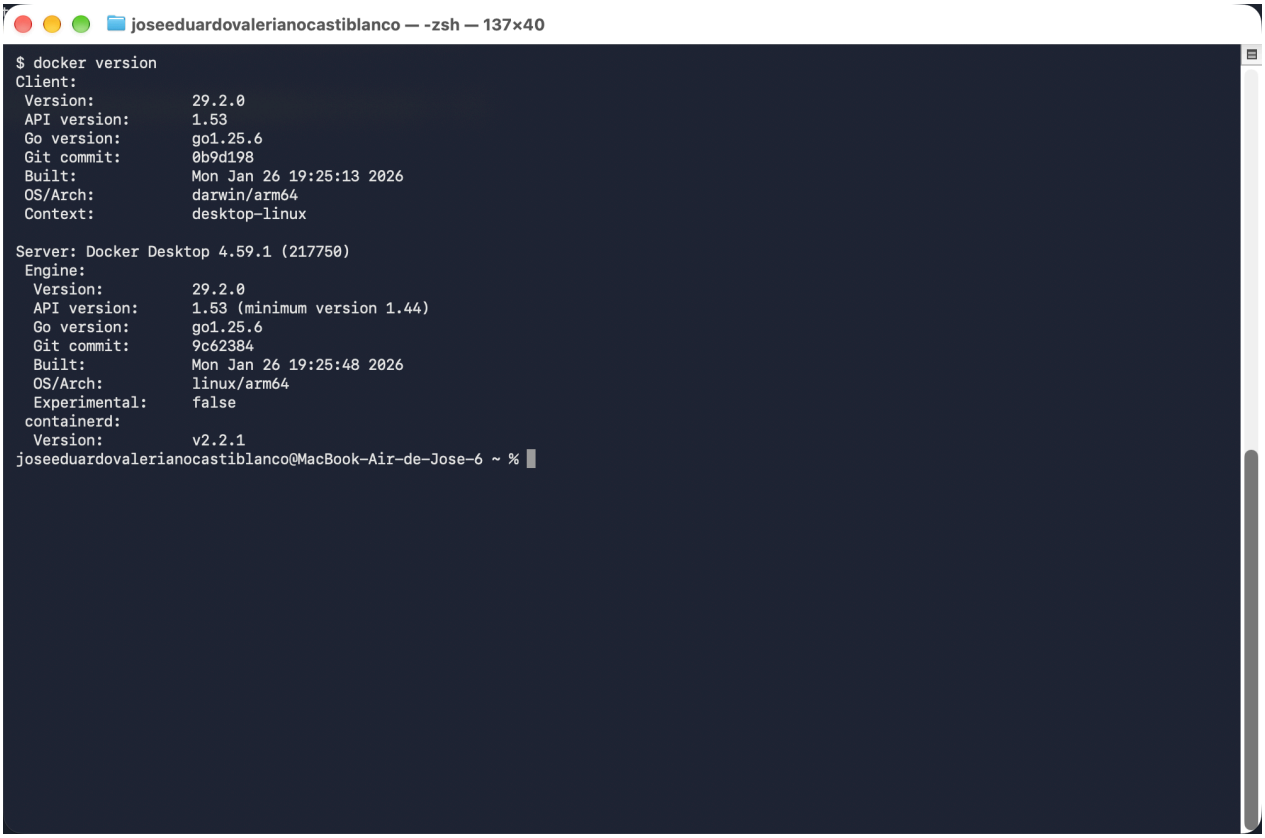
Paso 2. Homebrew y Git. Homebrew 7.0.10 y Git 2.54.0.



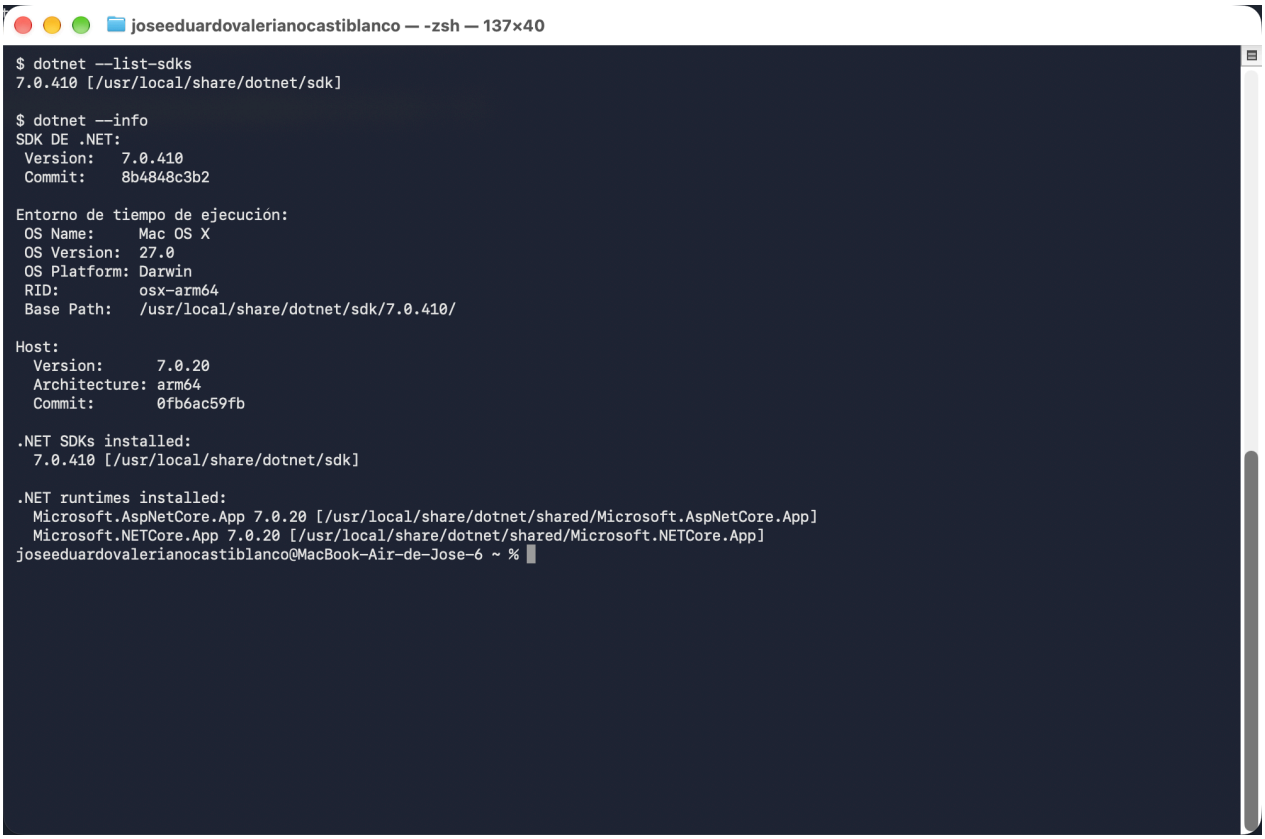
Paso 3. Docker Desktop con Rosetta. Se activó *Use Rosetta for x86_64/amd64 emulation on Apple Silicon* en Settings › General › Virtual Machine Options, necesario porque la imagen de SQL Server es `amd64`.



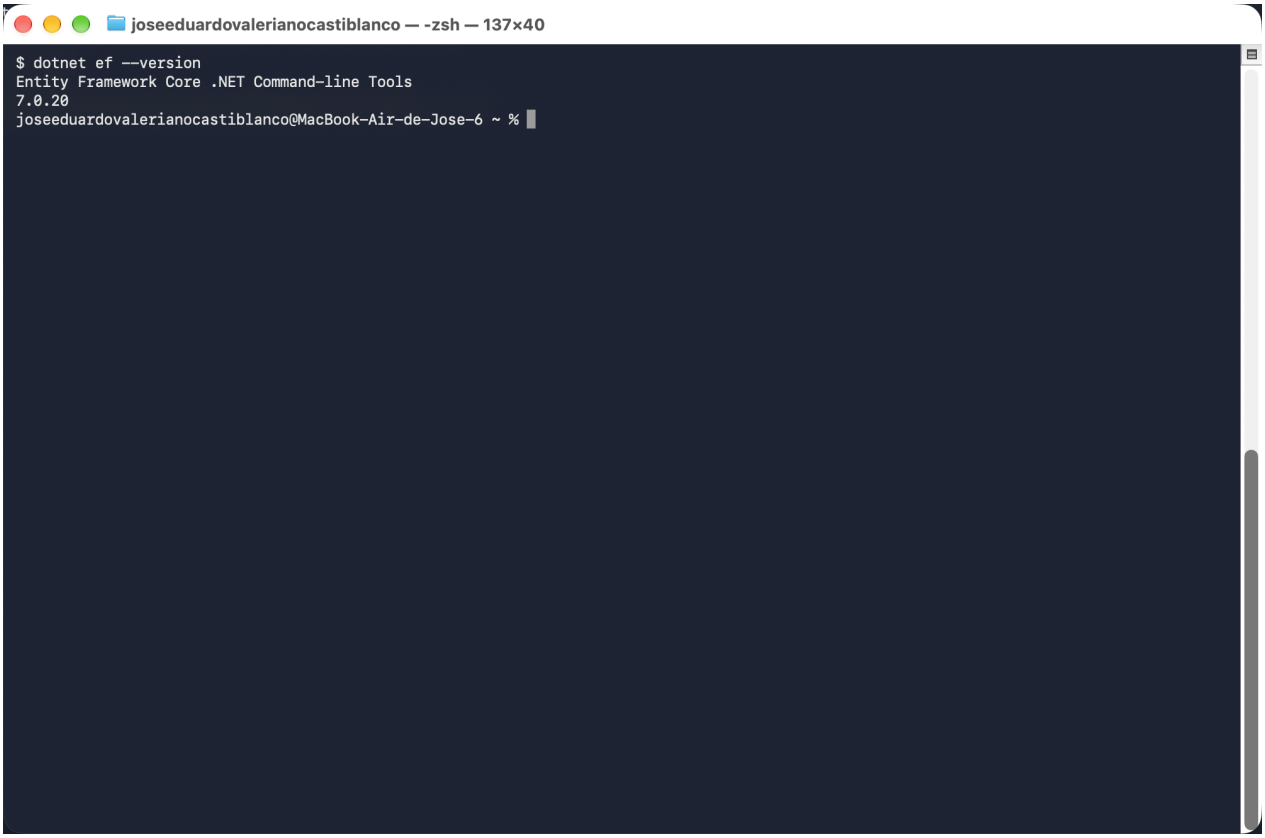
Paso 4. Docker. Versión del cliente y del servidor.



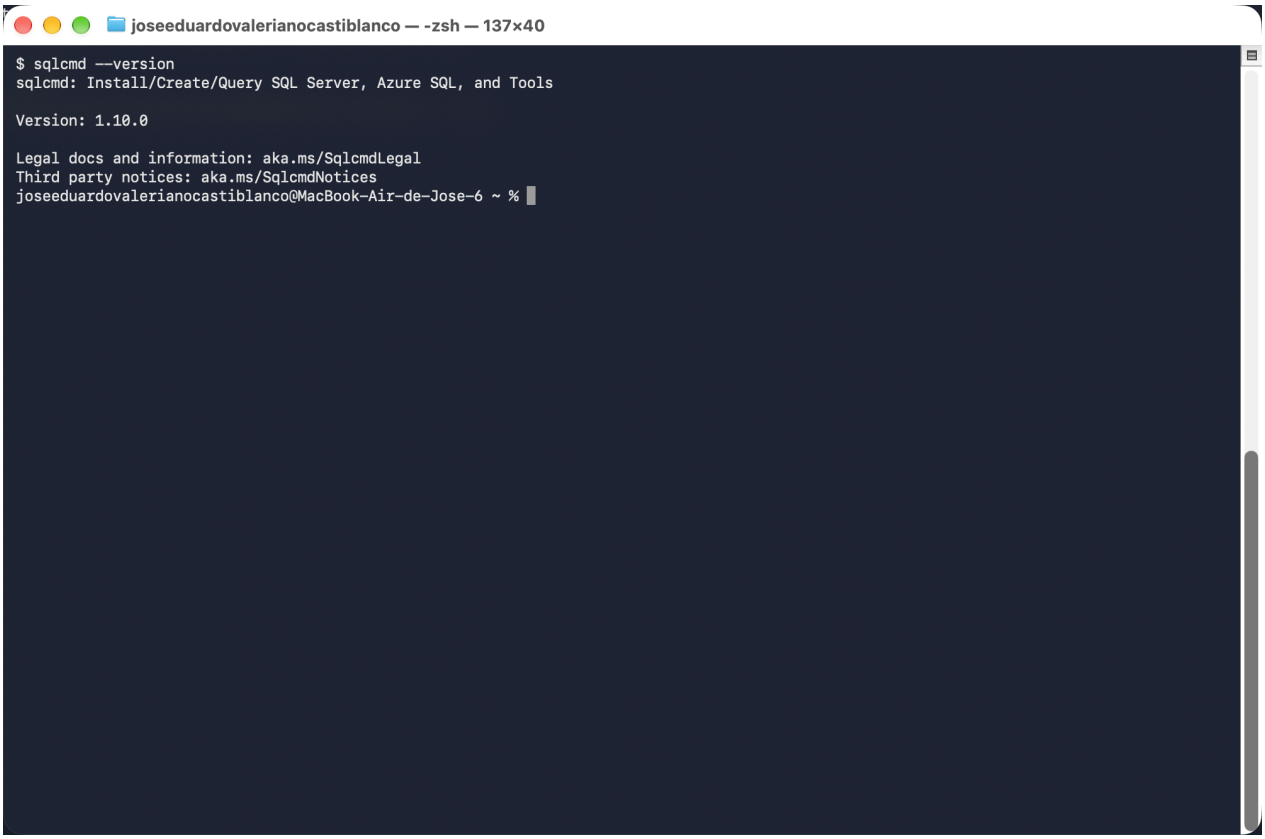
Paso 5. SDK de .NET 7. Se instaló el paquete oficial `dotnet-sdk-7.0.410-osx-arm64.pkg`.



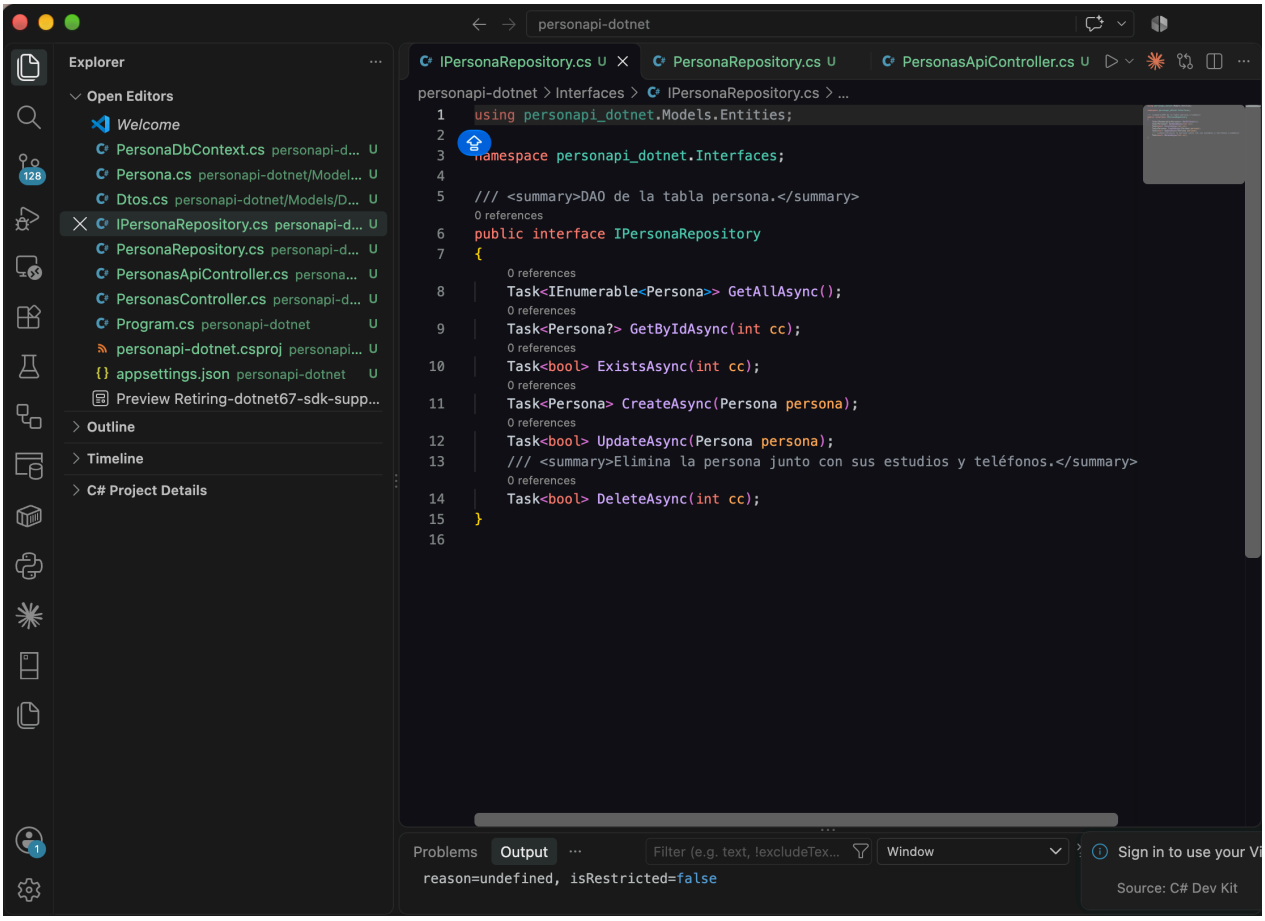
Paso 6. Herramienta EF Core. `dotnet tool install --global dotnet-ef --version 7.0.20`.



Paso 7. sqlcmd. Instalado con Homebrew para consultar SQL Server desde Terminal.

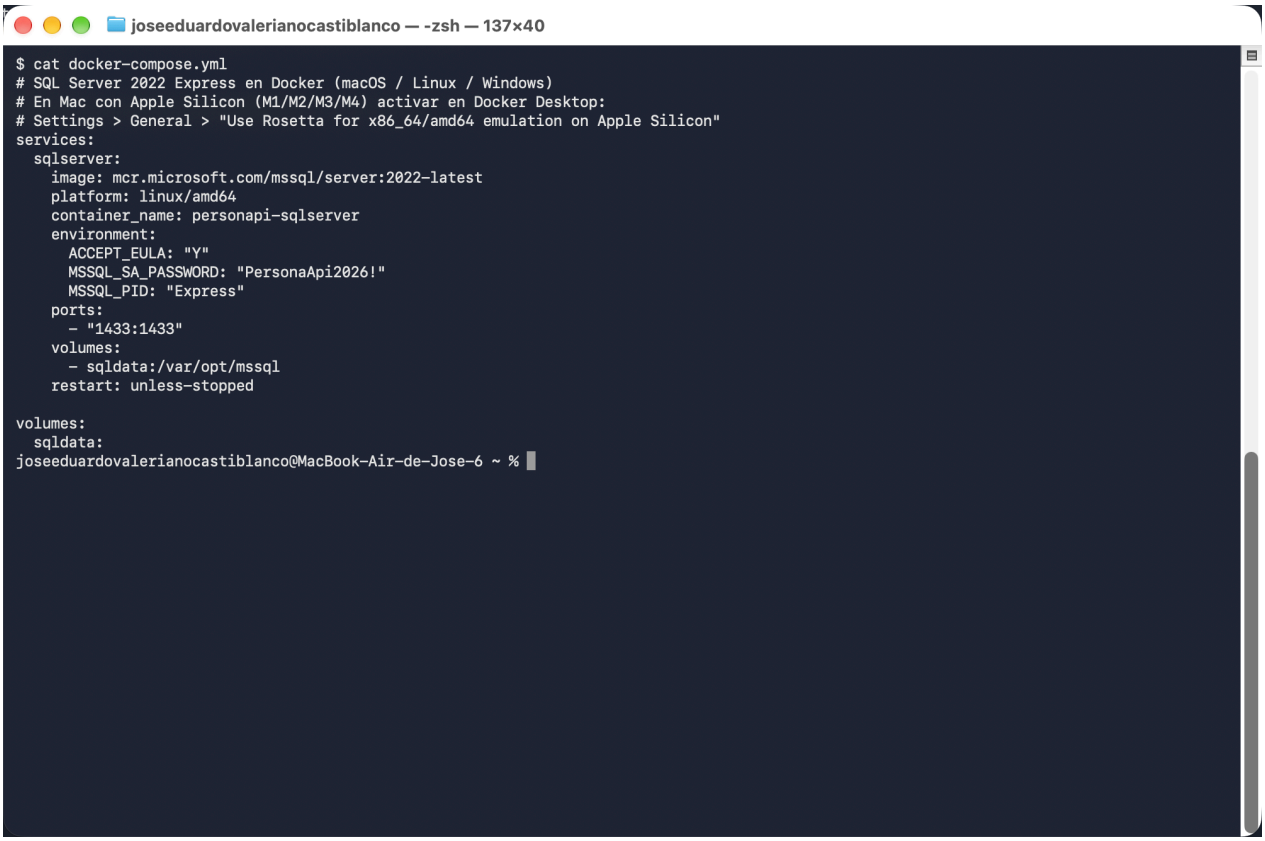


Paso 8. VS Code. Extensiones C# Dev Kit y SQL Server (mssql) instaladas.

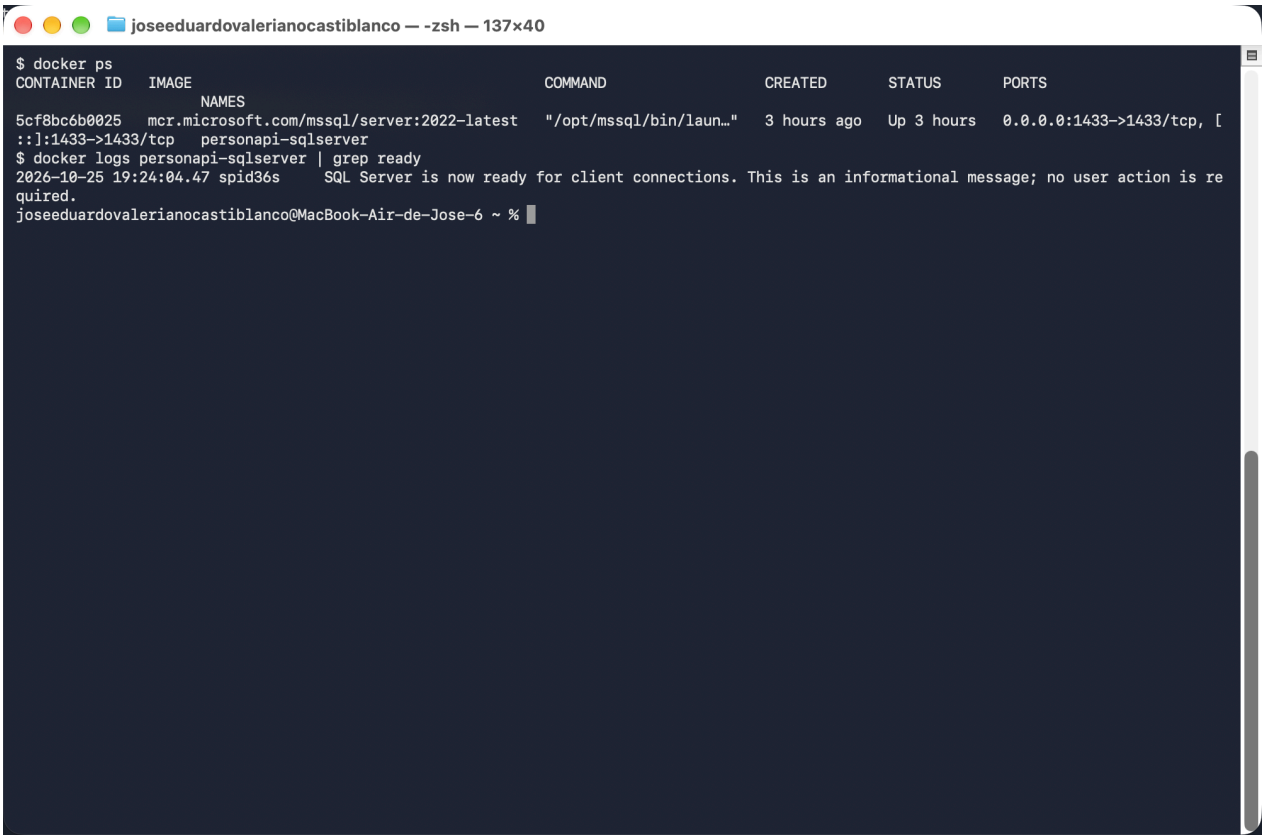


3.2 Base de datos

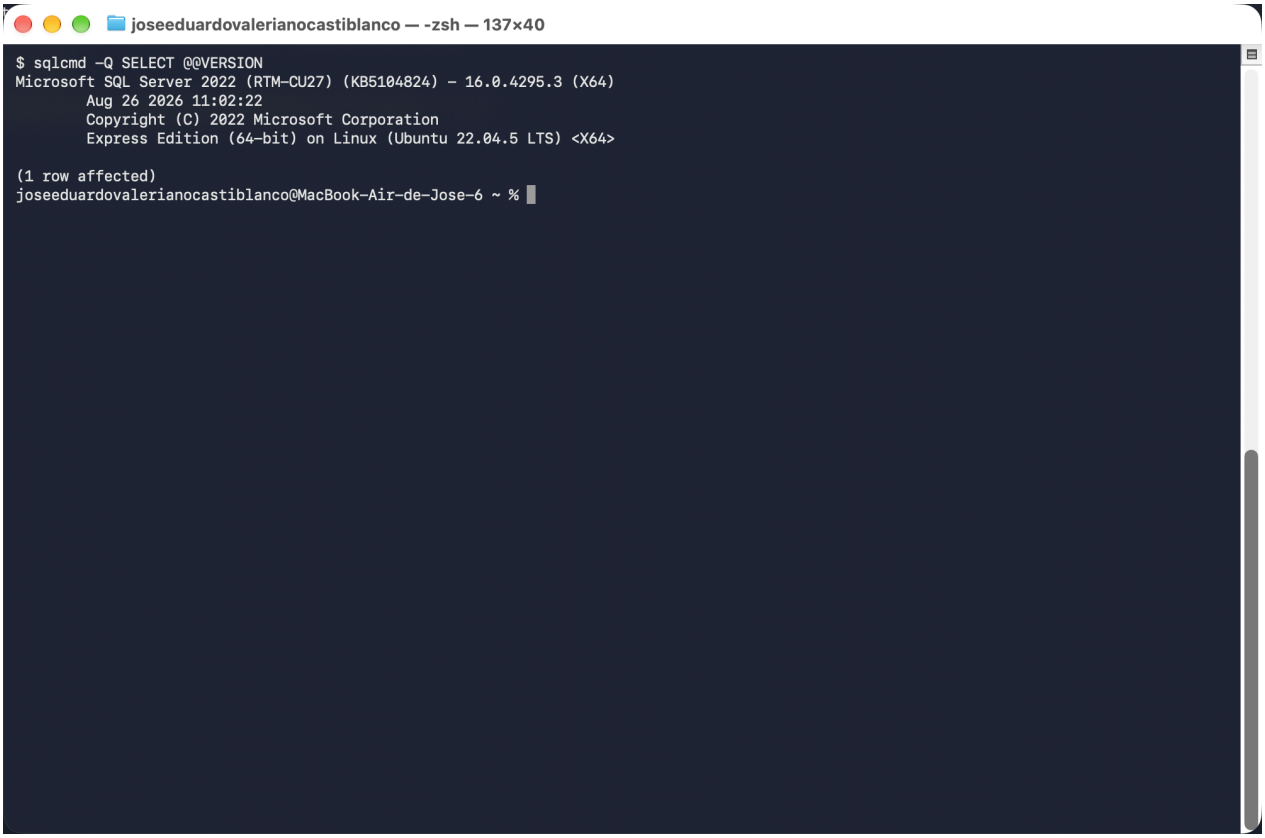
Paso 9. Docker Compose. `docker-compose.yml` define la imagen `mssql/server:2022-latest` con `MSSQL_PID=Express`.



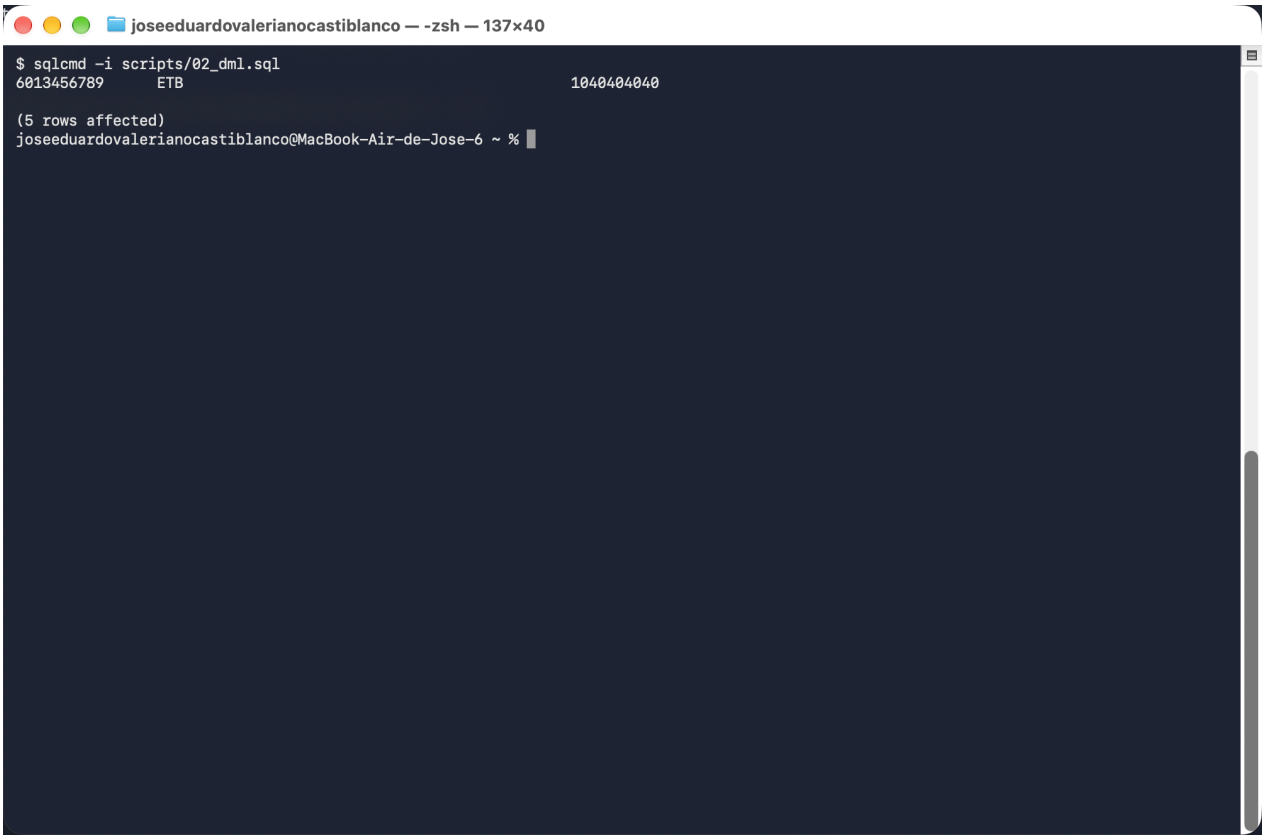
Paso 10. Levantar el contenedor. `docker compose up -d` y `docker ps`. Los logs confirman "SQL Server is now ready for client connections".



Paso 11. Versión del motor. `SELECT @@VERSION` confirma **SQL Server 2022 Express Edition**.

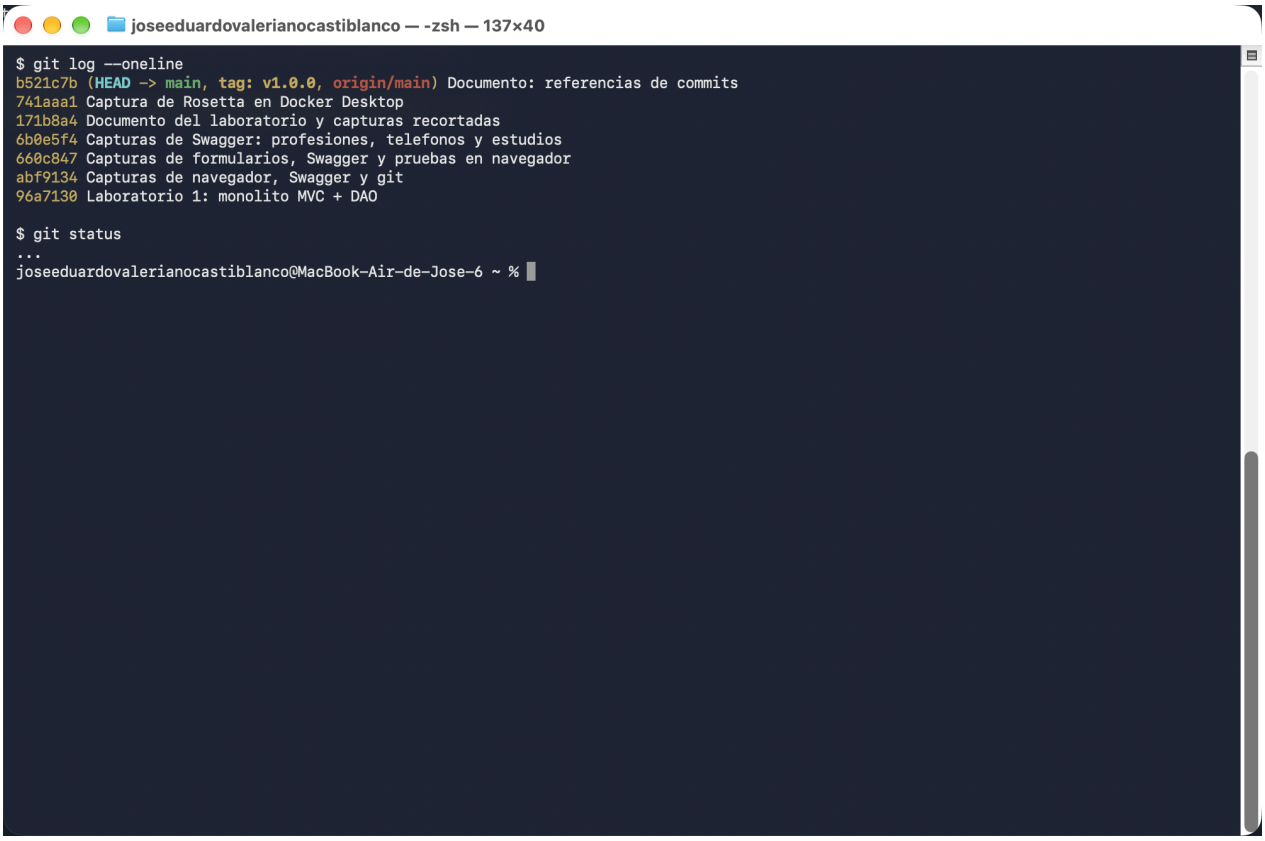


Paso 12. Scripts. Se ejecutaron `01_ddl.sql` (crea `persona_db` y las cuatro tablas) y `02_dml.sql` (datos de prueba). Al final de las pruebas se volvió a ejecutar `02_dml.sql` para restaurar los datos iniciales.

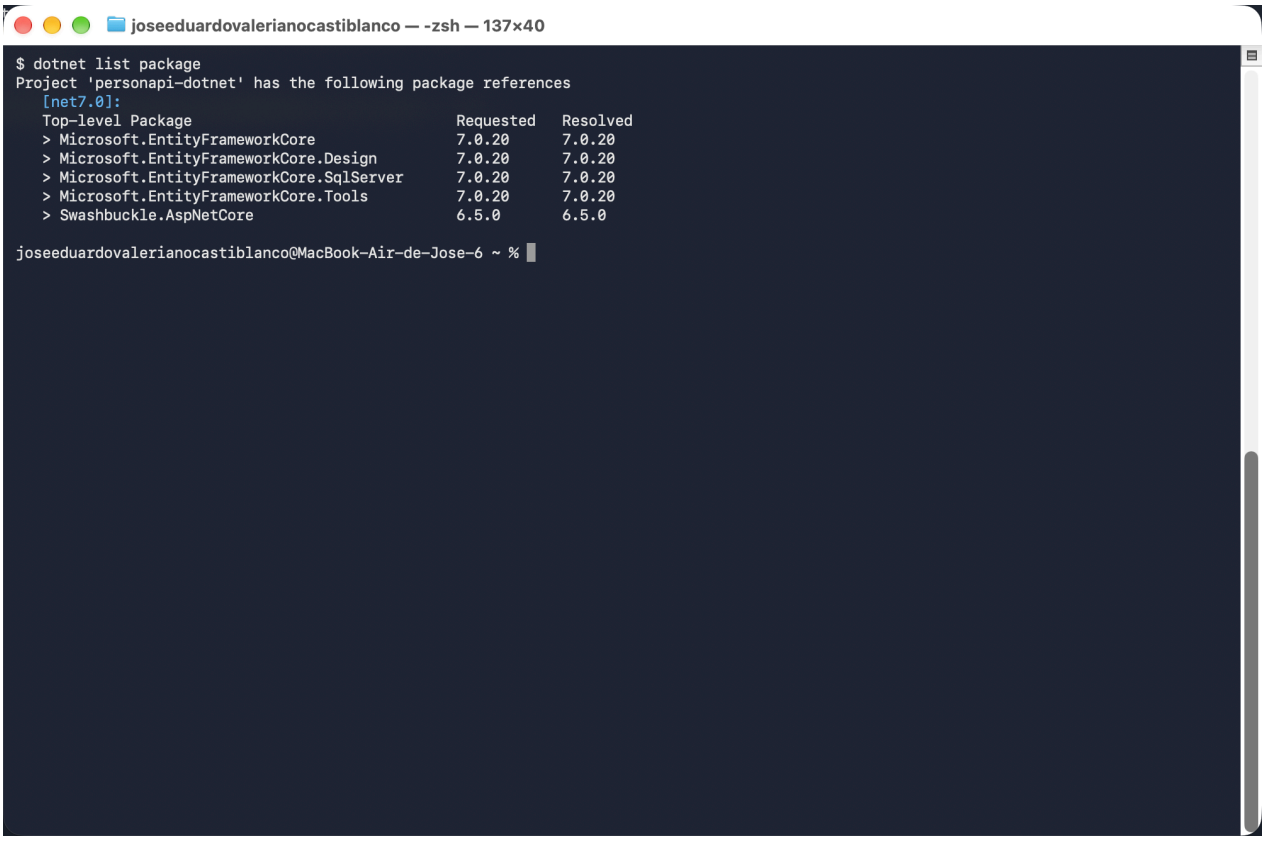


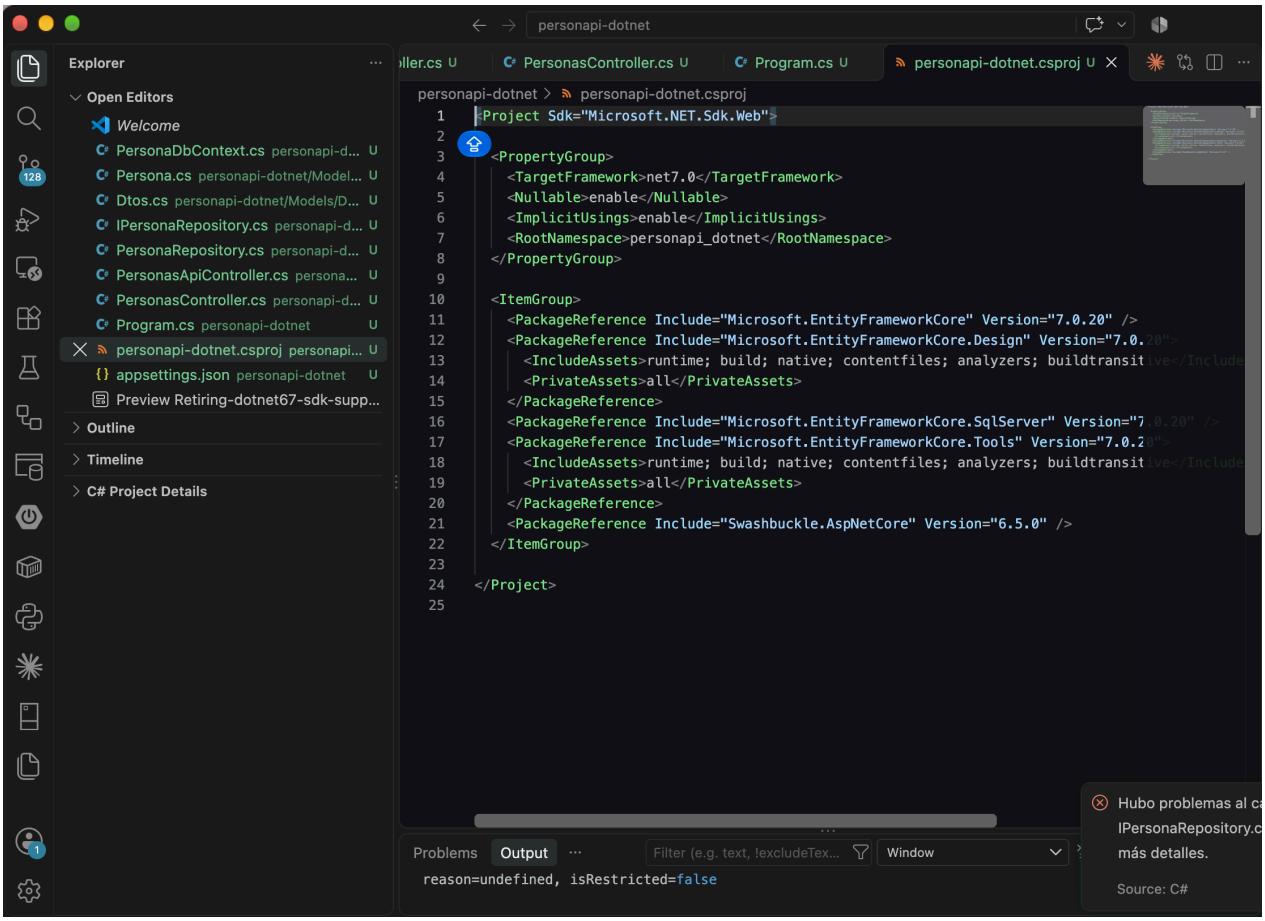
3.3 Creación del proyecto

Paso 13. Repositorio y solución. `git init`, `dotnet new sln` y `dotnet new mvc --framework net7.0 --no-https --auth None`, agregado a la solución.



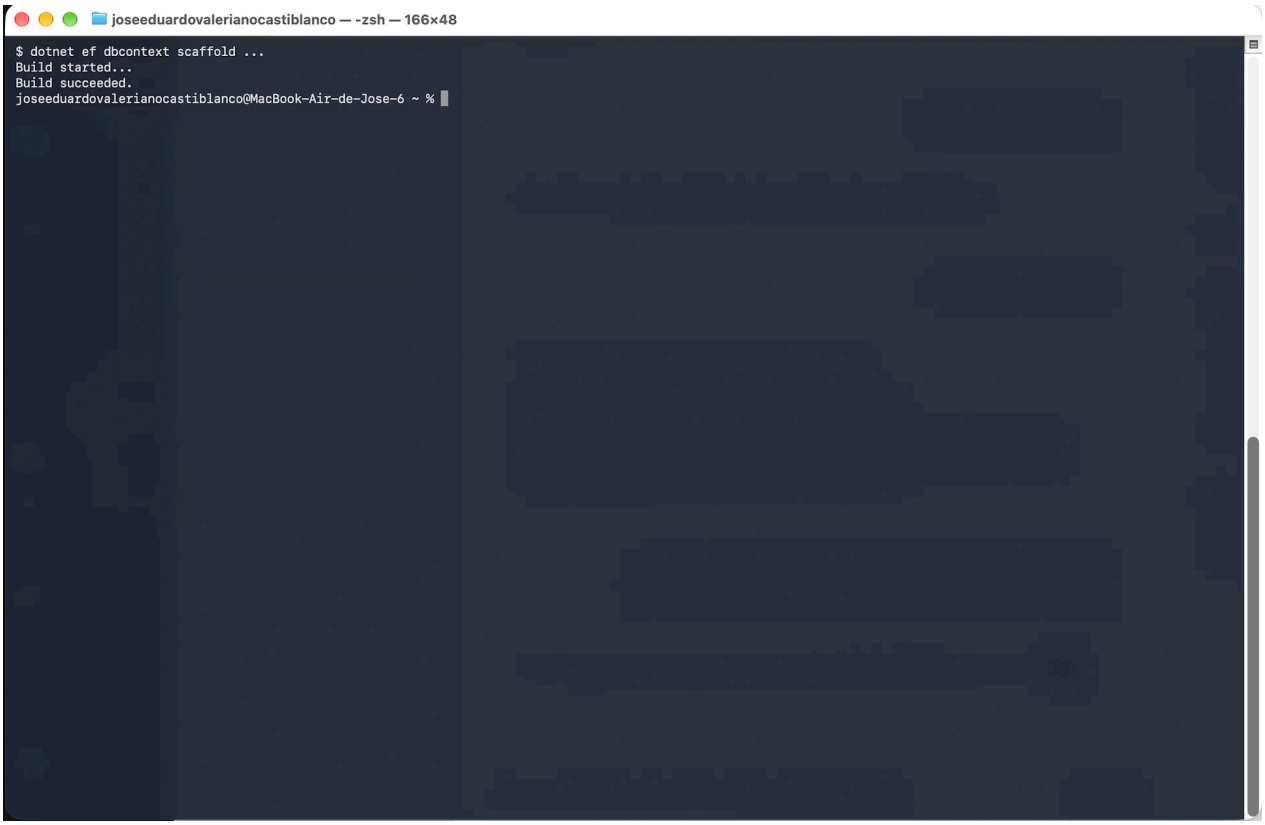
Paso 14. Paquetes NuGet. `Microsoft.EntityFrameworkCore`, `.SqlServer`, `.Tools` y `.Design` 7.0.20, y `Swashbuckle.AspNetCore` 6.5.0.

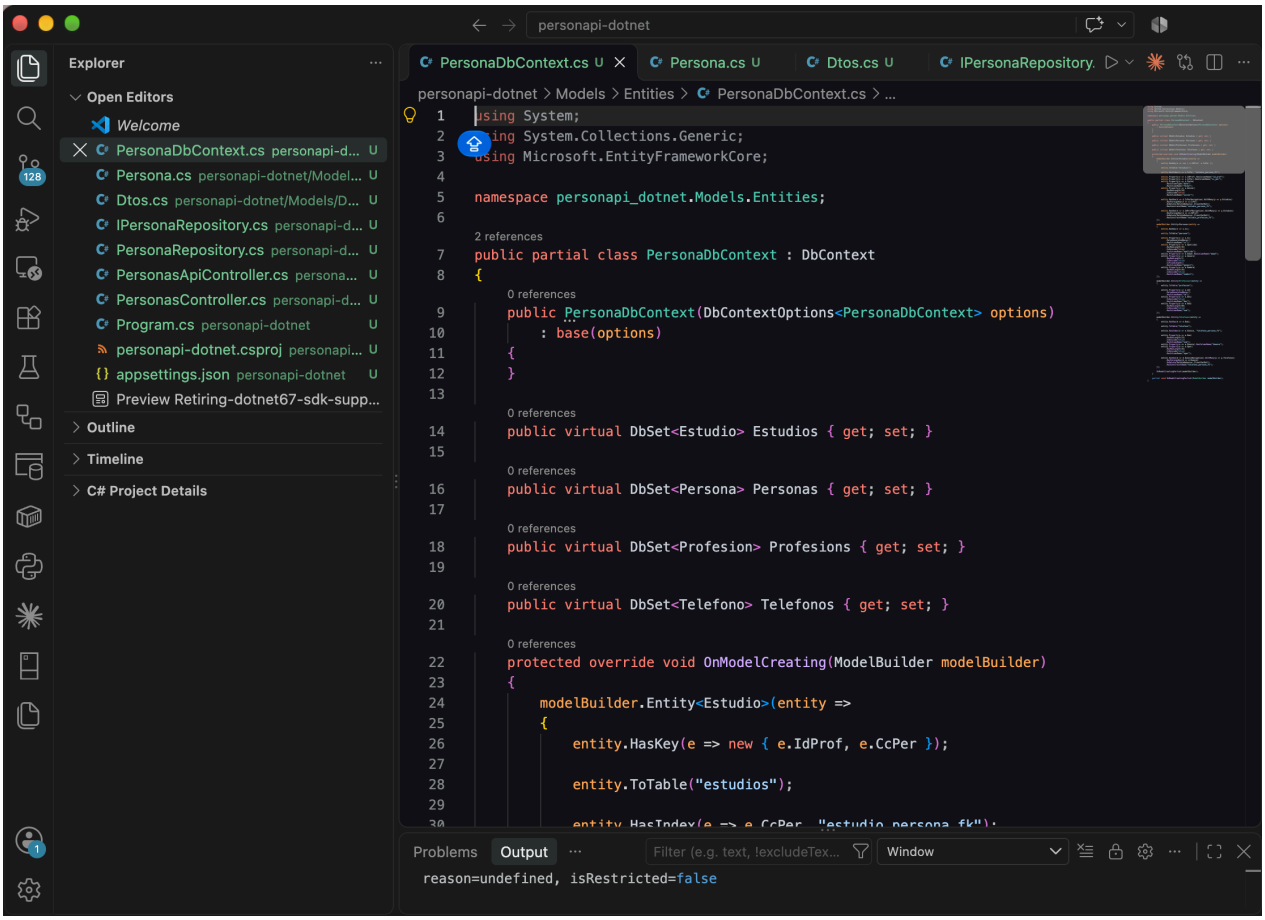


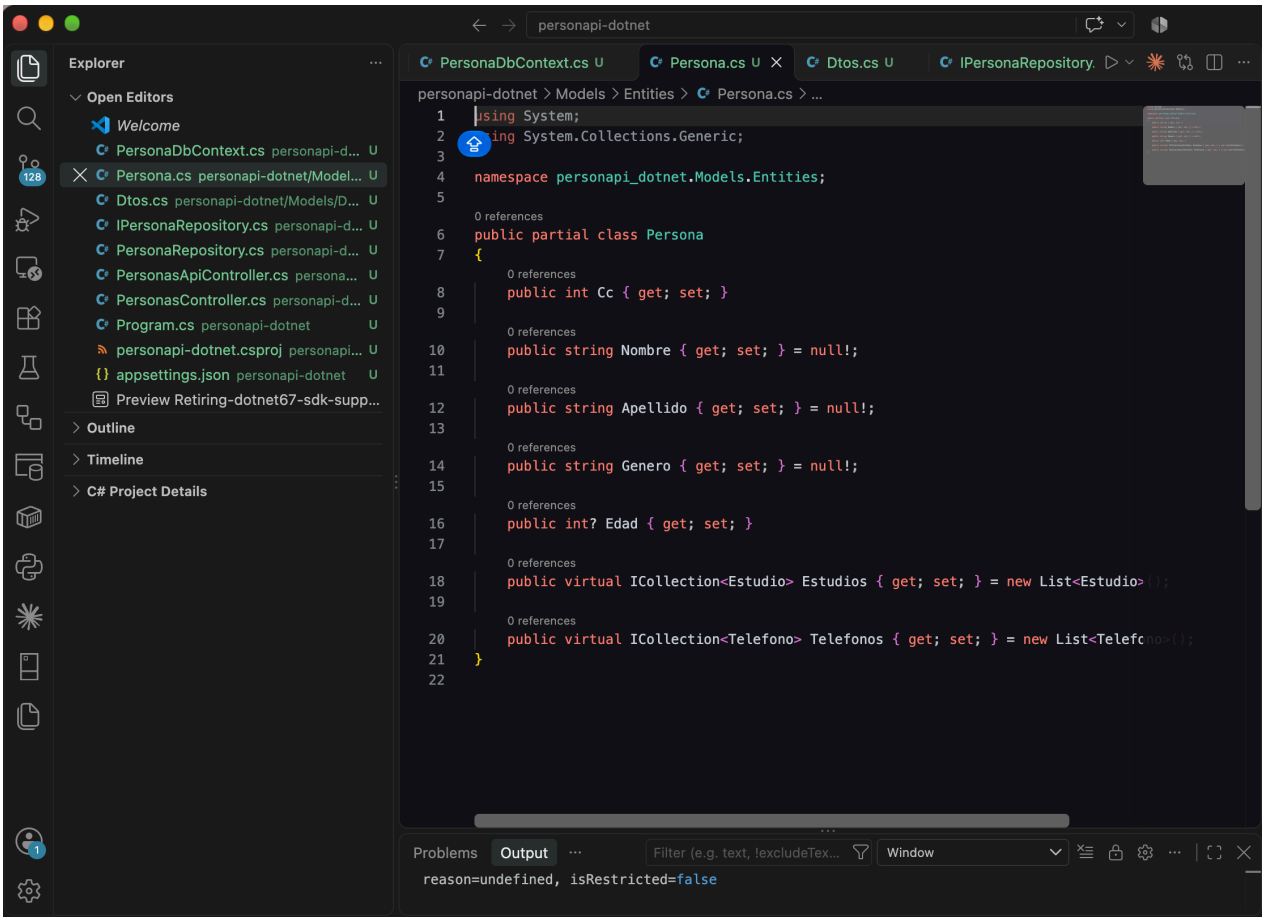


3.4 Entidades (Database First)

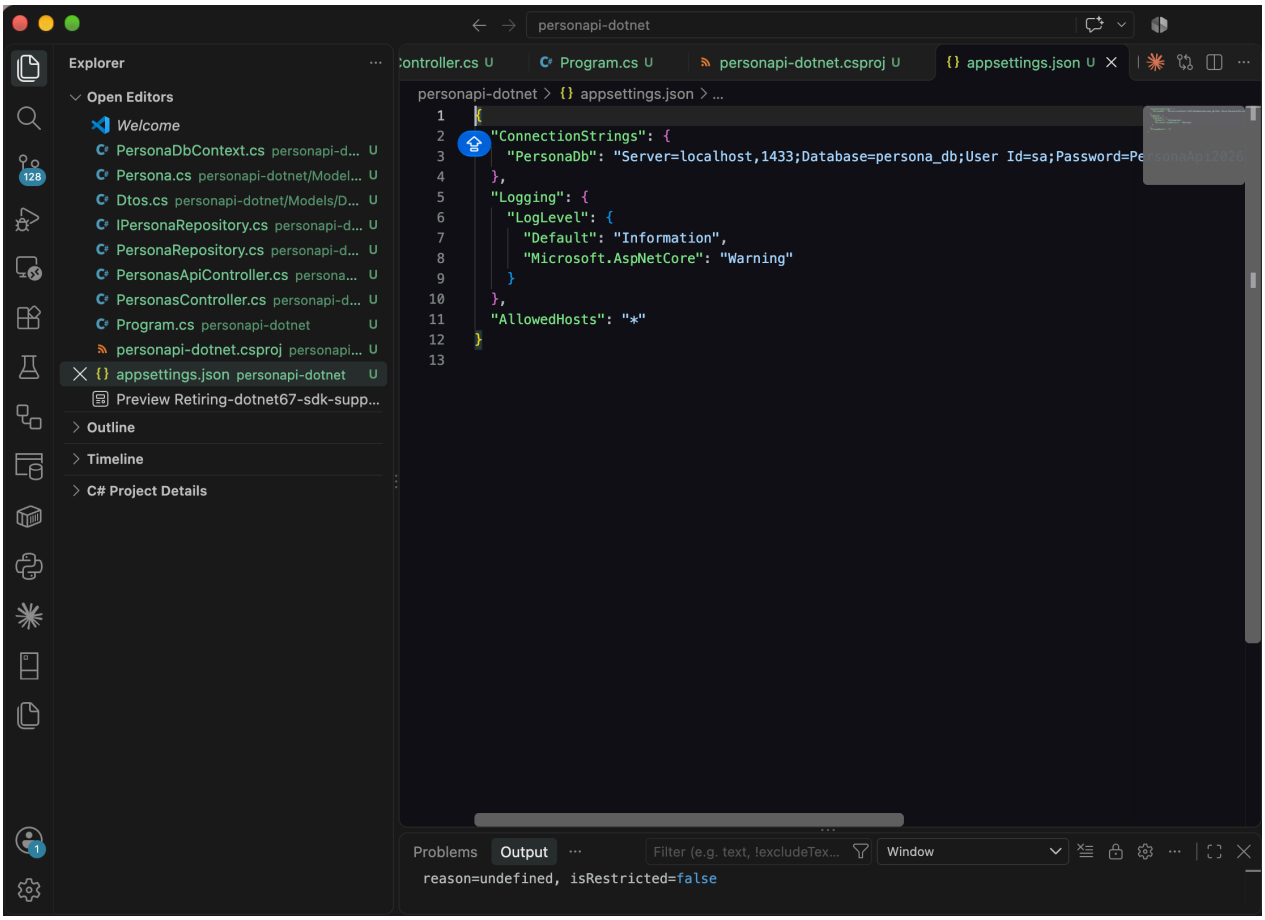
Paso 15. Scaffold. `dotnet ef dbcontext scaffold` con `--output-dir Models/Entities` y `--context PersonaDbContext`.





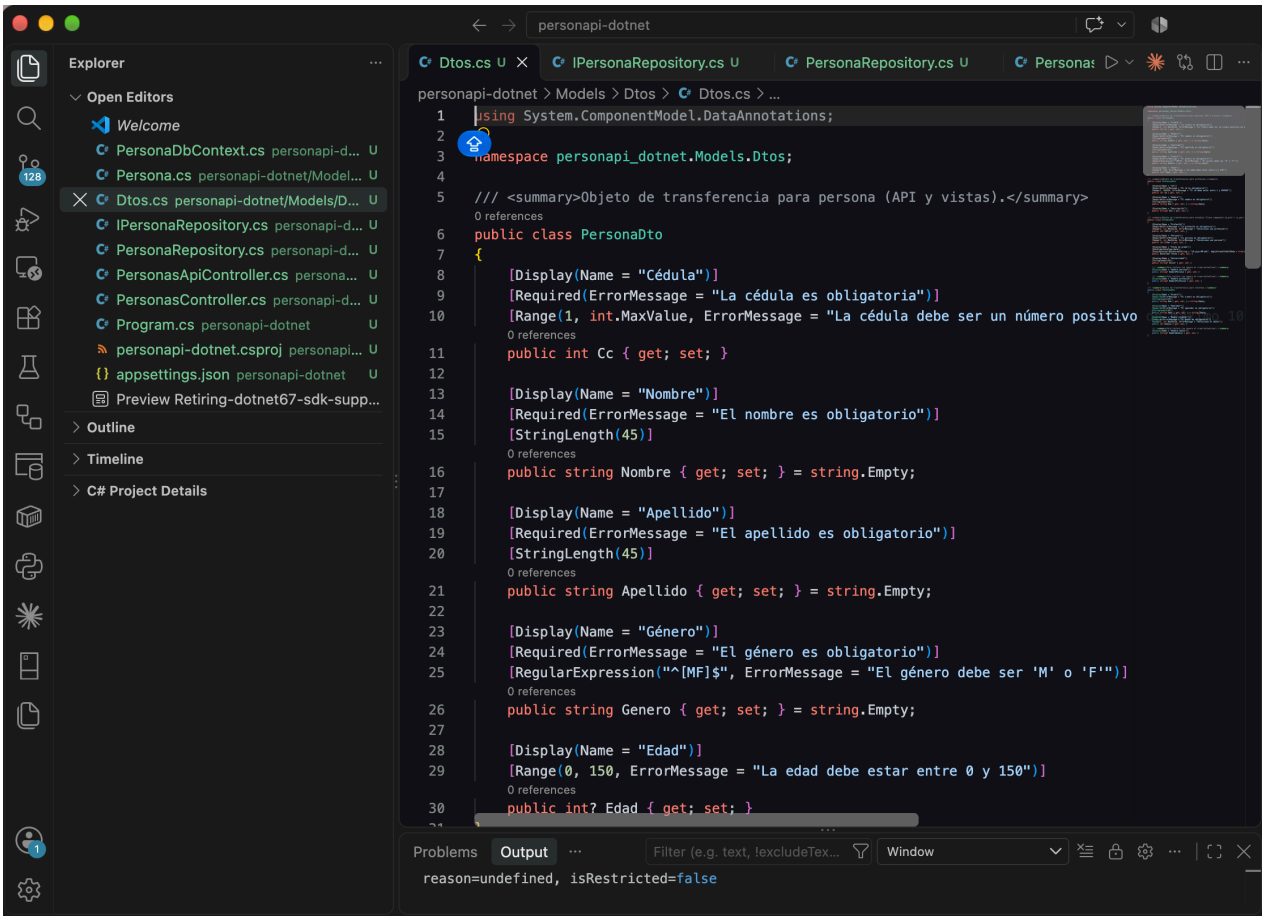


Paso 16. Cadena de conexión. Se agregó `PersonaDb` en `appsettings.json`.

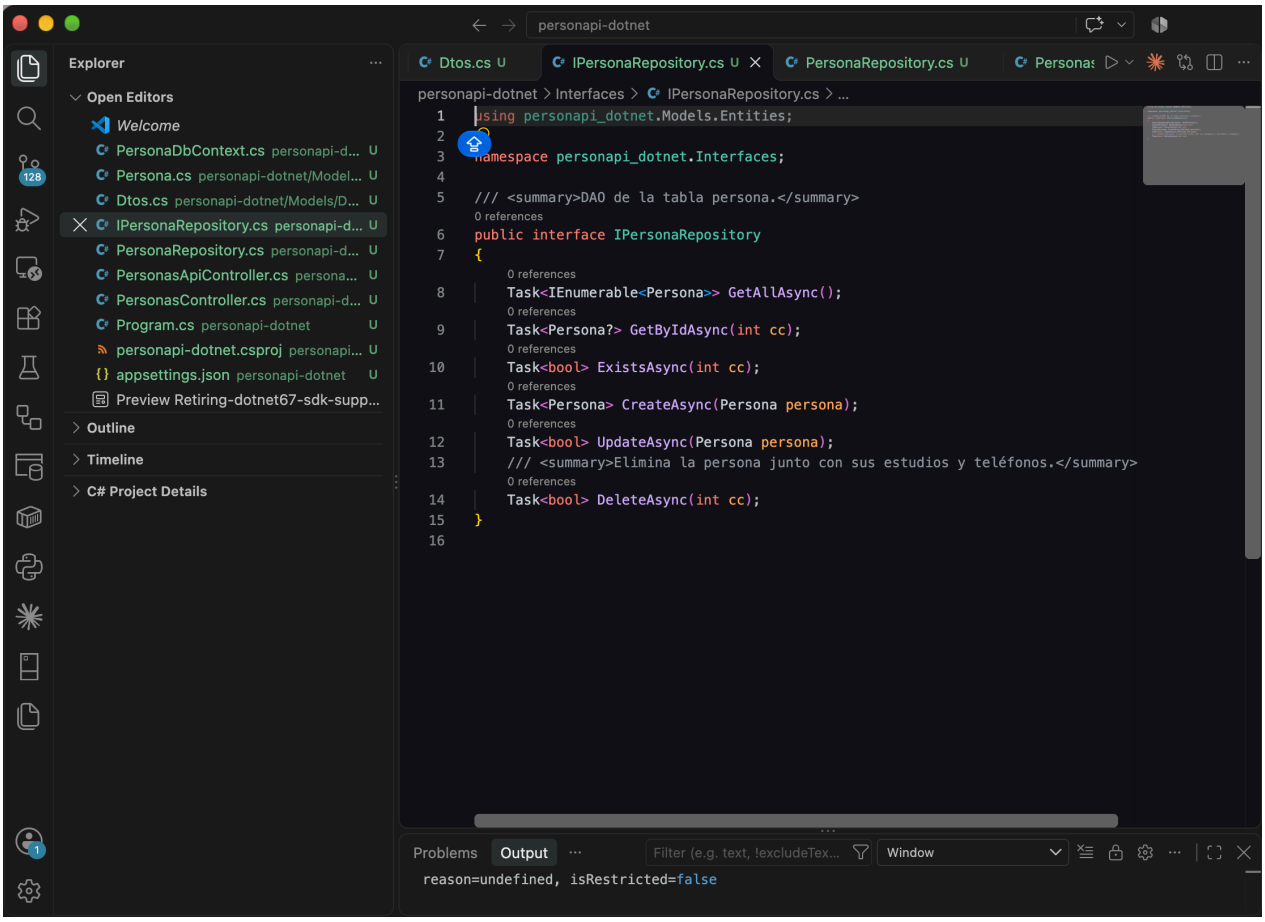


3.5 Código MVC y DAO

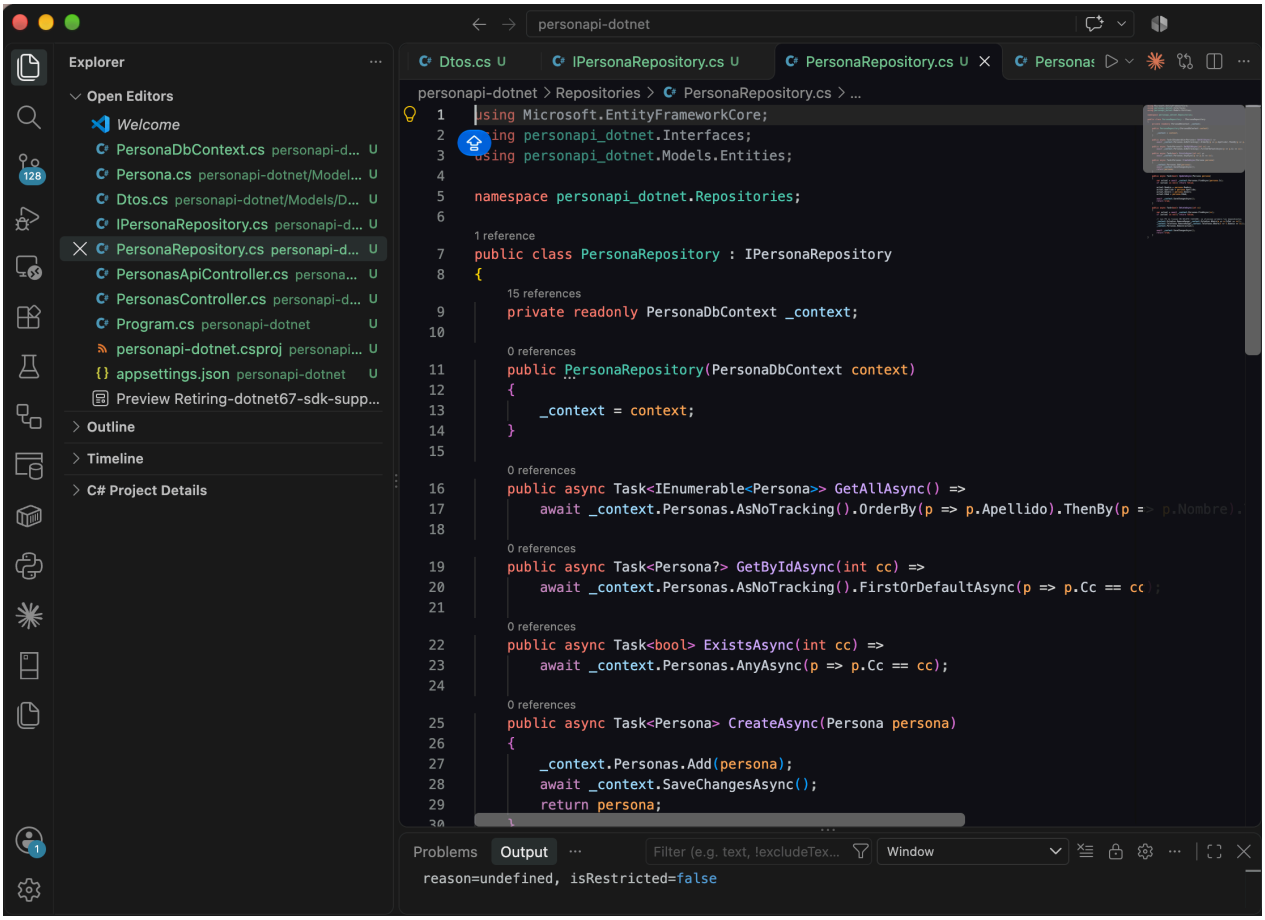
Paso 17. DTOs.



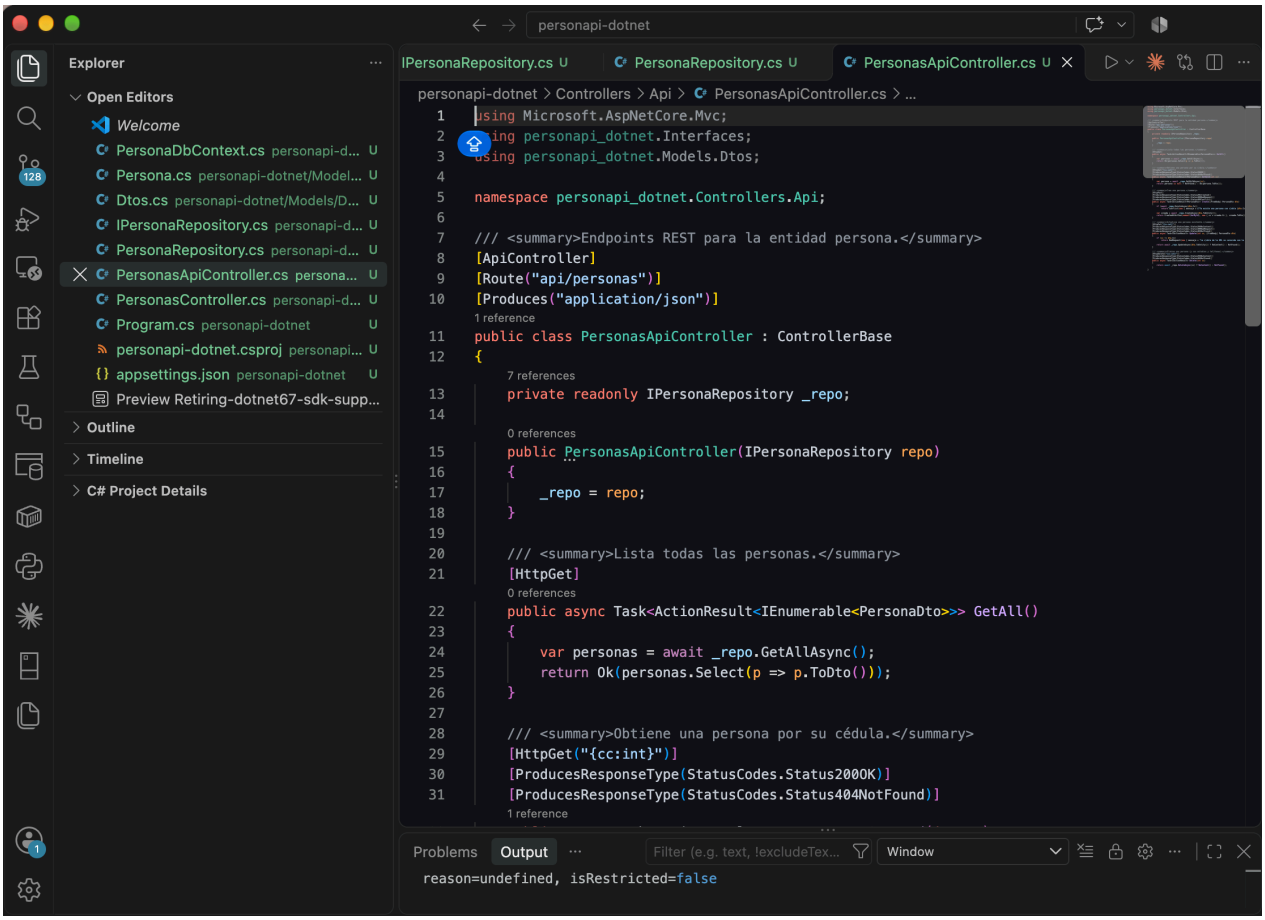
Paso 18. Interfaces.



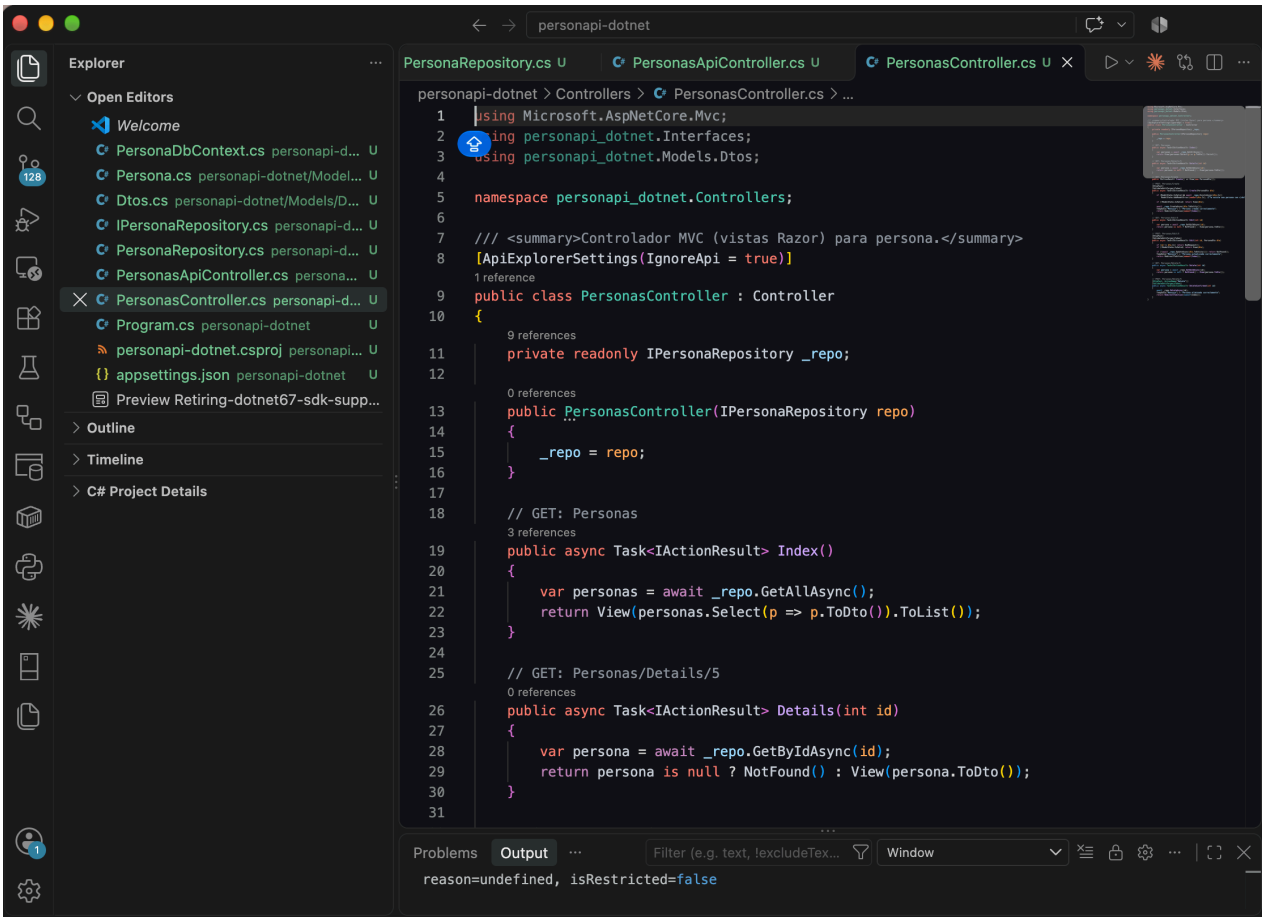
Paso 19. Repositorios.



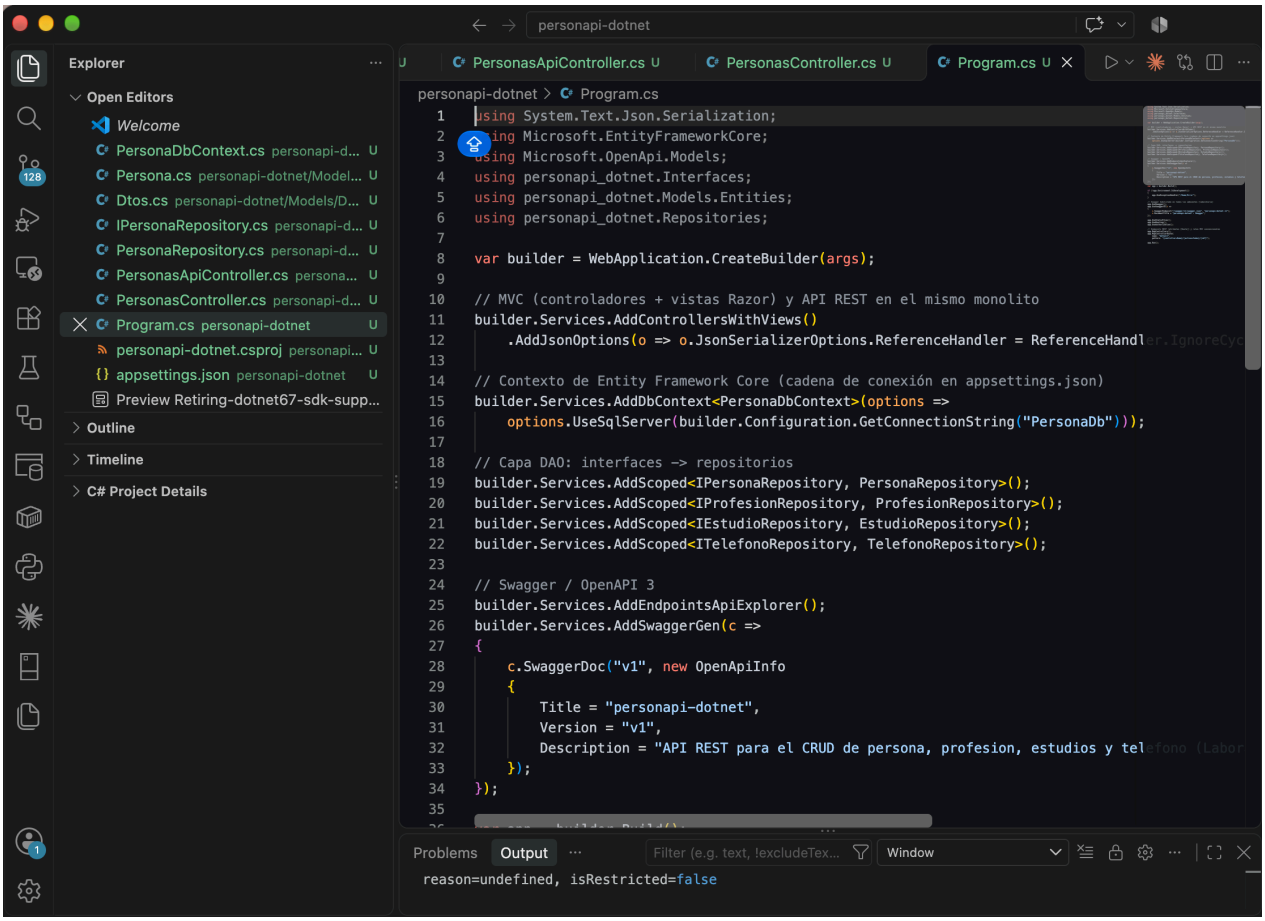
Paso 20. Controladores REST.



Paso 21. Controladores MVC.

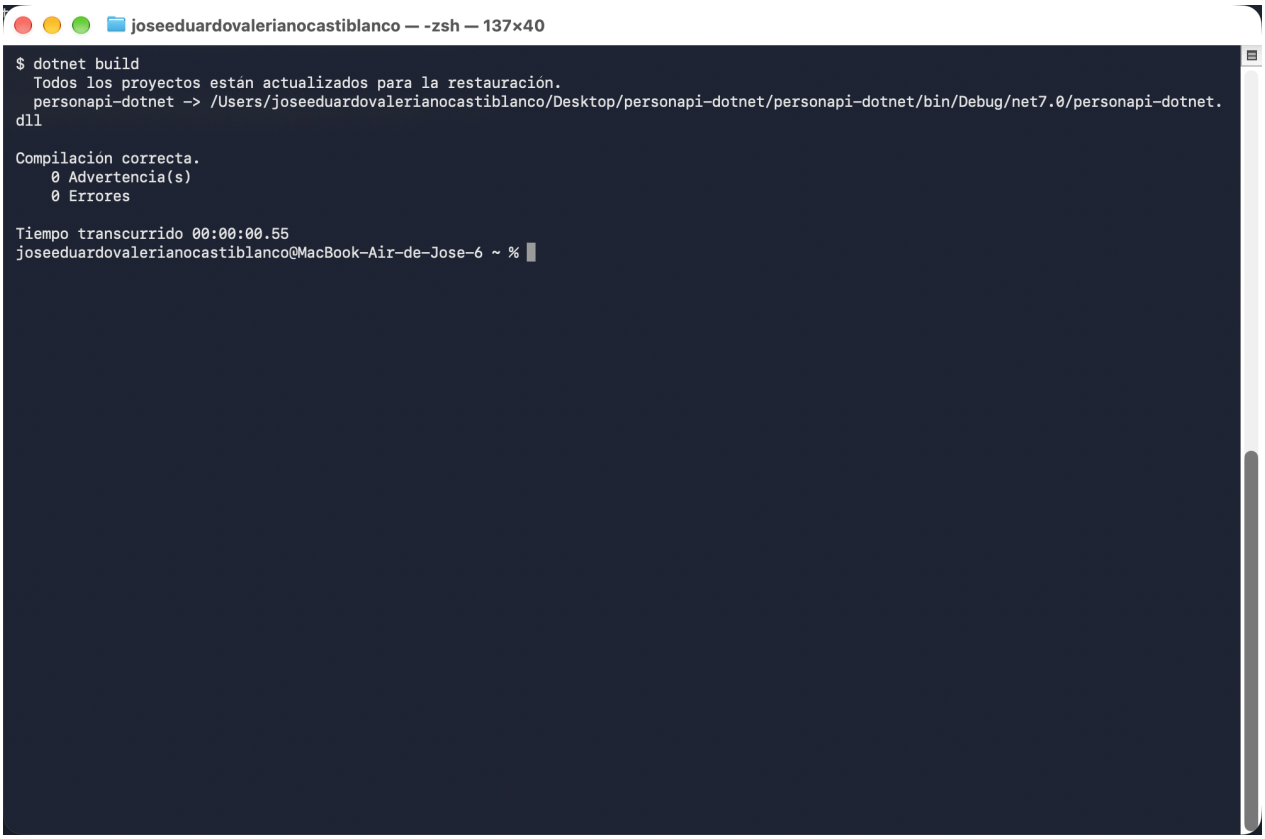


Paso 22. Program.cs. Registro del contexto, de los repositorios y de Swagger.

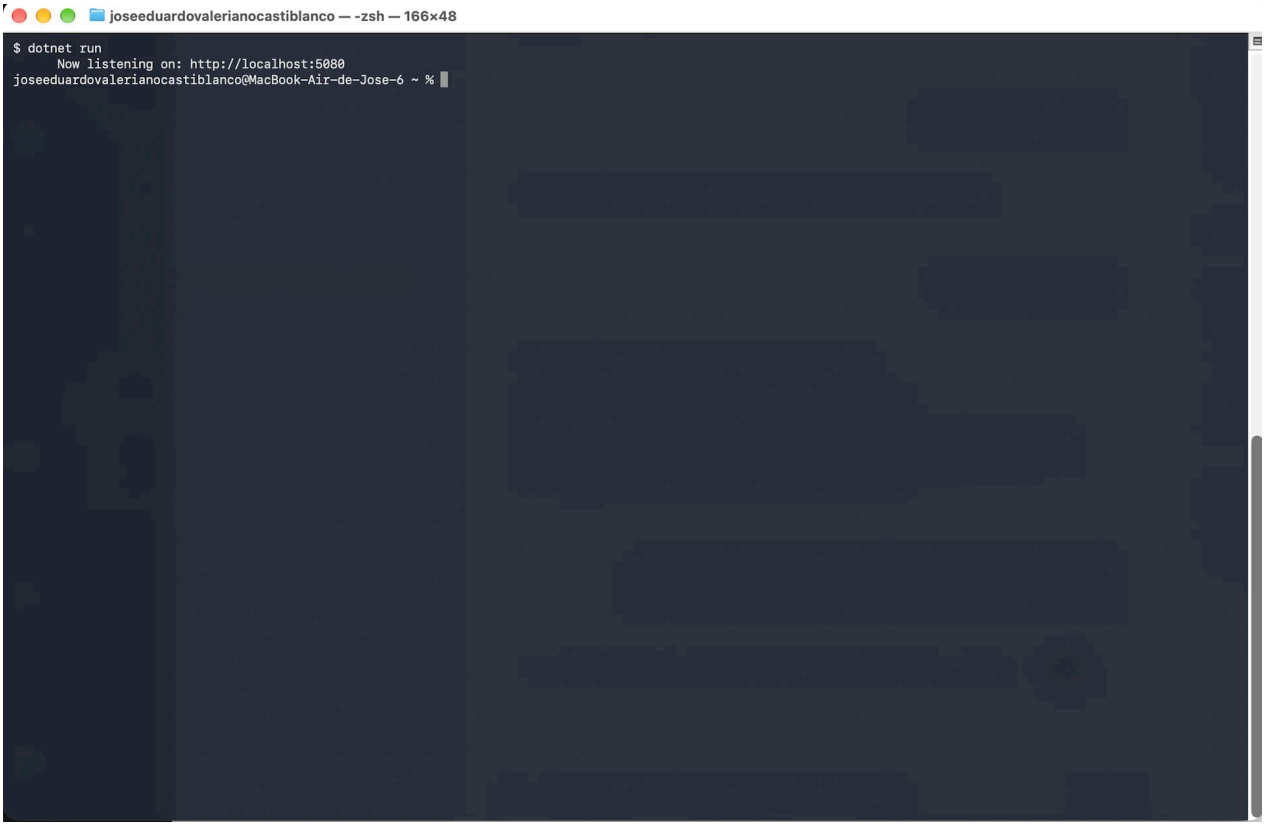


3.6 Compilación y ejecución

Paso 23. Compilación. `dotnet build` sin errores ni advertencias.

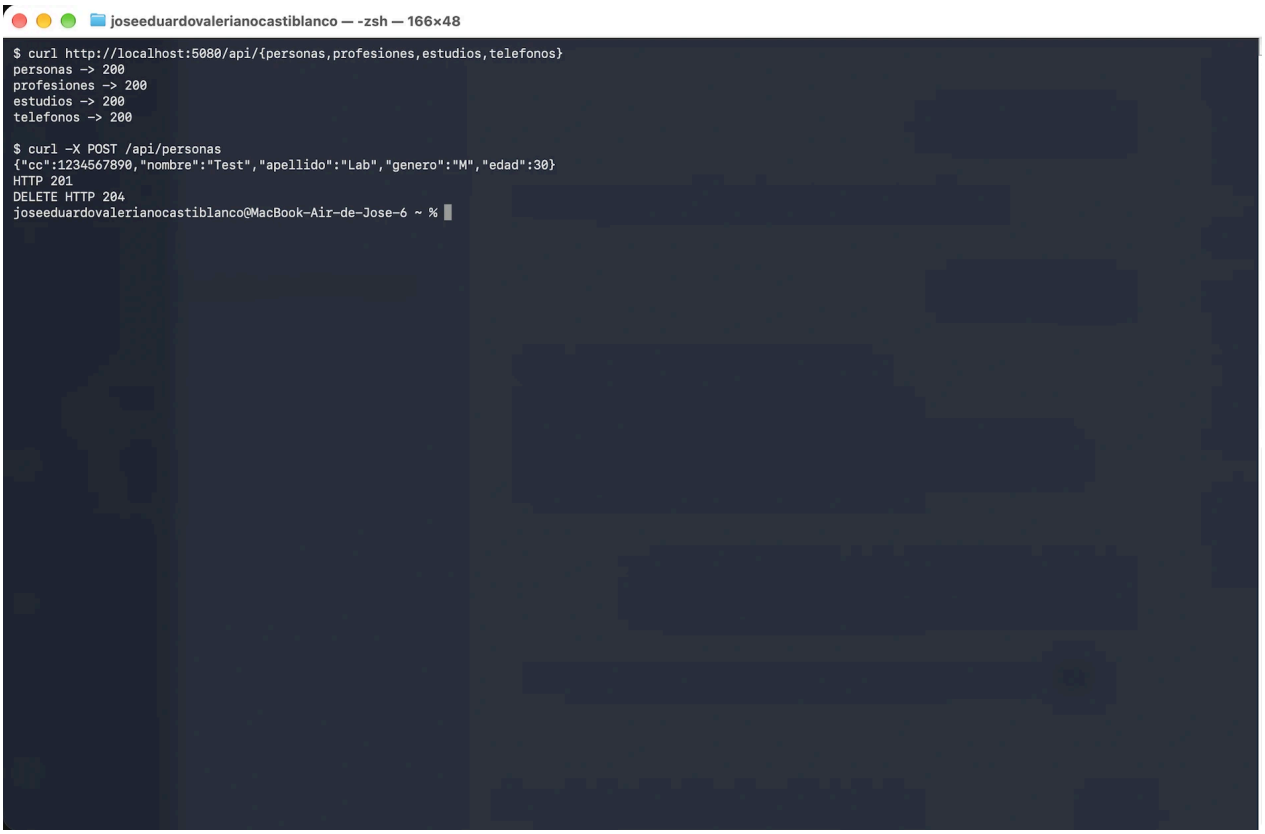


Paso 24. Ejecución. `dotnet run` escucha en `http://localhost:5080`.



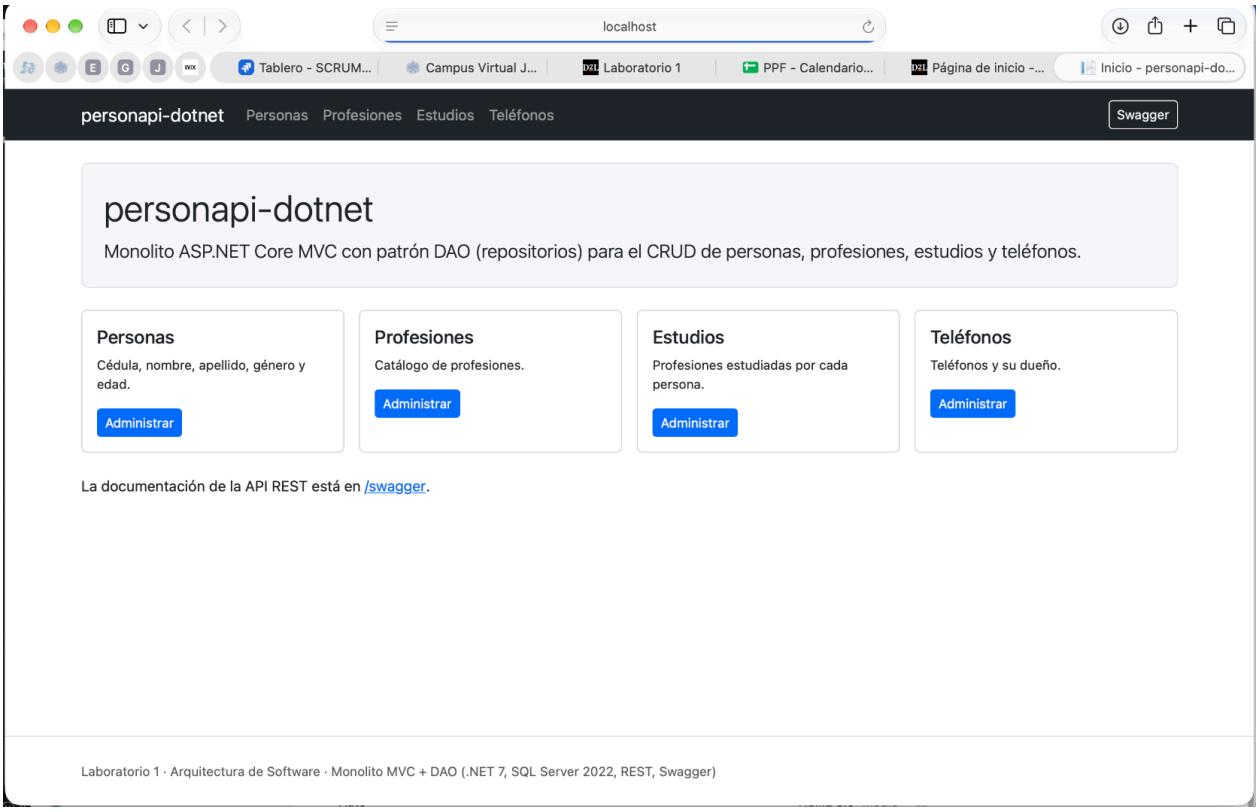
3.7 Pruebas con curl

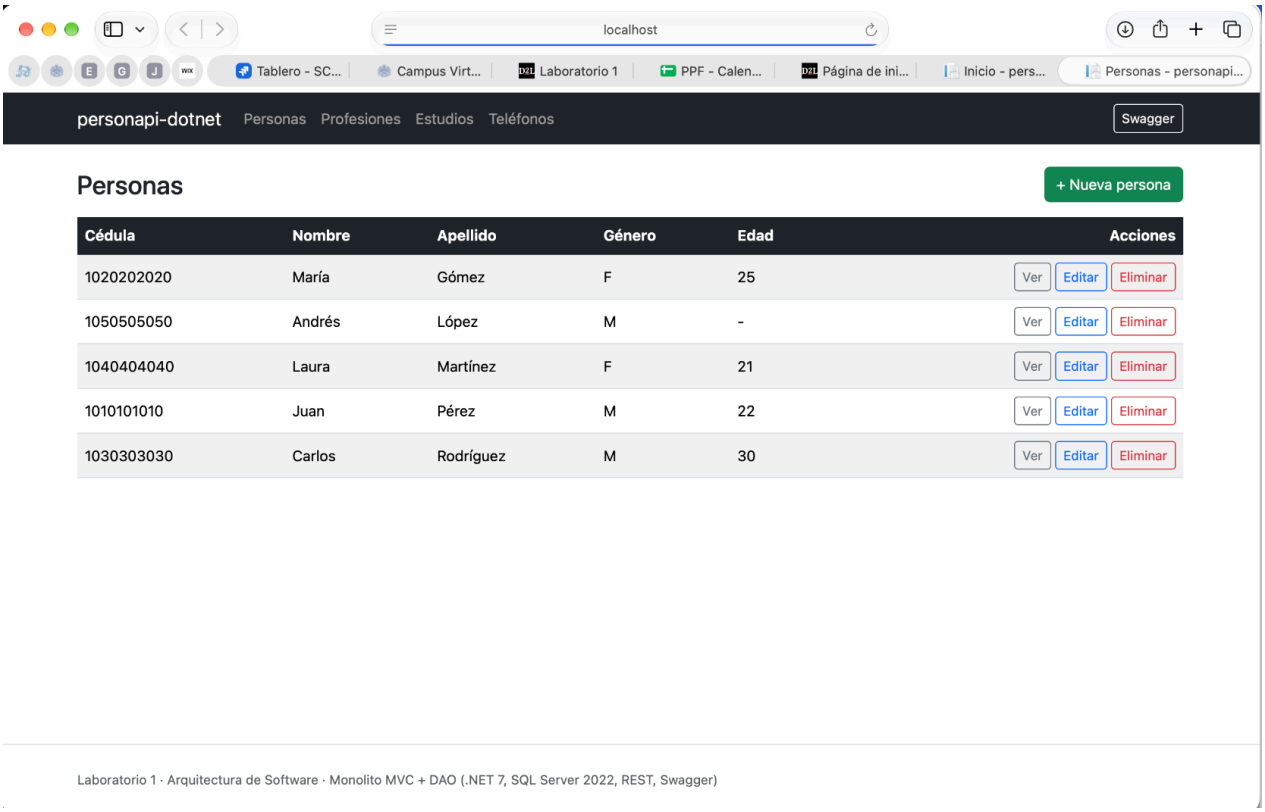
Paso 25. GET de las cuatro entidades (200), POST de una persona (201) y DELETE (204).

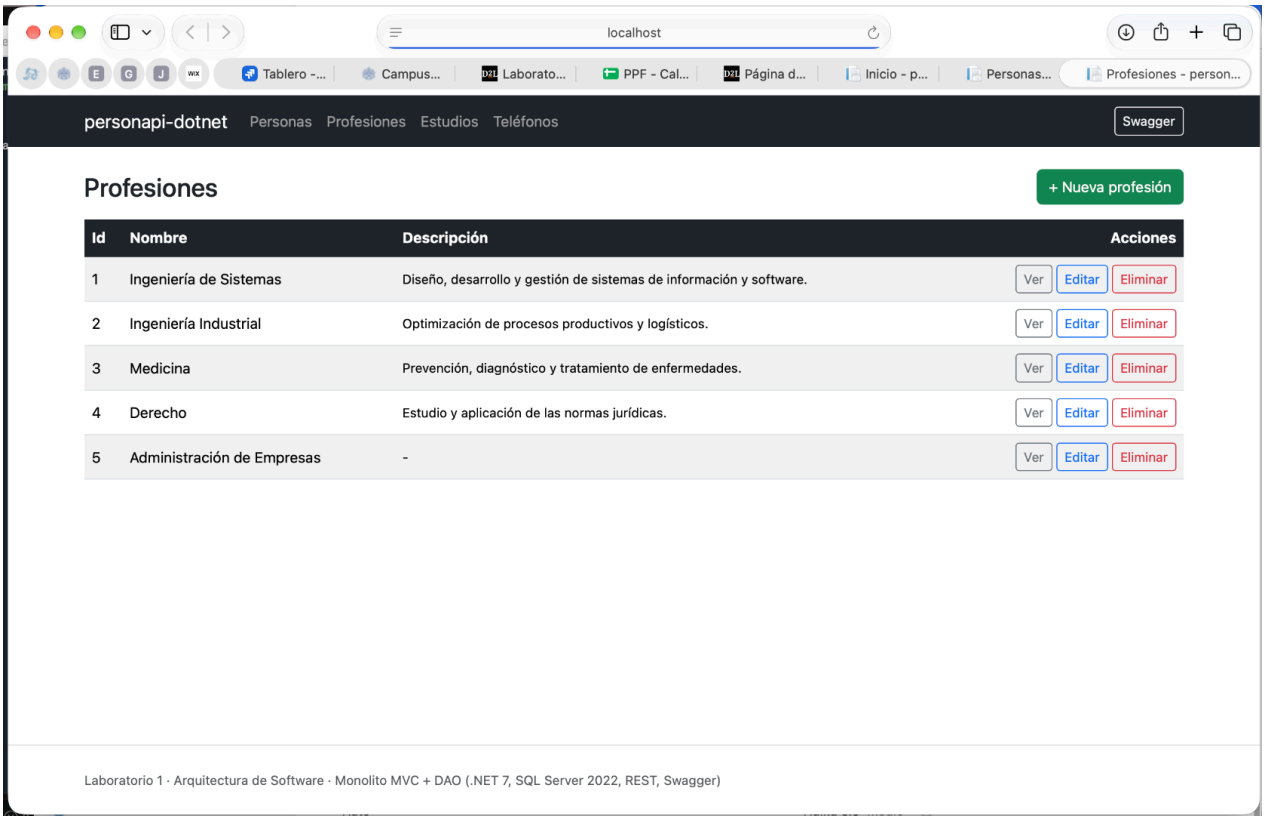


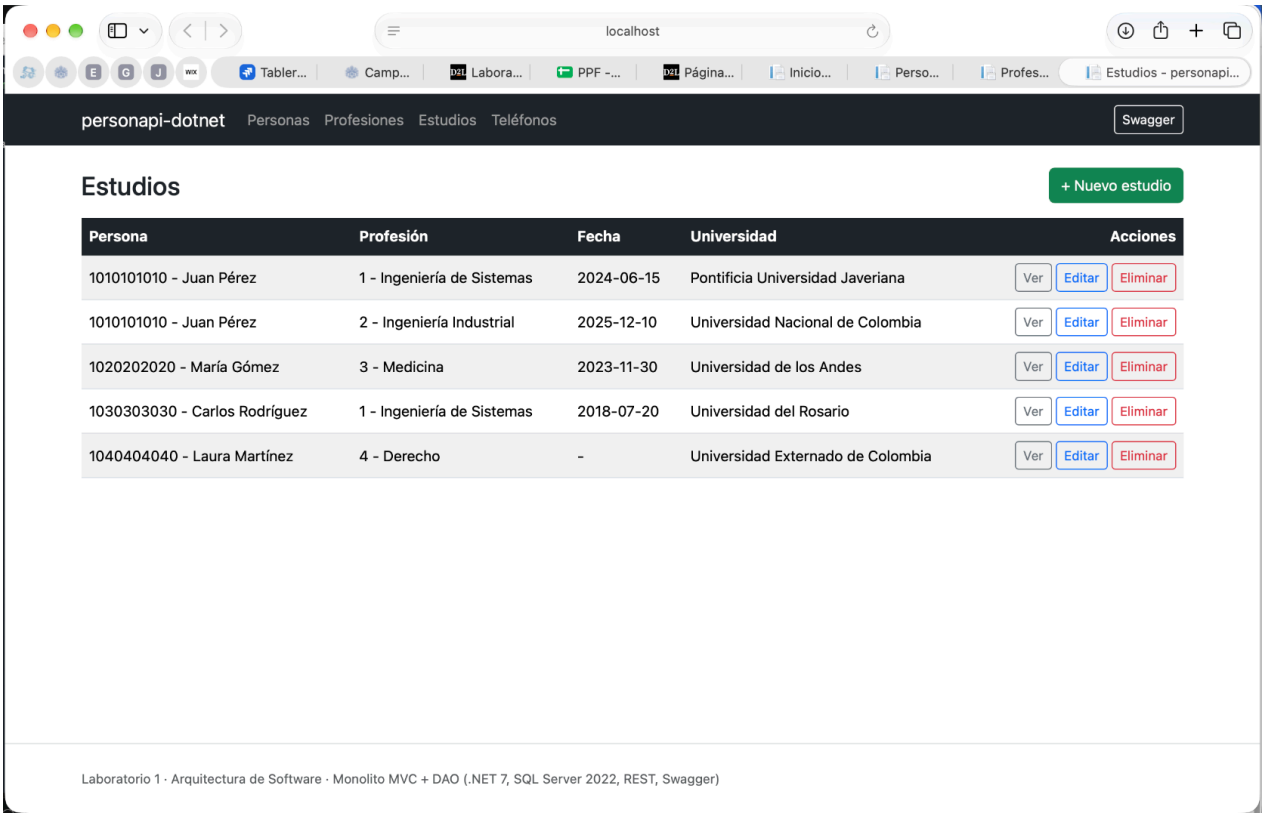
3.8 Pruebas en el navegador (vistas MVC)

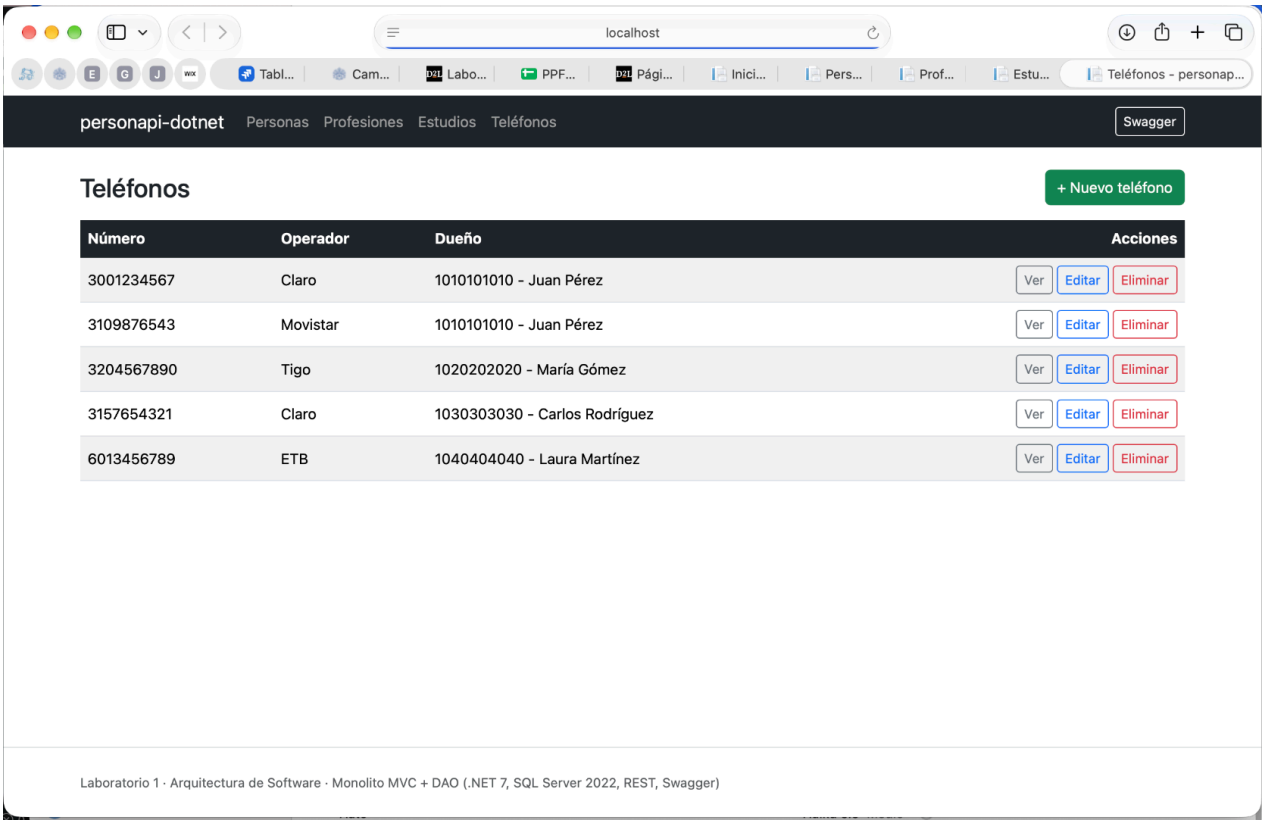
Paso 26. Inicio y listados.



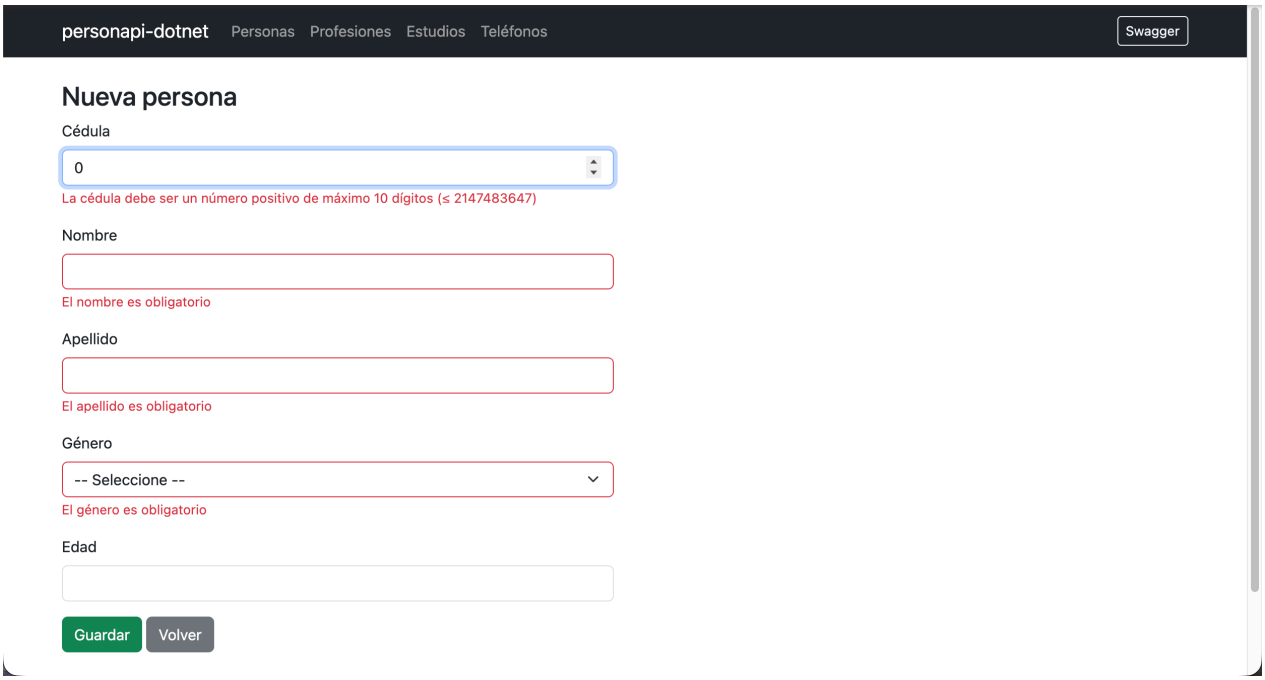

localhost
Tablero - SC...
Campus Virt...
Laboratorio 1
PPF - Calen...
Página de ini...
Inicio - pers...
Personas - personapi...
personapi-dotnet
Personas
Profesiones
Estudios
Teléfonos
Swagger
Personas
+ Nueva persona
Cédula
Nombre
Apellido
Género
Edad
Acciones
1020202020
María
Gómez
F
25
1050505050
Andrés
López
M
-
1040404040
Laura
Martínez
F
21
1010101010
Juan
Pérez
M
22
1030303030
Carlos
Rodríguez
M
30
Ver
Editar
Eliminar
Laboratorio 1 · Arquitectura de Software · Monolito MVC + DAO (.NET 7, SQL Server 2022, REST, Swagger)

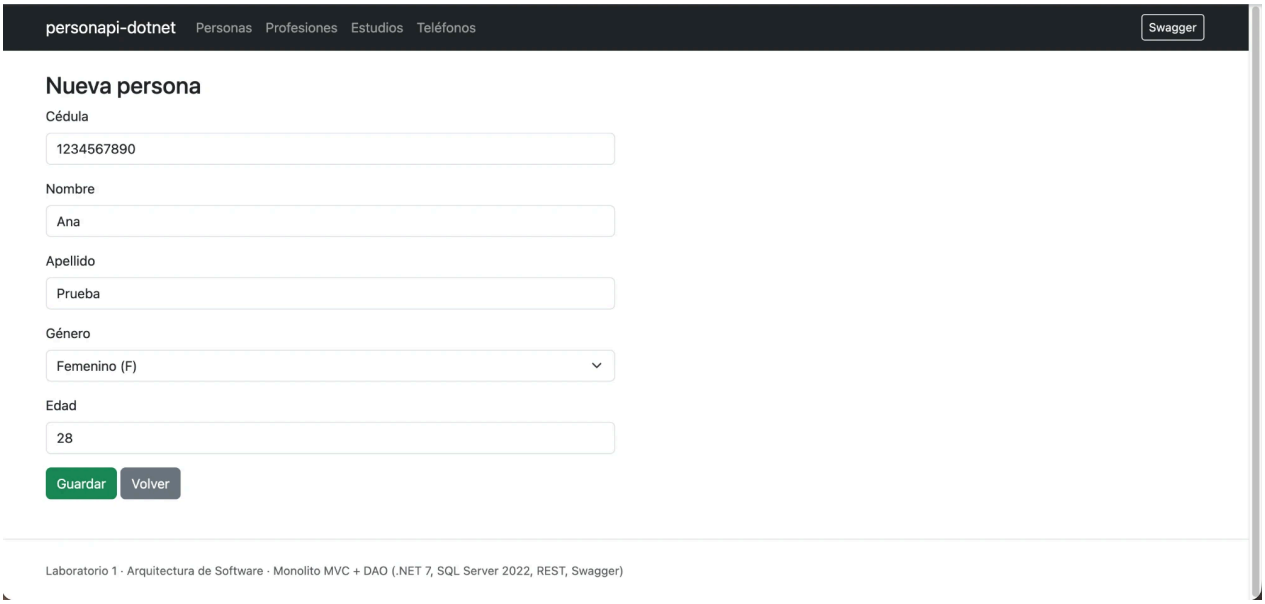

localhost
Tablero -...
Campus...
Laborato...
PPF - Cal...
Página d...
Inicio - p...
Personas...
Profesiones - person...
personapi-dotnet
Personas
Profesiones
Estudios
Teléfonos
Swagger
Profesiones
+ Nueva profesión
Id
Nombre
Descripción
Acciones
1
Ingeniería de Sistemas
Diseño, desarrollo y gestión de sistemas de información y software.
2
Ingeniería Industrial
Optimización de procesos productivos y logísticos.
3
Medicina
Prevención, diagnóstico y tratamiento de enfermedades.
4
Derecho
Estudio y aplicación de las normas jurídicas.
5
Administración de Empresas
-
Ver
Editar
Eliminar
Laboratorio 1 · Arquitectura de Software · Monolito MVC + DAO (.NET 7, SQL Server 2022, REST, Swagger)

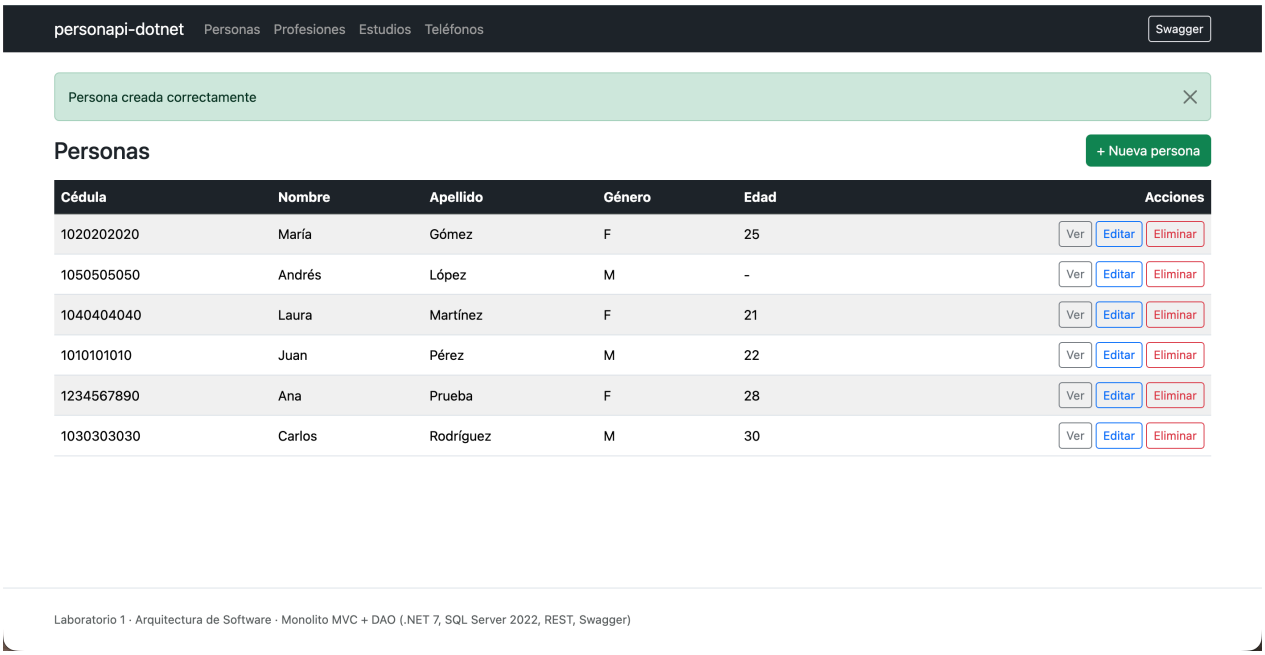


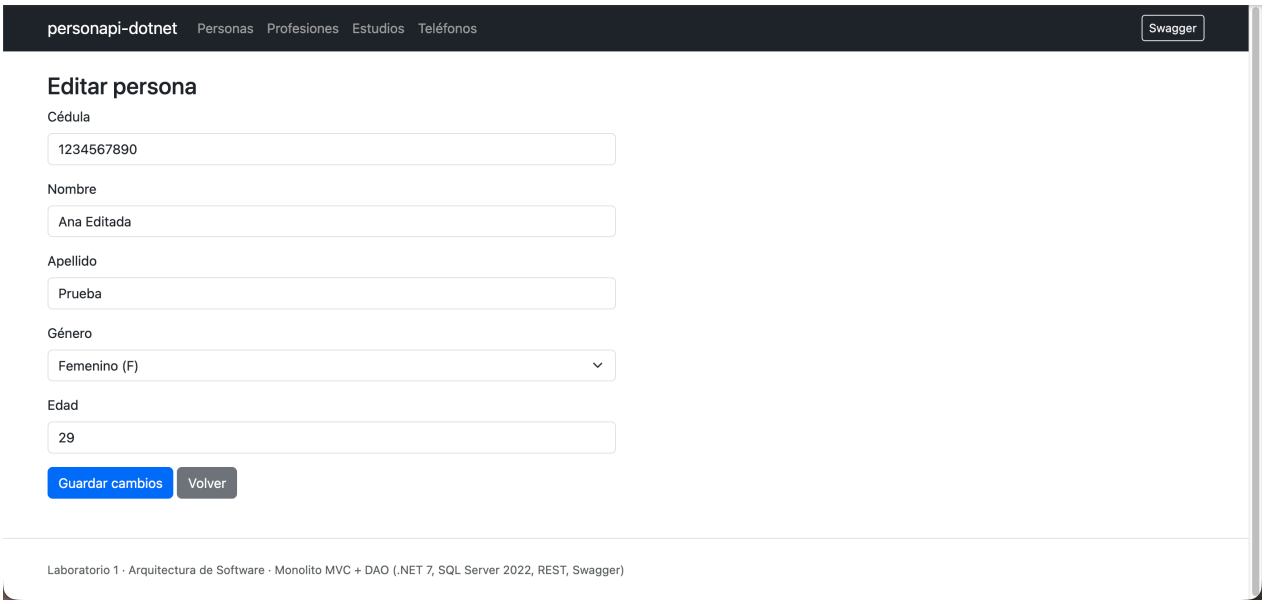


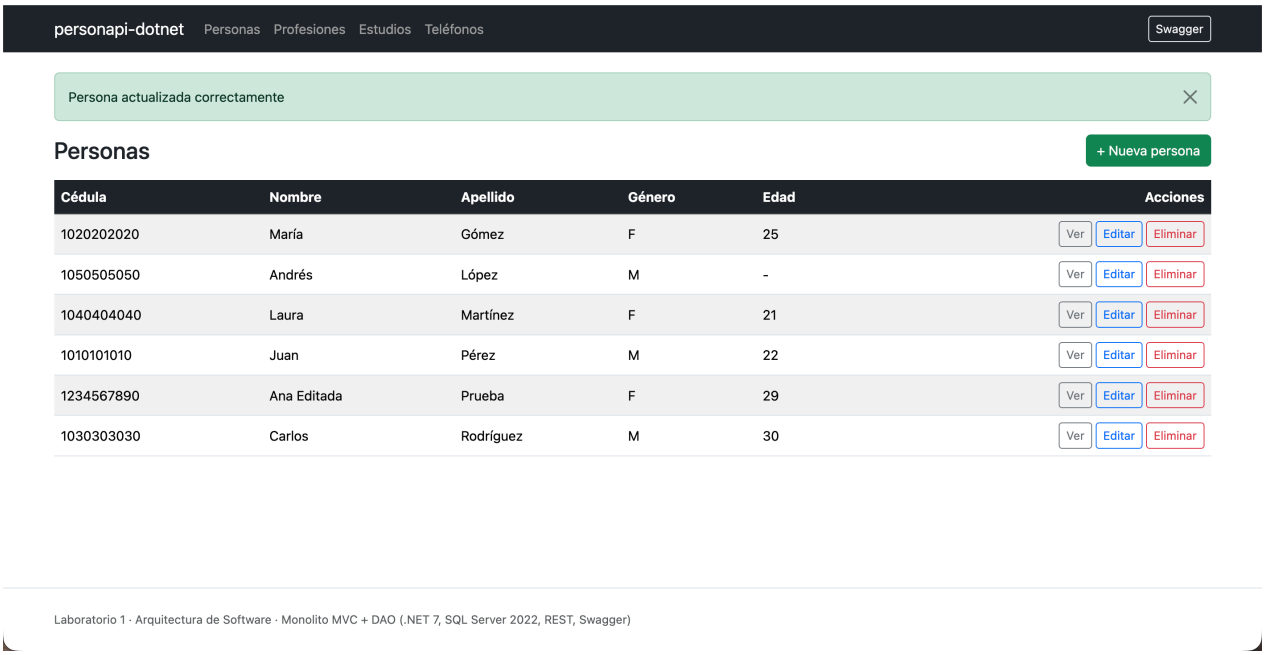
Paso 27. CRUD de personas. Errores de validación, crear, listado tras crear, editar, confirmar eliminar y listado tras eliminar.

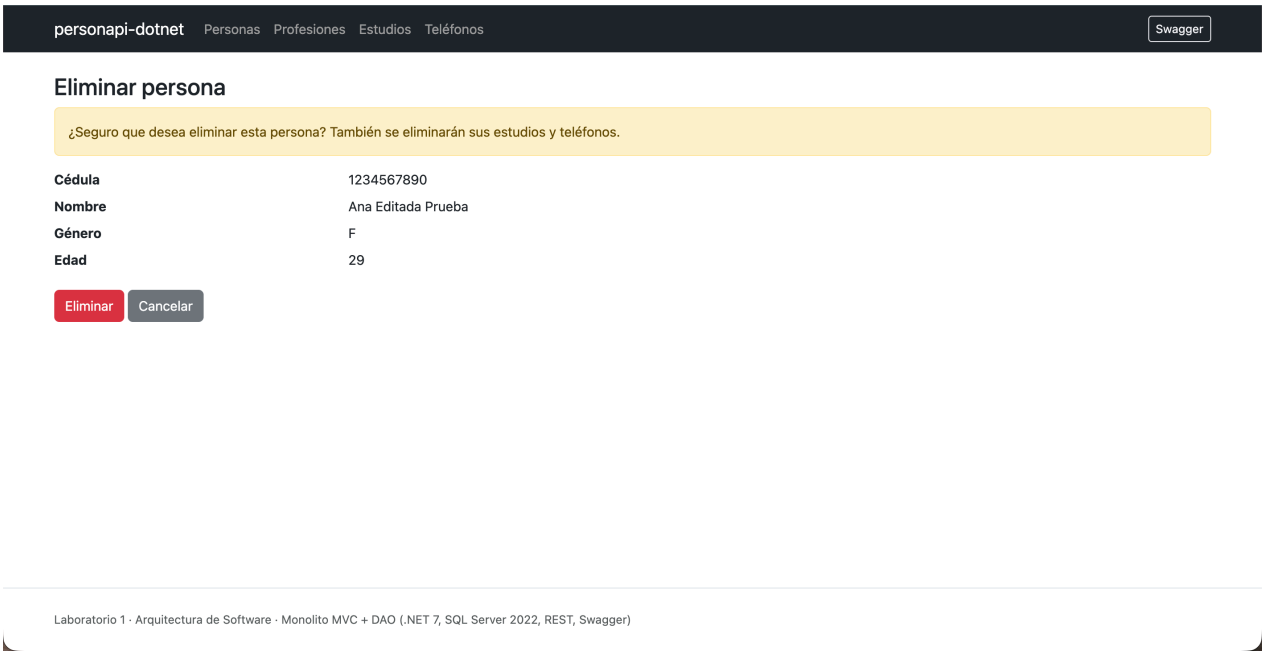

personapi-dotnet
Personas
Profesiones
Estudios
Teléfonos
Swagger
Nueva persona
Cédula
0
La cédula debe ser un número positivo de máximo 10 dígitos (≤ 2147483647)
Nombre
El nombre es obligatorio
Apellido
El apellido es obligatorio
Género
-- Seleccione --
El género es obligatorio
Edad
Guardar
Volver

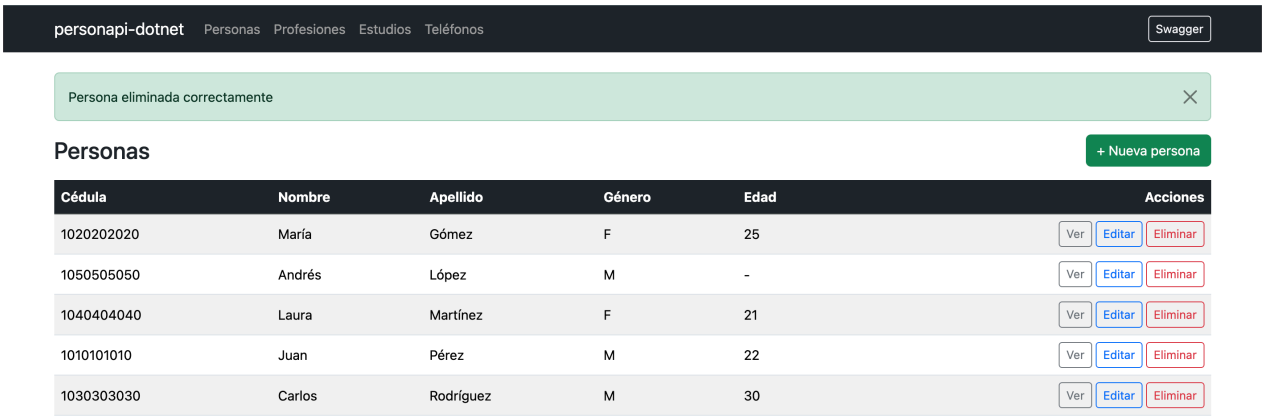

personapi-dotnet
Personas
Profesiones
Estudios
Teléfonos
Swagger
Nueva persona
Cédula
1234567890
Nombre
Ana
Apellido
Prueba
Género
Femenino (F)
Edad
28
Guardar
Volver
Laboratorio 1 · Arquitectura de Software · Monolito MVC + DAO (.NET 7, SQL Server 2022, REST, Swagger)


personapi-dotnet
Personas
Profesiones
Estudios
Teléfonos
Swagger
Persona creada correctamente
Personas
+ Nueva persona
Cédula
Nombre
Apellido
Género
Edad
Acciones
1020202020 María Gómez F 25 Ver Editar Eliminar
1050505050 Andrés López M - Ver Editar Eliminar
1040404040 Laura Martínez F 21 Ver Editar Eliminar
1010101010 Juan Pérez M 22 Ver Editar Eliminar
1234567890 Ana Prueba F 28 Ver Editar Eliminar
1030303030 Carlos Rodríguez M 30 Ver Editar Eliminar
Laboratorio 1 · Arquitectura de Software · Monolito MVC + DAO (.NET 7, SQL Server 2022, REST, Swagger)

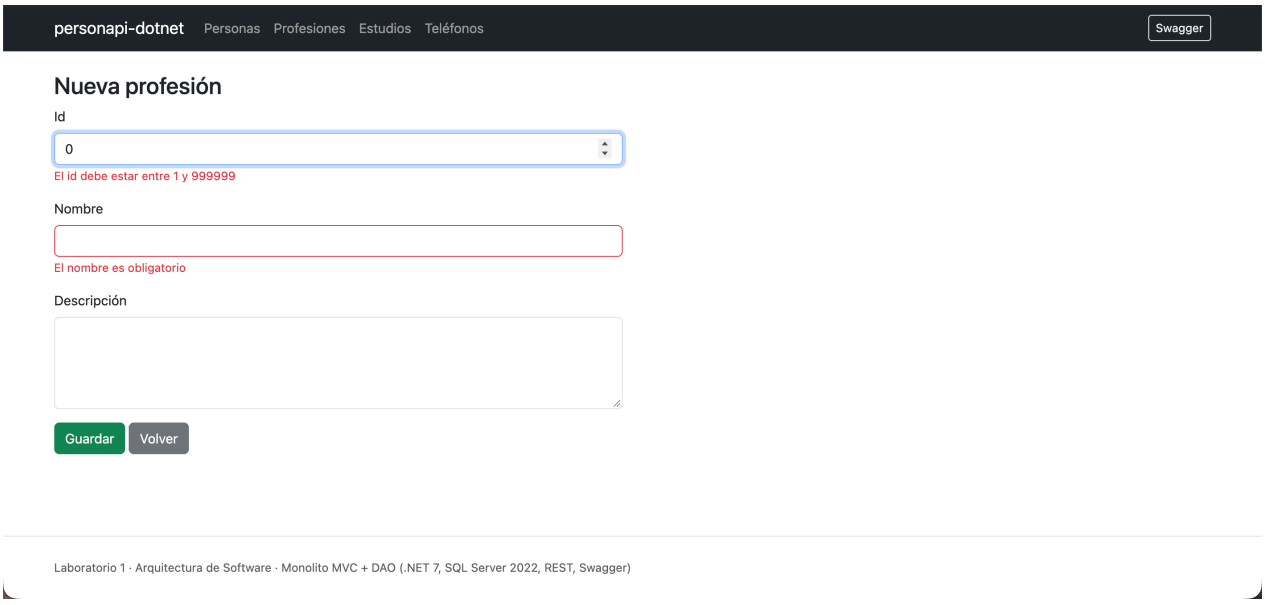

personapi-dotnet
Personas
Profesiones
Estudios
Teléfonos
Swagger
Editar persona
Cédula
1234567890
Nombre
Ana Editada
Apellido
Prueba
Género
Femenino (F)
Edad
29
Guardar cambios
Volver
Laboratorio 1 · Arquitectura de Software · Monolito MVC + DAO (.NET 7, SQL Server 2022, REST, Swagger)

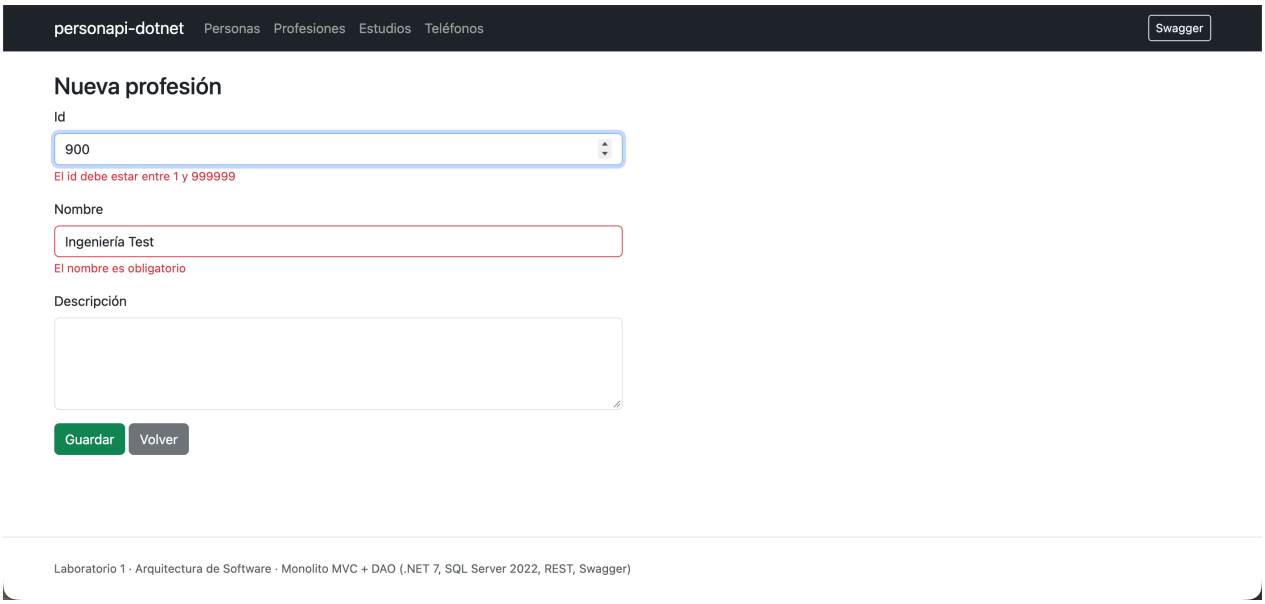

personapi-dotnet
Personas
Profesiones
Estudios
Teléfonos
Swagger
Persona actualizada correctamente
Personas
+ Nueva persona
Cédula
Nombre
Apellido
Género
Edad
Acciones
1020202020 María Gómez F 25 Ver Editar Eliminar
1050505050 Andrés López M - Ver Editar Eliminar
1040404040 Laura Martínez F 21 Ver Editar Eliminar
1010101010 Juan Pérez M 22 Ver Editar Eliminar
1234567890 Ana Editada Prueba F 29 Ver Editar Eliminar
1030303030 Carlos Rodríguez M 30 Ver Editar Eliminar
Laboratorio 1 · Arquitectura de Software · Monolito MVC + DAO (.NET 7, SQL Server 2022, REST, Swagger)

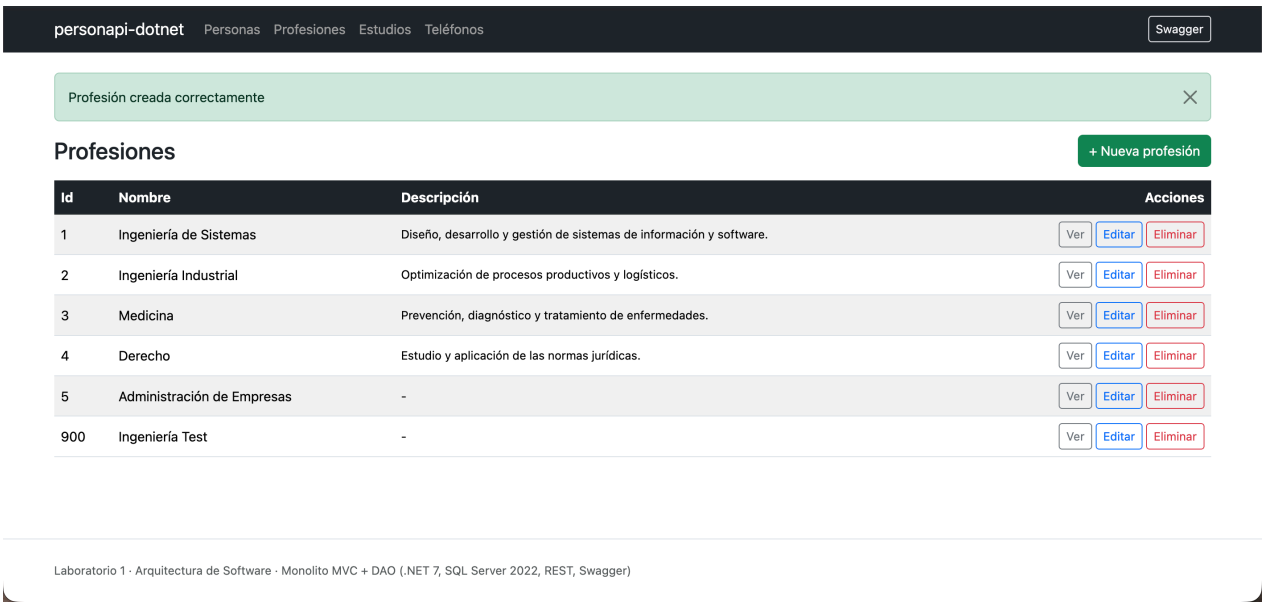

personapi-dotnet
Personas
Profesiones
Estudios
Teléfonos
Swagger
Eliminar persona
¿Seguro que desea eliminar esta persona? También se eliminarán sus estudios y teléfonos.
Cédula 1234567890
Nombre Ana Editada Prueba
Género F
Edad 29
Eliminar
Cancelar
Laboratorio 1 · Arquitectura de Software · Monolito MVC + DAO (.NET 7, SQL Server 2022, REST, Swagger)

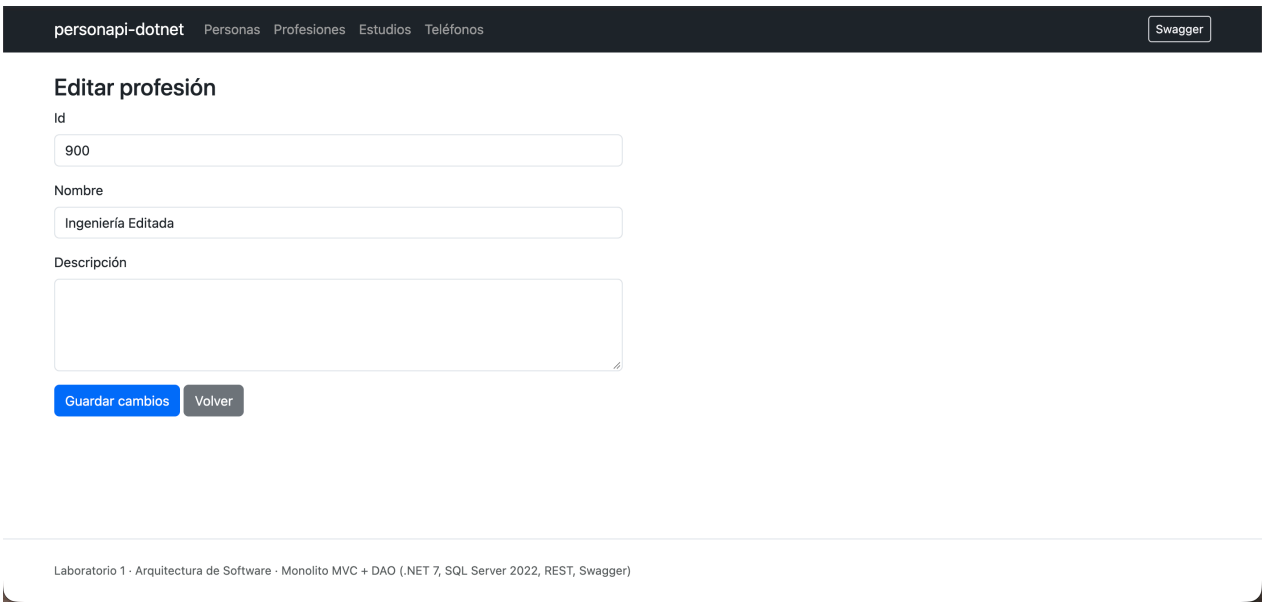


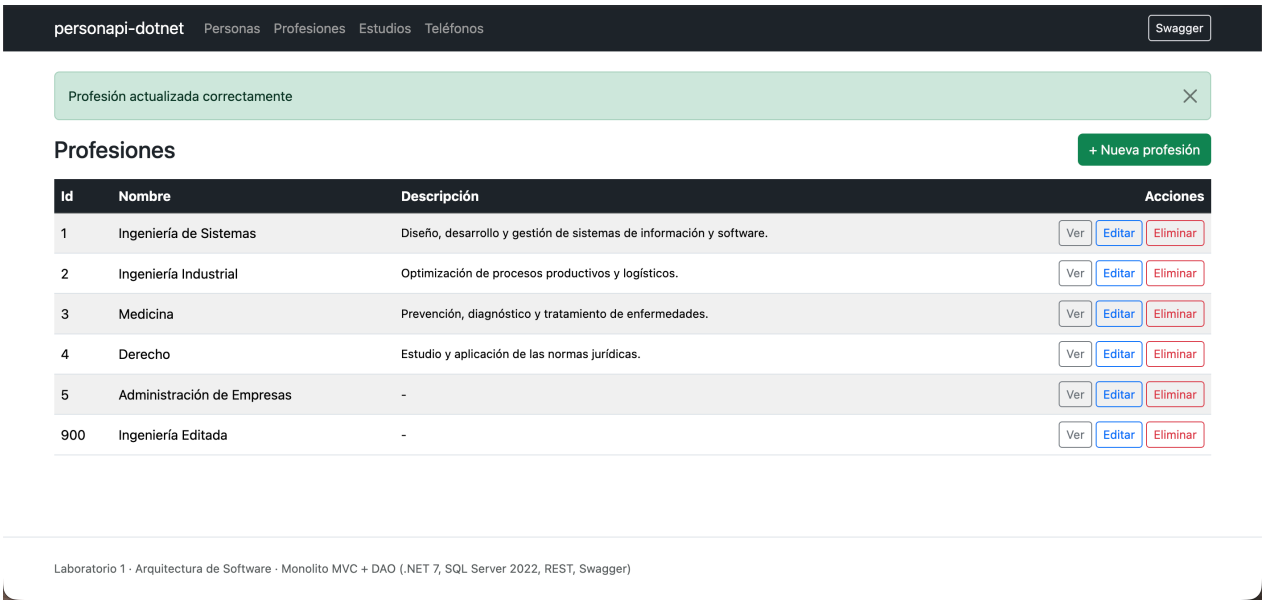
Paso 28. CRUD de profesiones.

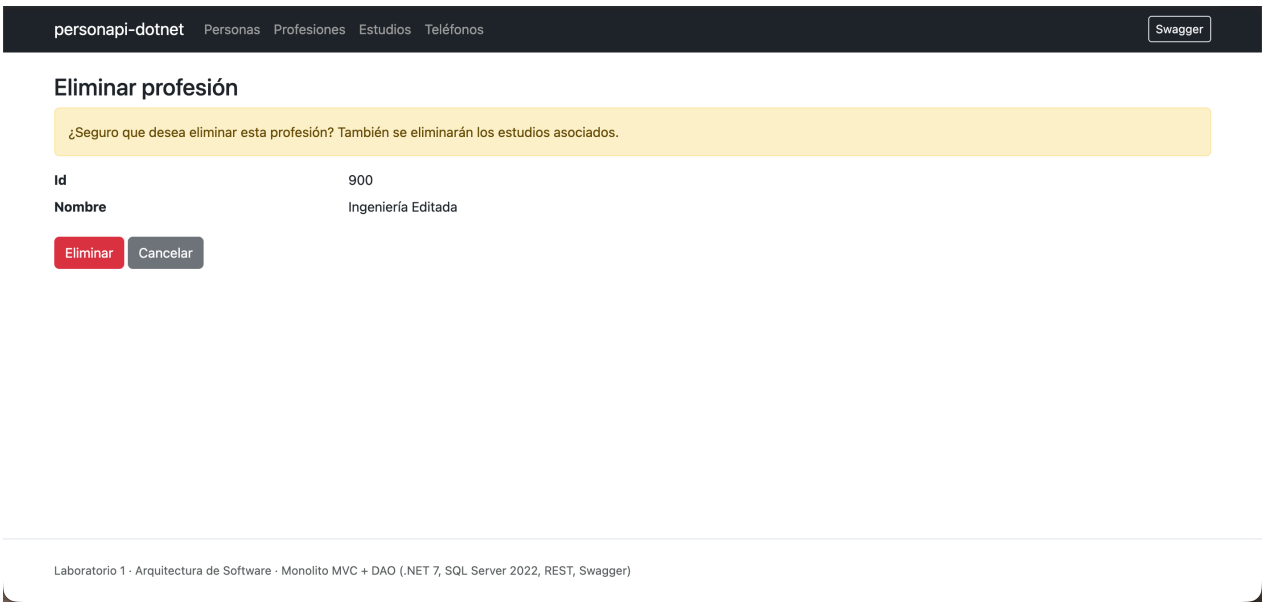


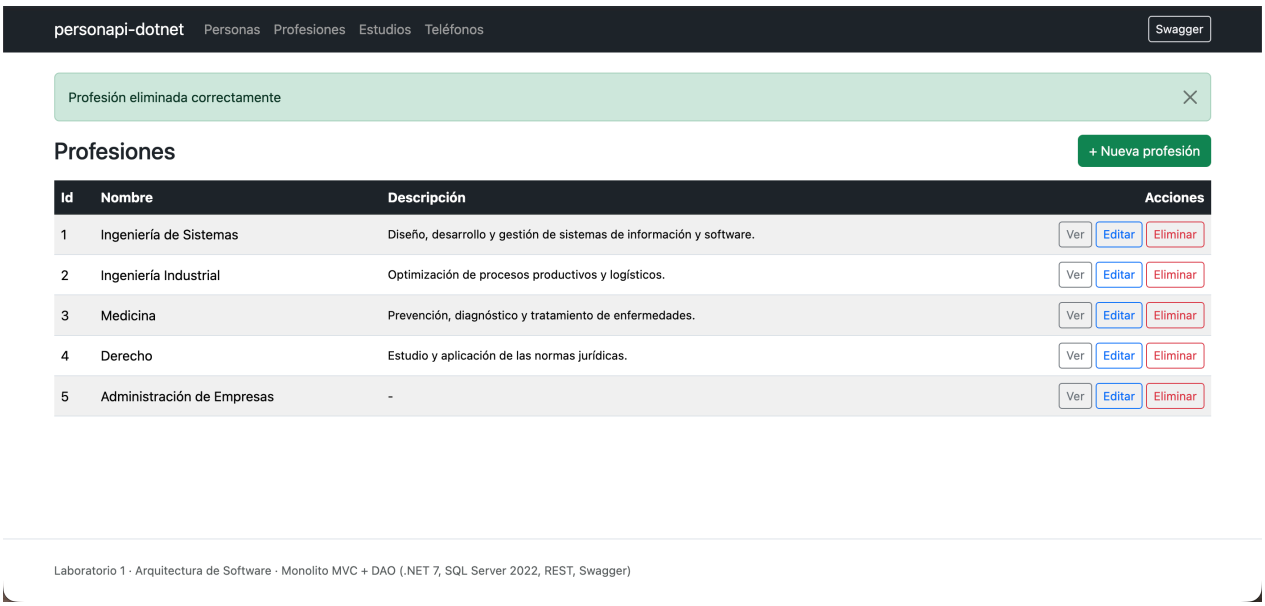

personapi-dotnet
Personas
Profesiones
Estudios
Teléfonos
Swagger
Nueva profesión
Id
900
El id debe estar entre 1 y 999999
Nombre
Ingeniería Test
El nombre es obligatorio
Descripción
Guardar
Volver


personapi-dotnet
Personas
Profesiones
Estudios
Teléfonos
Swagger
Profesión creada correctamente
Profesiones
+ Nueva profesión
Id
Nombre
Descripción
Acciones
1
Ingeniería de Sistemas
Diseño, desarrollo y gestión de sistemas de información y software.
2
Ingeniería Industrial
Optimización de procesos productivos y logísticos.
3
Medicina
Prevención, diagnóstico y tratamiento de enfermedades.
4
Derecho
Estudio y aplicación de las normas jurídicas.
5
Administración de Empresas
-
900
Ingeniería Test
-
Ver
Editar
Eliminar

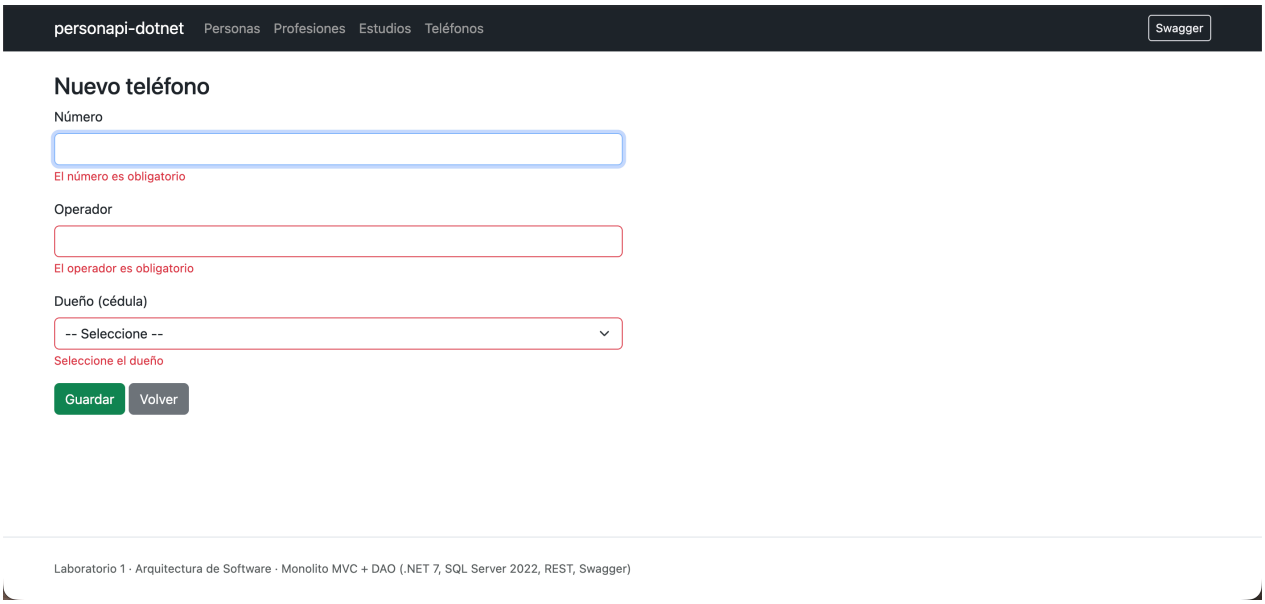

personapi-dotnet
Personas
Profesiones
Estudios
Teléfonos
Swagger
Editar profesión
Id
900
Nombre
Ingeniería Editada
Descripción
Guardar cambios
Volver

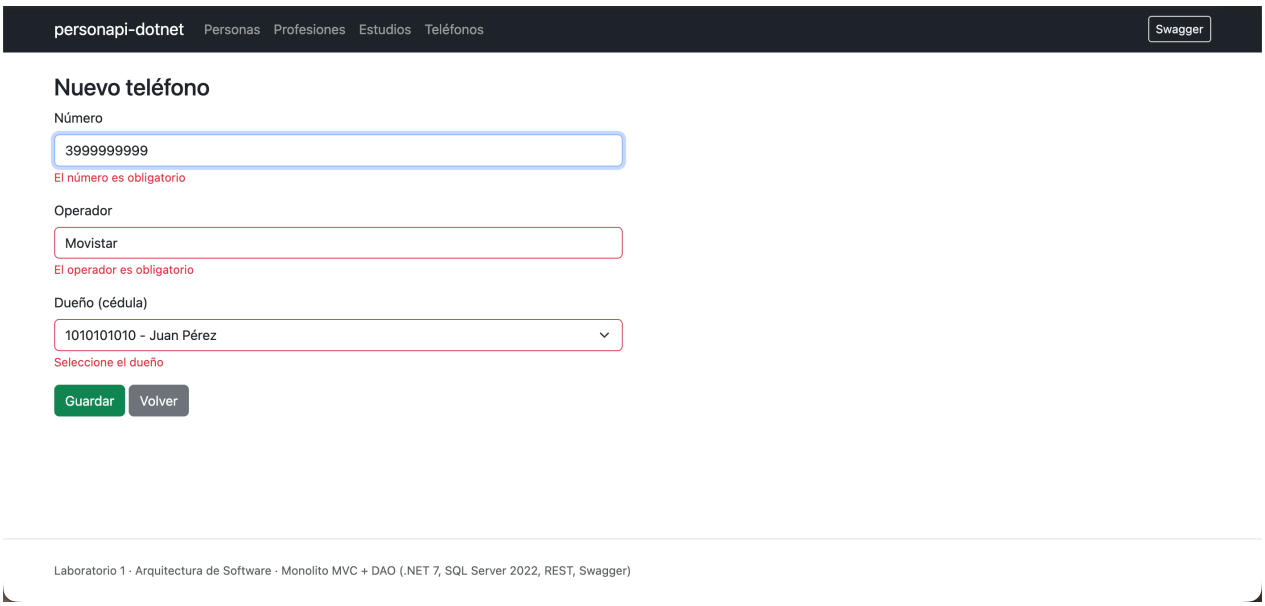


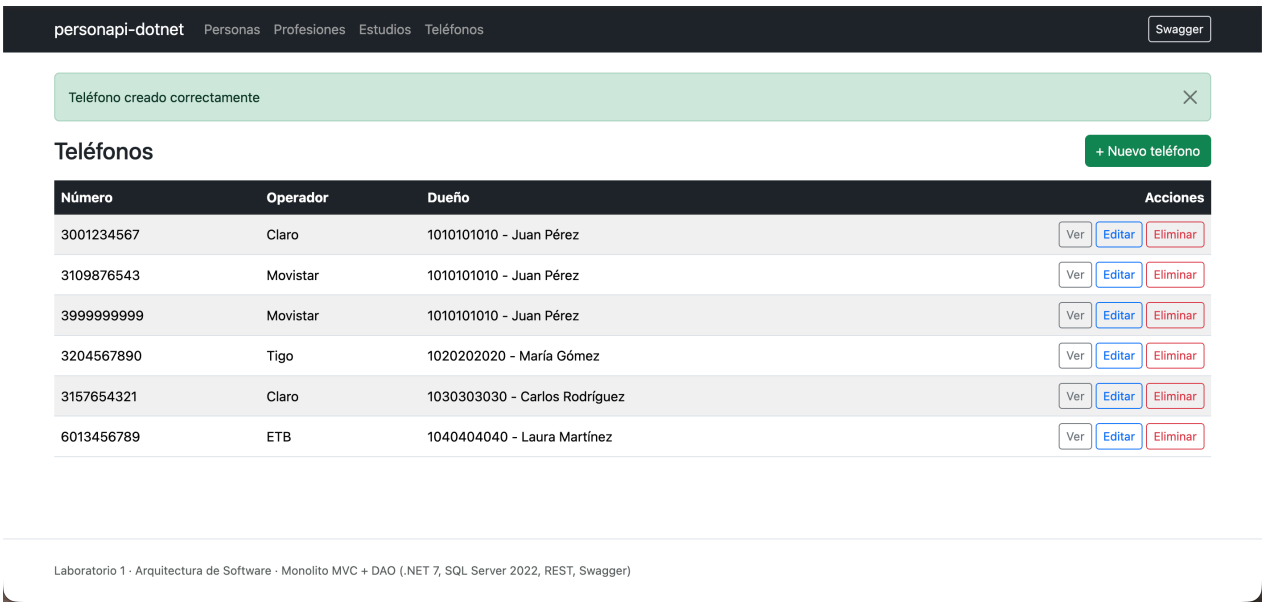


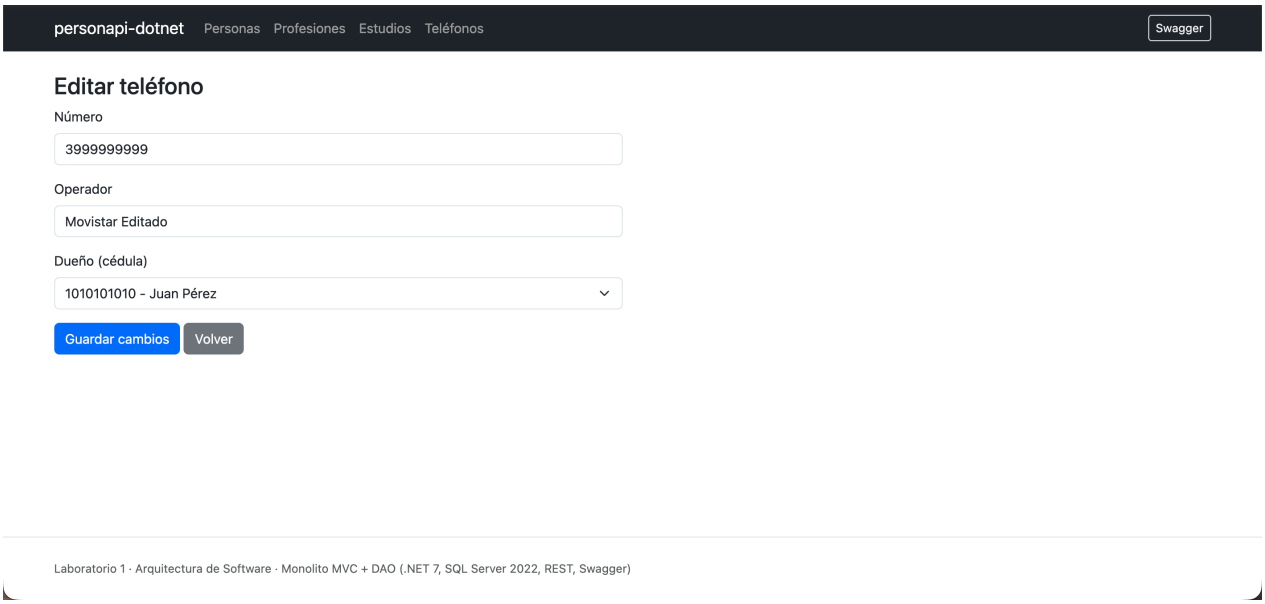


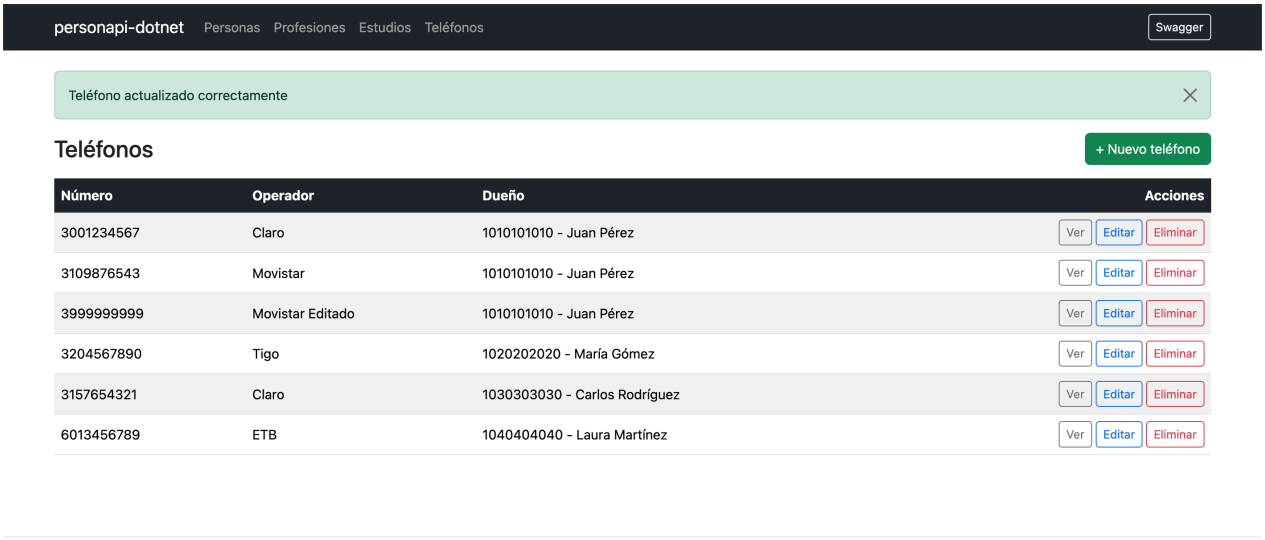
Paso 29. CRUD de teléfonos.

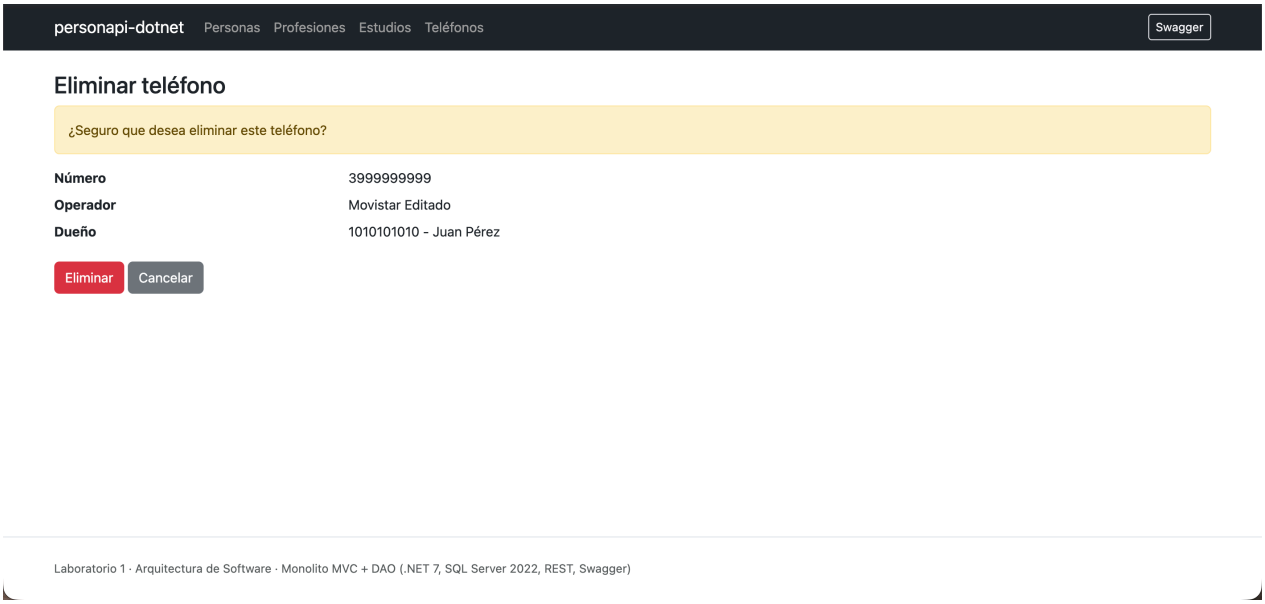

personapi-dotnet
Personas
Profesiones
Estudios
Teléfonos
Swagger
Nuevo teléfono
Número
El número es obligatorio
Operador
El operador es obligatorio
Dueño (cédula)
-- Seleccione --
Seleccione el dueño
Guardar
Volver

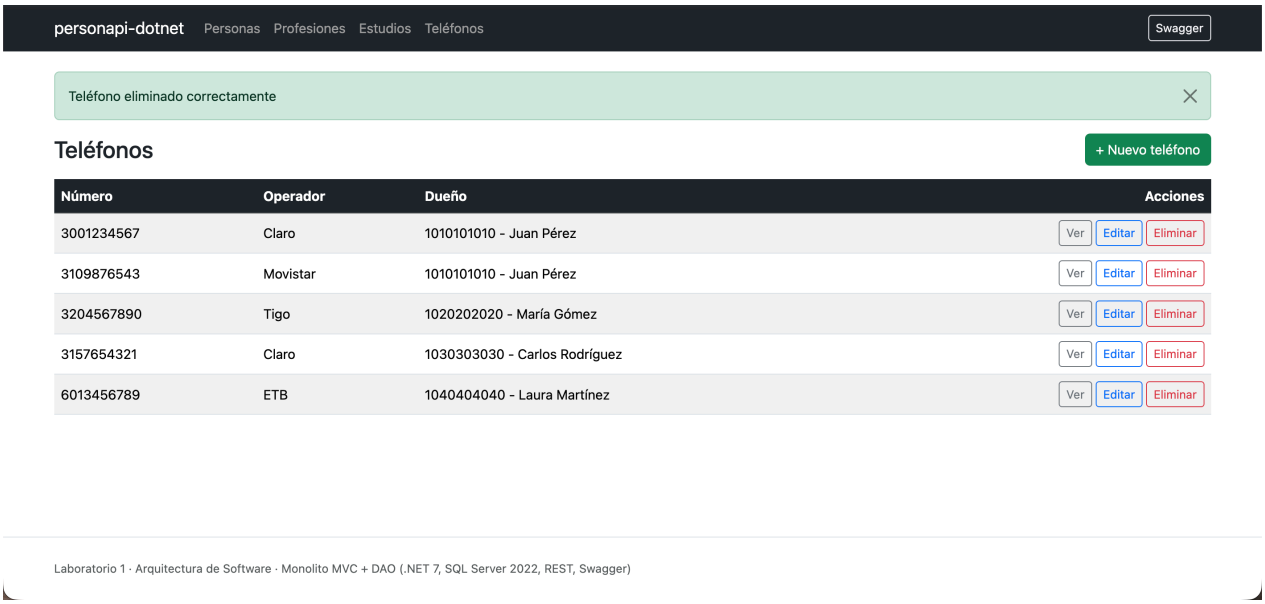

personapi-dotnet
Personas
Profesiones
Estudios
Teléfonos
Swagger
Nuevo teléfono
Número
3999999999
El número es obligatorio
Operador
Movistar
El operador es obligatorio
Dueño (cédula)
1010101010 - Juan Pérez
Seleccione el dueño
Guardar
Volver


personapi-dotnet
Personas
Profesiones
Estudios
Teléfonos
Swagger
Teléfono creado correctamente
Teléfonos
+ Nuevo teléfono
Número
Operador
Dueño
Acciones
3001234567
Claro
1010101010 - Juan Pérez
Ver
Editar
Eliminar
3109876543
Movistar
1010101010 - Juan Pérez
Ver
Editar
Eliminar
3999999999
Movistar
1010101010 - Juan Pérez
Ver
Editar
Eliminar
3204567890
Tigo
1020202020 - María Gómez
Ver
Editar
Eliminar
3157654321
Claro
1030303030 - Carlos Rodríguez
Ver
Editar
Eliminar
6013456789
ETB
1040404040 - Laura Martínez
Ver
Editar
Eliminar

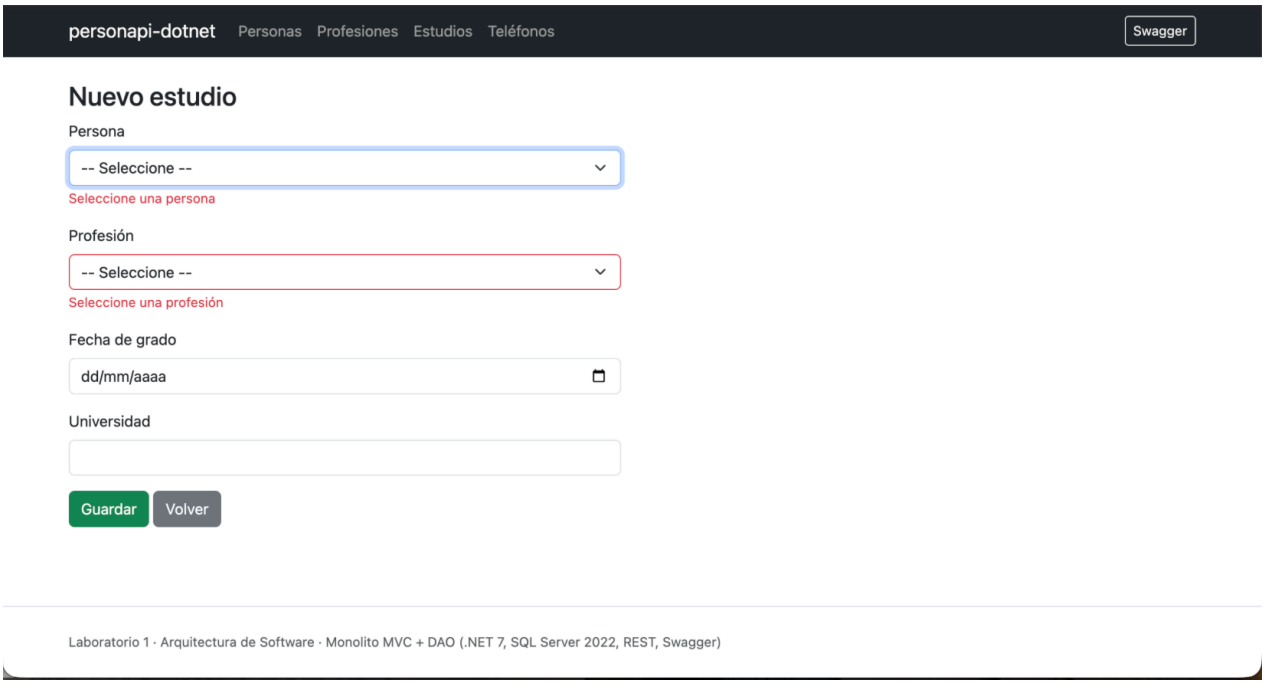

personapi-dotnet
Personas
Profesiones
Estudios
Teléfonos
Swagger
Editar teléfono
Número
3999999999
Operador
Movistar Editado
Dueño (cédula)
1010101010 - Juan Pérez
Guardar cambios
Volver

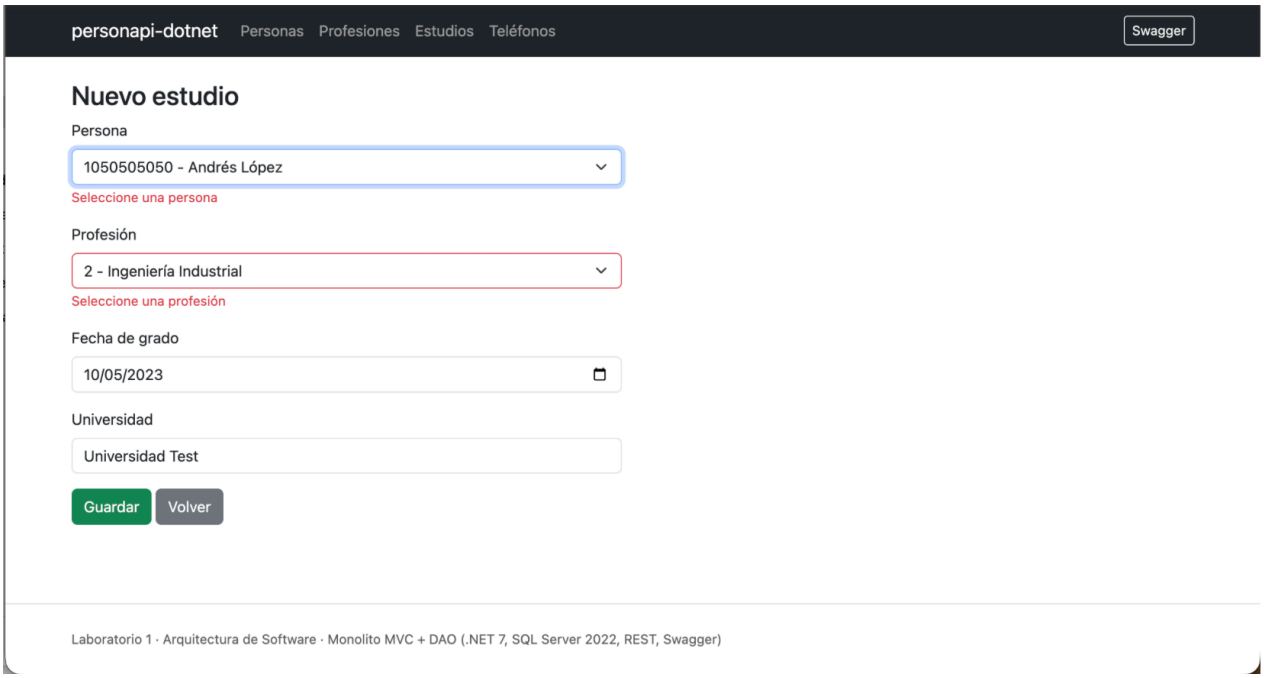

personapi-dotnet
Personas
Profesiones
Estudios
Teléfonos
Swagger
Teléfono actualizado correctamente
Teléfonos
+ Nuevo teléfono
Número
Operador
Dueño
Acciones
3001234567
Claro
1010101010 - Juan Pérez
3109876543
Movistar
1010101010 - Juan Pérez
3999999999
Movistar Editado
1010101010 - Juan Pérez
3204567890
Tigo
1020202020 - María Gómez
3157654321
Claro
1030303030 - Carlos Rodríguez
6013456789
ETB
1040404040 - Laura Martínez
Ver
Editar
Eliminar

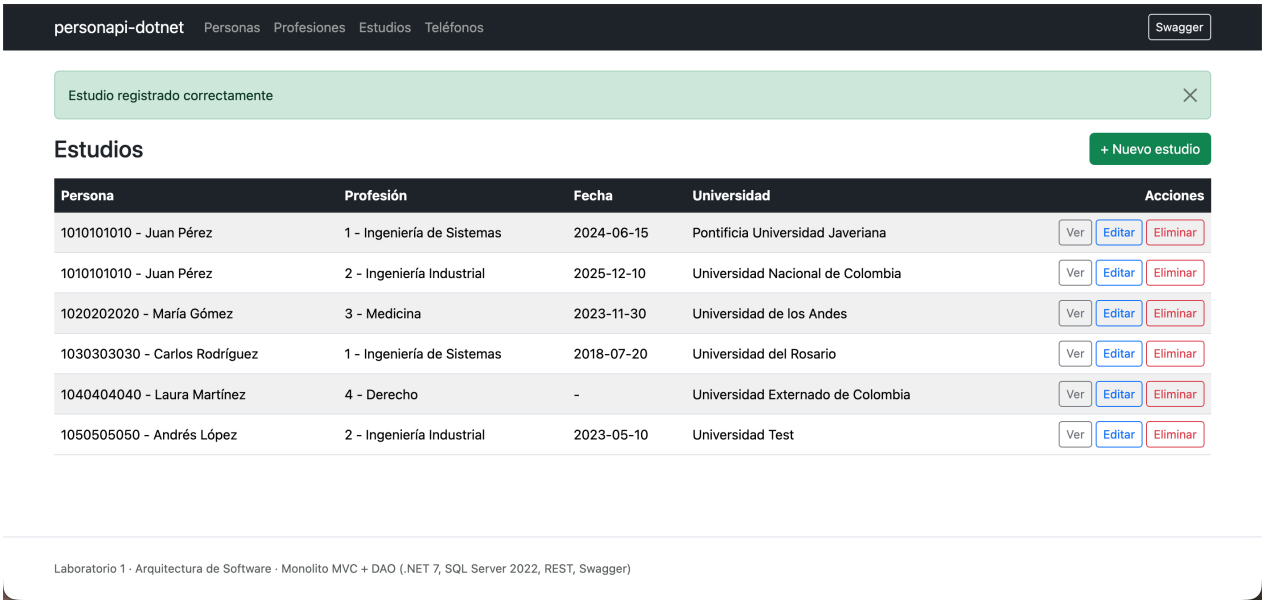

personapi-dotnet
Personas
Profesiones
Estudios
Teléfonos
Swagger
Eliminar teléfono
¿Seguro que desea eliminar este teléfono?
Número
3999999999
Operador
Movistar Editado
Dueño
1010101010 - Juan Pérez
Eliminar
Cancelar

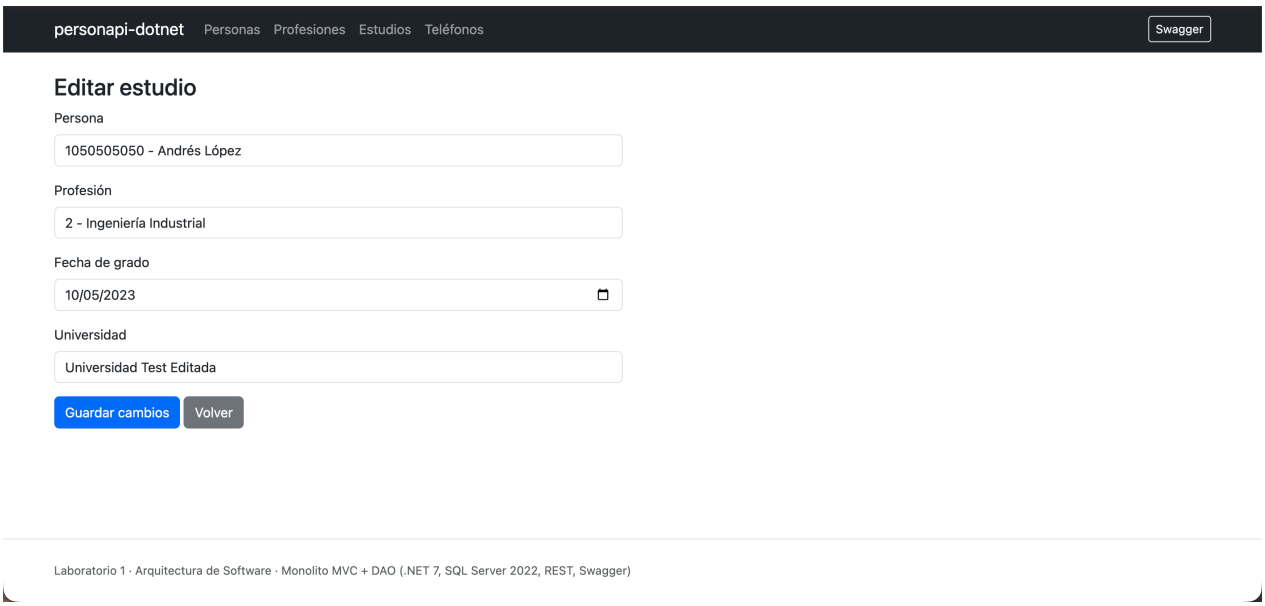


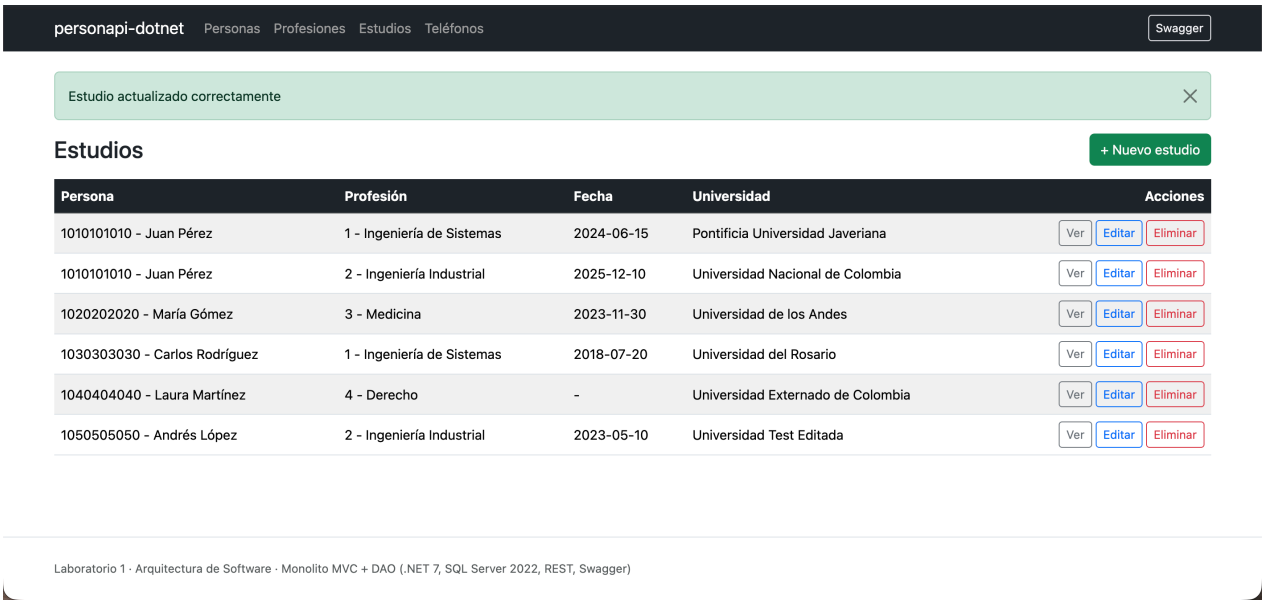
Paso 30. CRUD de estudios.

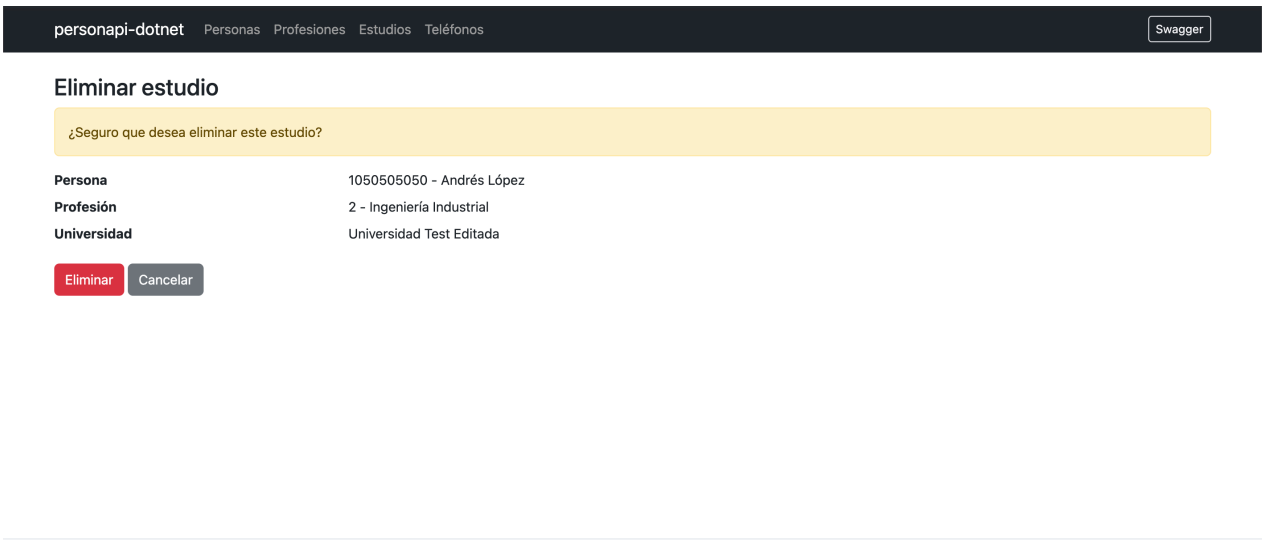


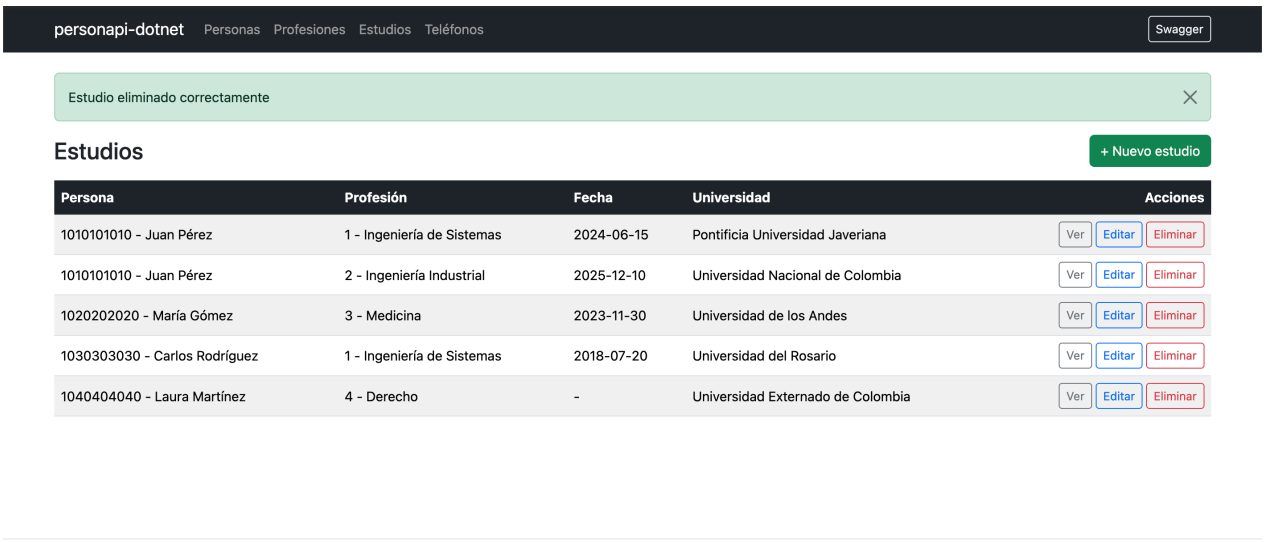

personapi-dotnet
Personas
Profesiones
Estudios
Teléfonos
Swagger
Nuevo estudio
Persona
1050505050 - Andrés López
Seleccione una persona
Profesión
2 - Ingeniería Industrial
Seleccione una profesión
Fecha de grado
10/05/2023
Universidad
Universidad Test
Guardar
Volver
Laboratorio 1 · Arquitectura de Software · Monolito MVC + DAO (.NET 7, SQL Server 2022, REST, Swagger)


personapi-dotnet
Personas
Profesiones
Estudios
Teléfonos
Swagger
Estudio registrado correctamente
Estudios
+ Nuevo estudio
Persona
Profesión
Fecha
Universidad
Acciones
1010101010 - Juan Pérez
1 - Ingeniería de Sistemas
2024-06-15
Pontificia Universidad Javeriana
1010101010 - Juan Pérez
2 - Ingeniería Industrial
2025-12-10
Universidad Nacional de Colombia
1020202020 - María Gómez
3 - Medicina
2023-11-30
Universidad de los Andes
1030303030 - Carlos Rodríguez
1 - Ingeniería de Sistemas
2018-07-20
Universidad del Rosario
1040404040 - Laura Martínez
4 - Derecho
-
Universidad Externado de Colombia
1050505050 - Andrés López
2 - Ingeniería Industrial
2023-05-10
Universidad Test
Ver
Editar
Eliminar
Laboratorio 1 · Arquitectura de Software · Monolito MVC + DAO (.NET 7, SQL Server 2022, REST, Swagger)


personapi-dotnet
Personas
Profesiones
Estudios
Teléfonos
Swagger
Editar estudio
Persona
1050505050 - Andrés López
Profesión
2 - Ingeniería Industrial
Fecha de grado
10/05/2023
Universidad
Universidad Test Editada
Guardar cambios
Volver
Laboratorio 1 · Arquitectura de Software · Monolito MVC + DAO (.NET 7, SQL Server 2022, REST, Swagger)

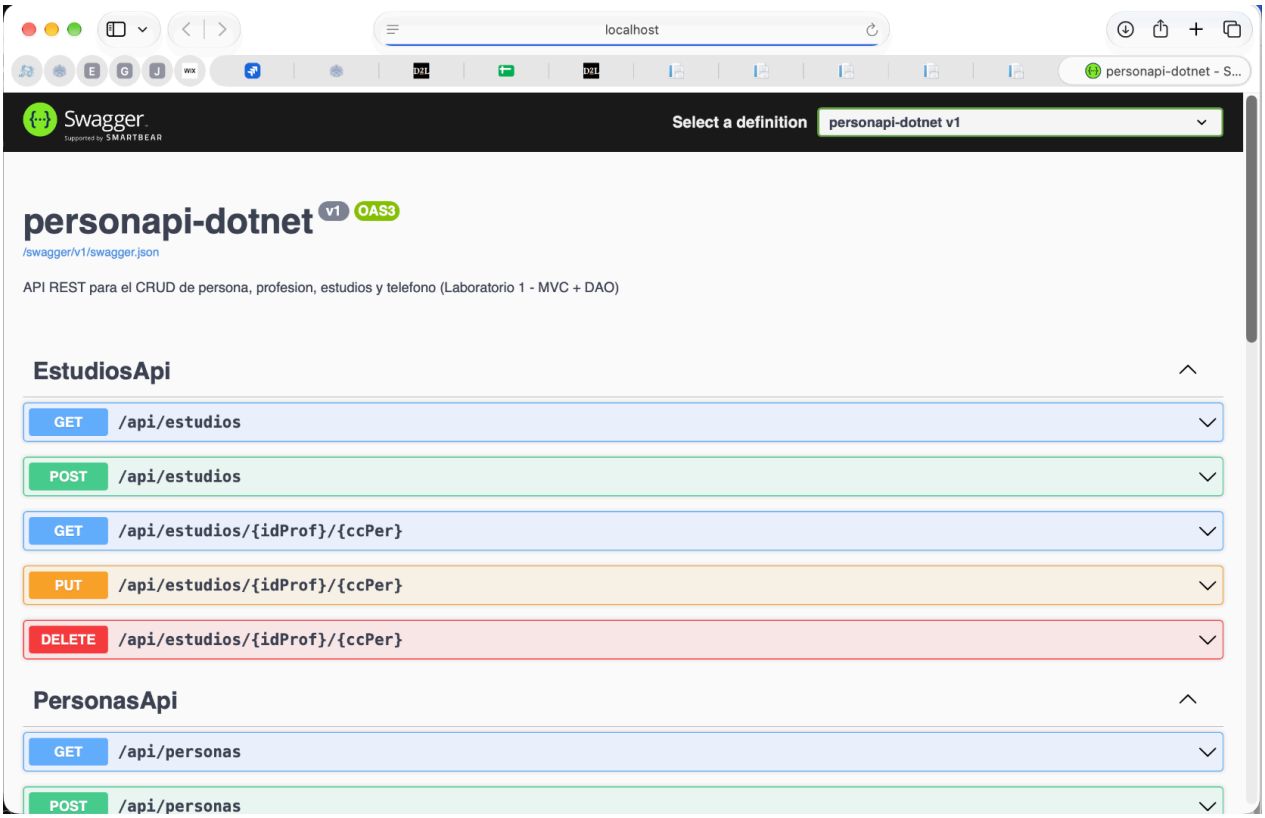

personapi-dotnet
Personas
Profesiones
Estudios
Teléfonos
Swagger
Estudio actualizado correctamente
Estudios
+ Nuevo estudio
Persona
Profesión
Fecha
Universidad
Acciones
1010101010 - Juan Pérez
1 - Ingeniería de Sistemas
2024-06-15
Pontificia Universidad Javeriana
Ver
Editar
Eliminar
1010101010 - Juan Pérez
2 - Ingeniería Industrial
2025-12-10
Universidad Nacional de Colombia
Ver
Editar
Eliminar
1020202020 - María Gómez
3 - Medicina
2023-11-30
Universidad de los Andes
Ver
Editar
Eliminar
1030303030 - Carlos Rodríguez
1 - Ingeniería de Sistemas
2018-07-20
Universidad del Rosario
Ver
Editar
Eliminar
1040404040 - Laura Martínez
4 - Derecho
-
Universidad Externado de Colombia
Ver
Editar
Eliminar
1050505050 - Andrés López
2 - Ingeniería Industrial
2023-05-10
Universidad Test Editada
Ver
Editar
Eliminar


personapi-dotnet
Personas
Profesiones
Estudios
Teléfonos
Swagger
Eliminar estudio
¿Seguro que desea eliminar este estudio?
Persona
1050505050 - Andrés López
Profesión
2 - Ingeniería Industrial
Universidad
Universidad Test Editada
Eliminar
Cancelar


personapi-dotnet
Personas
Profesiones
Estudios
Teléfonos
Swagger
Estudio eliminado correctamente
Estudios
+ Nuevo estudio
Persona
Profesión
Fecha
Universidad
Acciones
1010101010 - Juan Pérez
1 - Ingeniería de Sistemas
2024-06-15
Pontificia Universidad Javeriana
Ver
Editar
Eliminar
1010101010 - Juan Pérez
2 - Ingeniería Industrial
2025-12-10
Universidad Nacional de Colombia
Ver
Editar
Eliminar
1020202020 - María Gómez
3 - Medicina
2023-11-30
Universidad de los Andes
Ver
Editar
Eliminar
1030303030 - Carlos Rodríguez
1 - Ingeniería de Sistemas
2018-07-20
Universidad del Rosario
Ver
Editar
Eliminar
1040404040 - Laura Martínez
4 - Derecho
-
Universidad Externado de Colombia
Ver
Editar
Eliminar

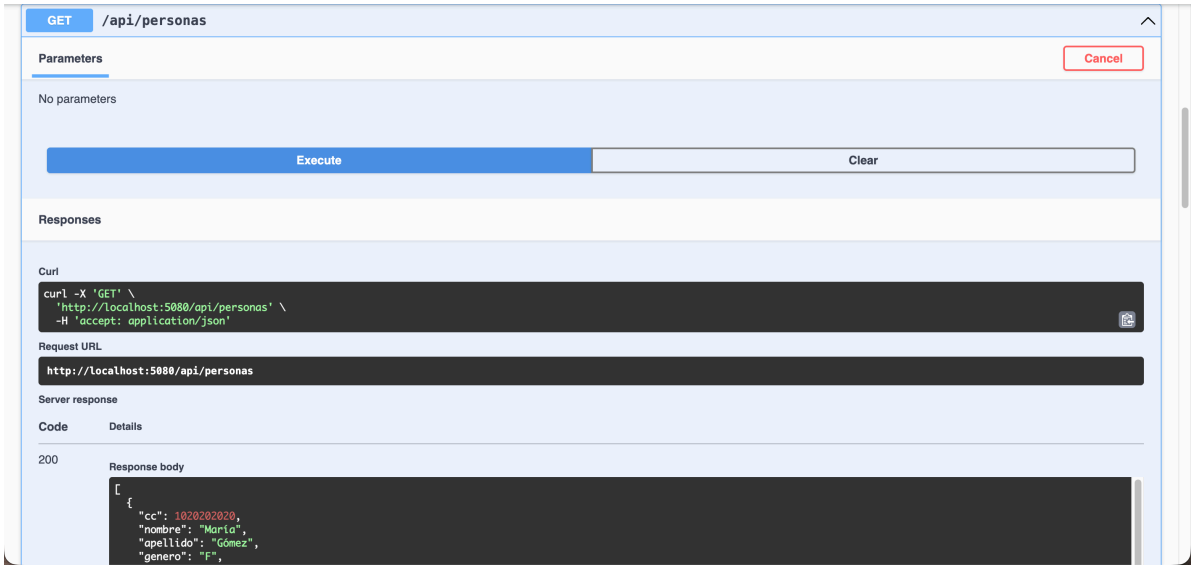
3.9 Pruebas en Swagger

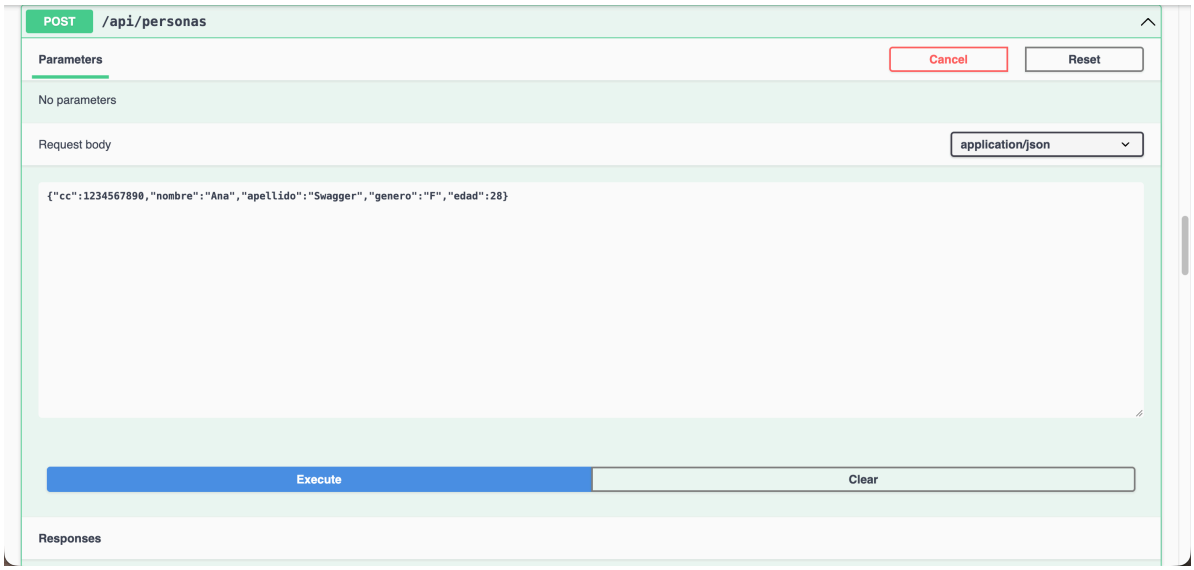
Paso 31. Interfaz general.

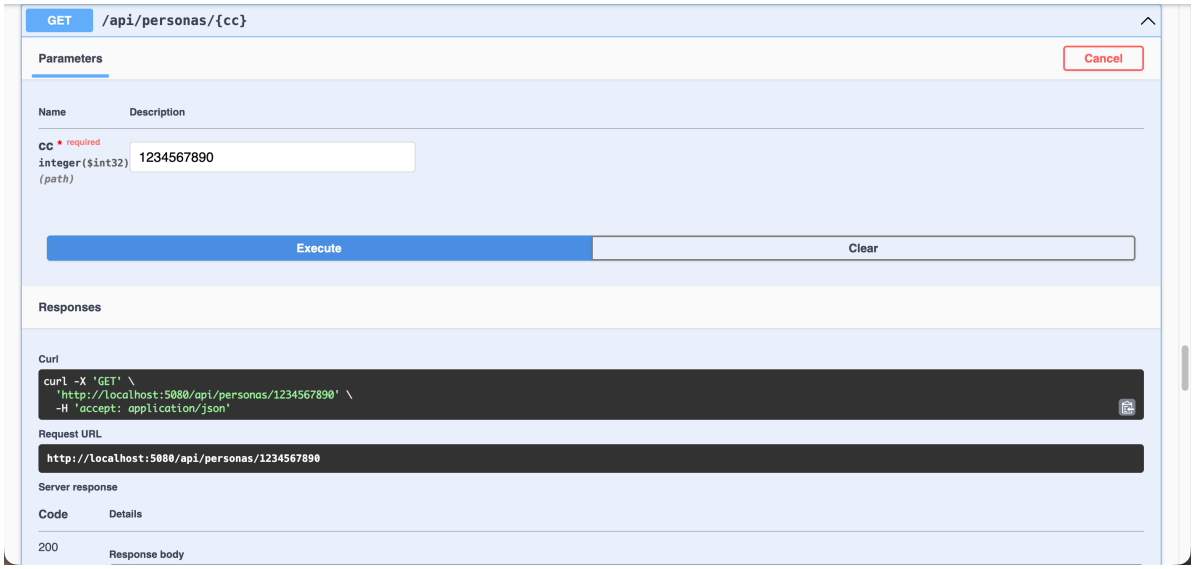


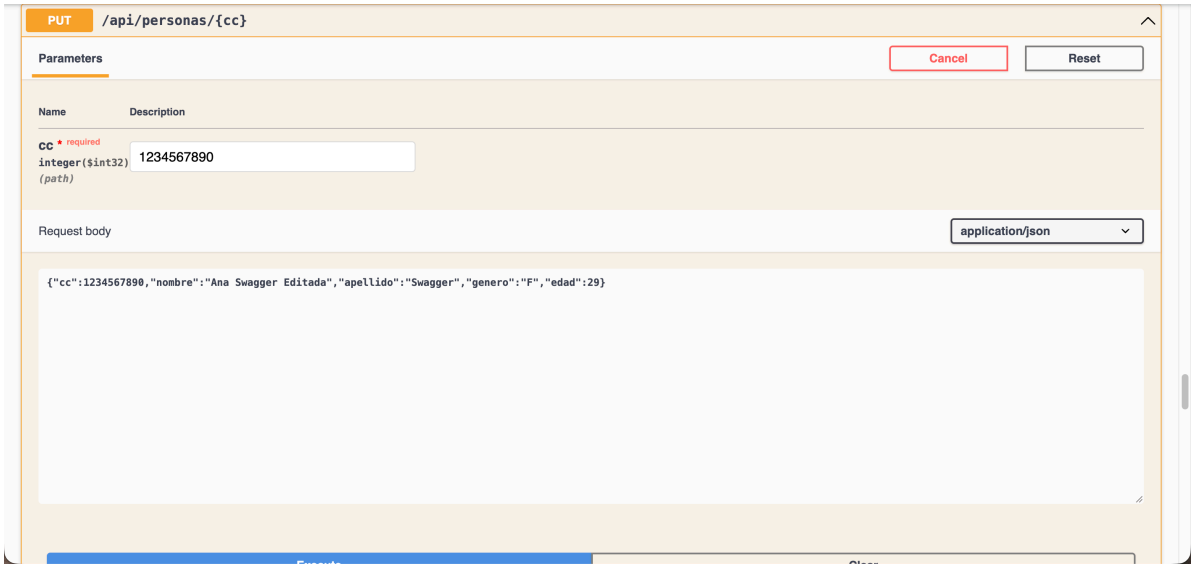
Paso 32. Pruebas por entidad. Para cada una: GET de lista (200), POST (201), GET por id (200), PUT (204), DELETE (204) y GET tras eliminar (404).

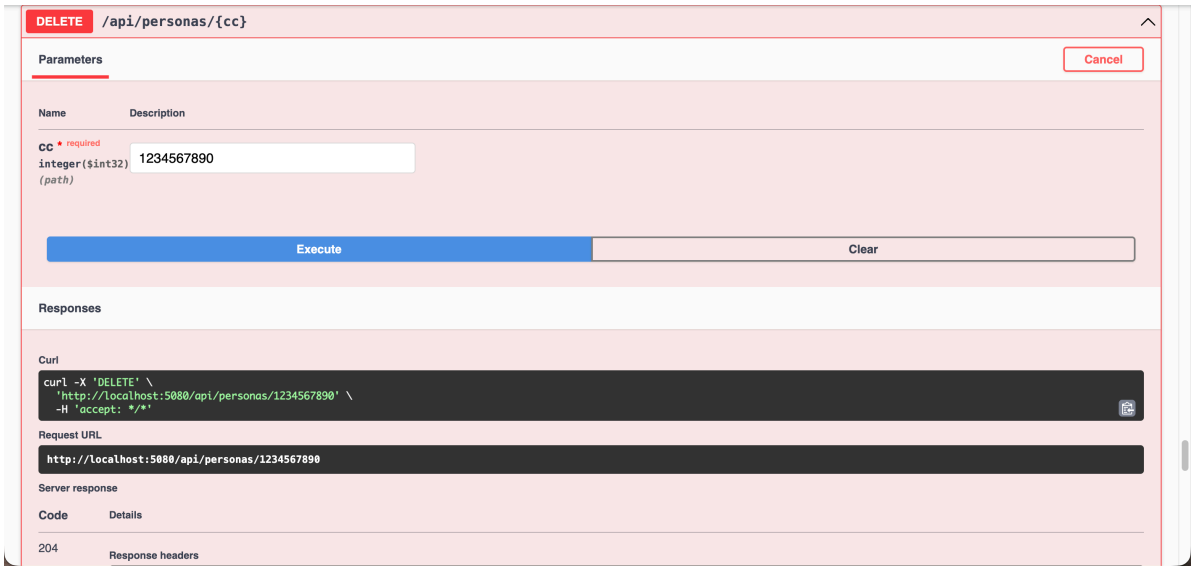
- Personas:

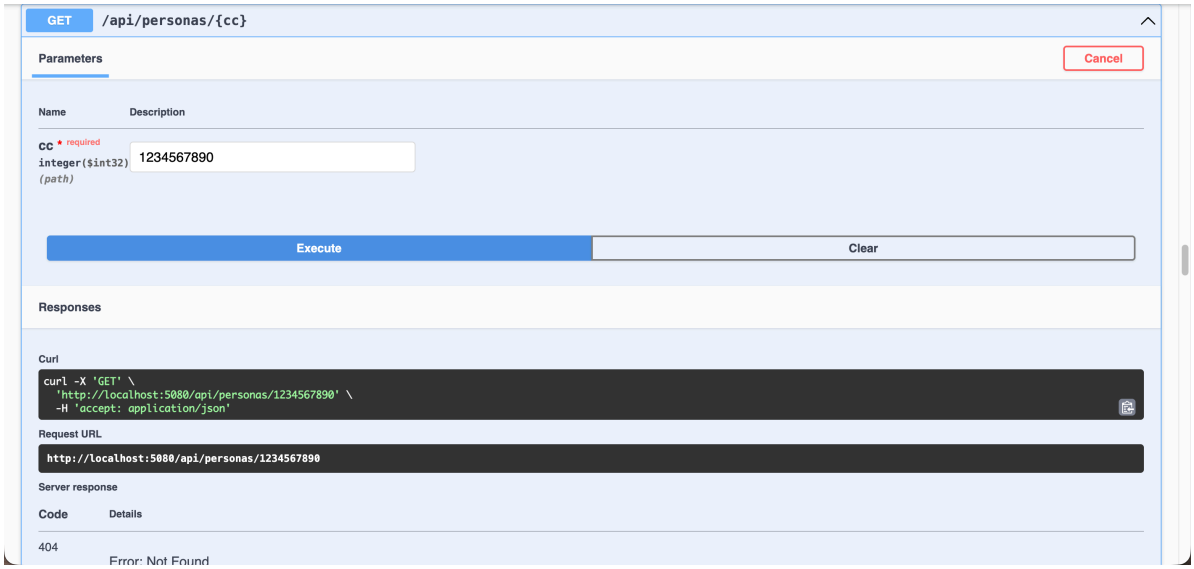



POST
/api/personas
Parameters
Cancel
Reset
No parameters
Request body
application/json
{"cc":1234567890,"nombre":"Ana","apellido":"Swagger","genero":"F","edad":28}
Execute
Clear
Responses

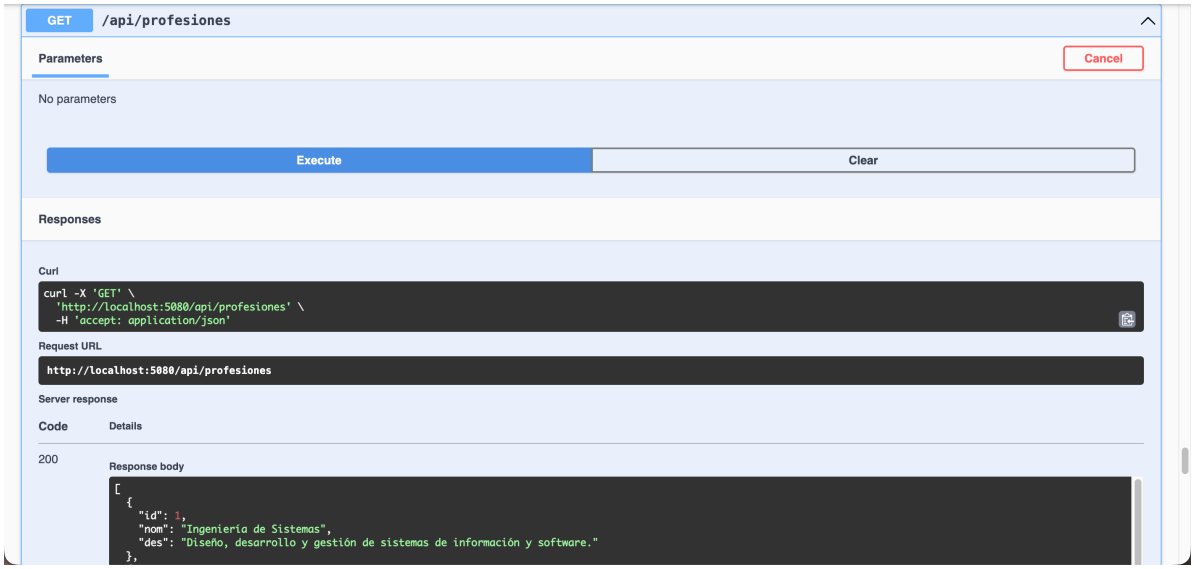

GET
/api/personas/{cc}
Parameters
Cancel
Name
Description
cc * required
integer($int32)
(path)
1234567890
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://localhost:5080/api/personas/1234567890' \
-H 'accept: application/json'
Request URL
http://localhost:5080/api/personas/1234567890
Server response
Code
Details
200
Response body

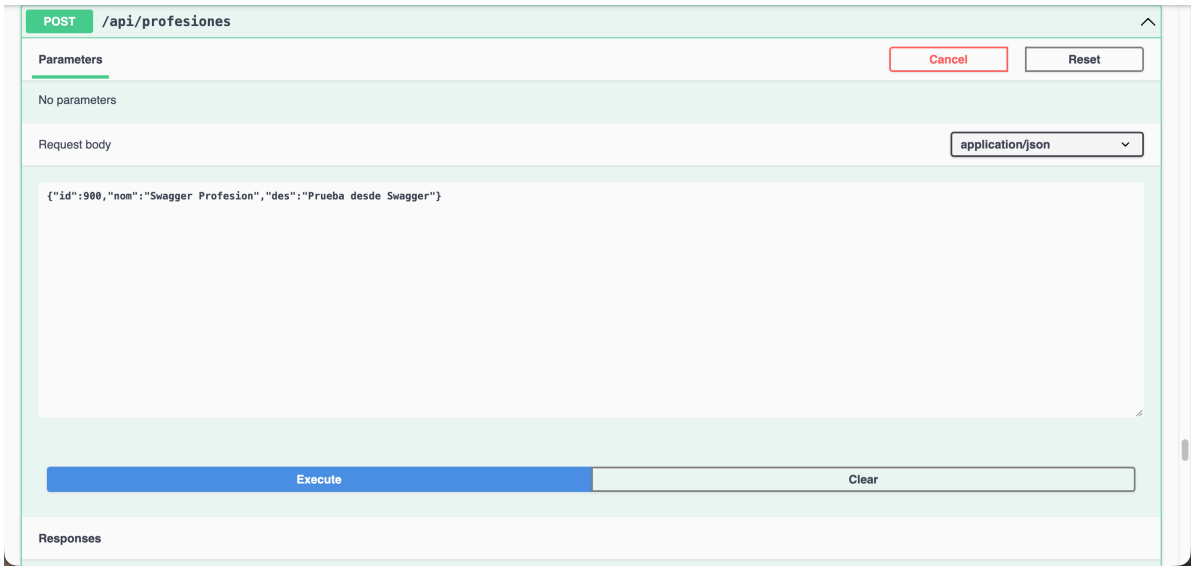

PUT
/api/personas/{cc}
Parameters
Cancel
Reset
Name
Description
cc * required
integer($int32)
(path)
1234567890
Request body
application/json
{"cc":1234567890,"nombre":"Ana Swagger Editada","apellido":"Swagger","genero":"F","edad":29}

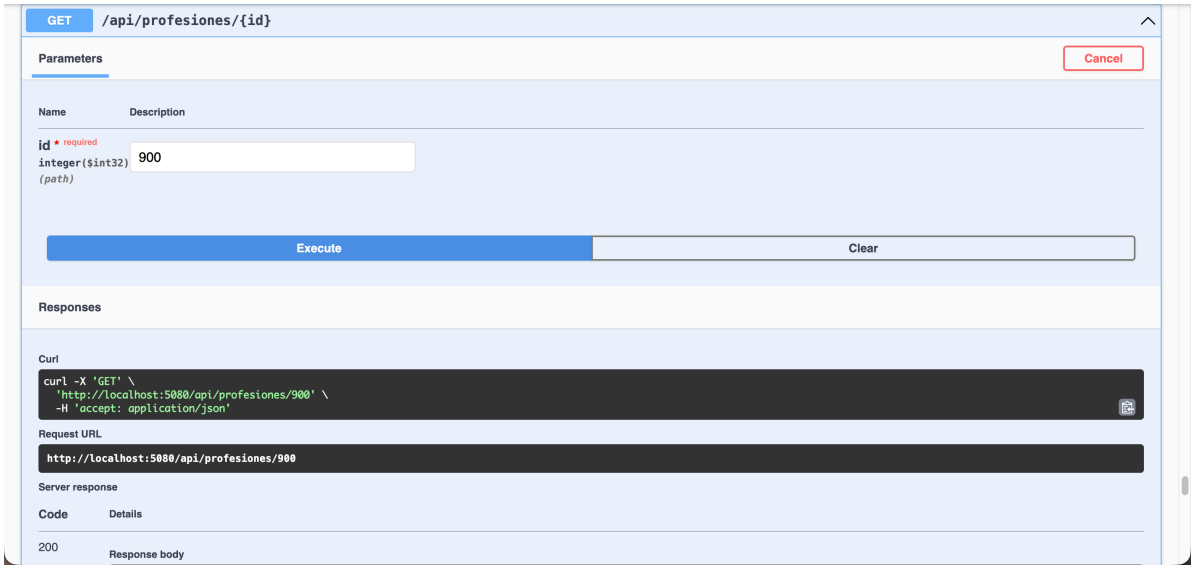


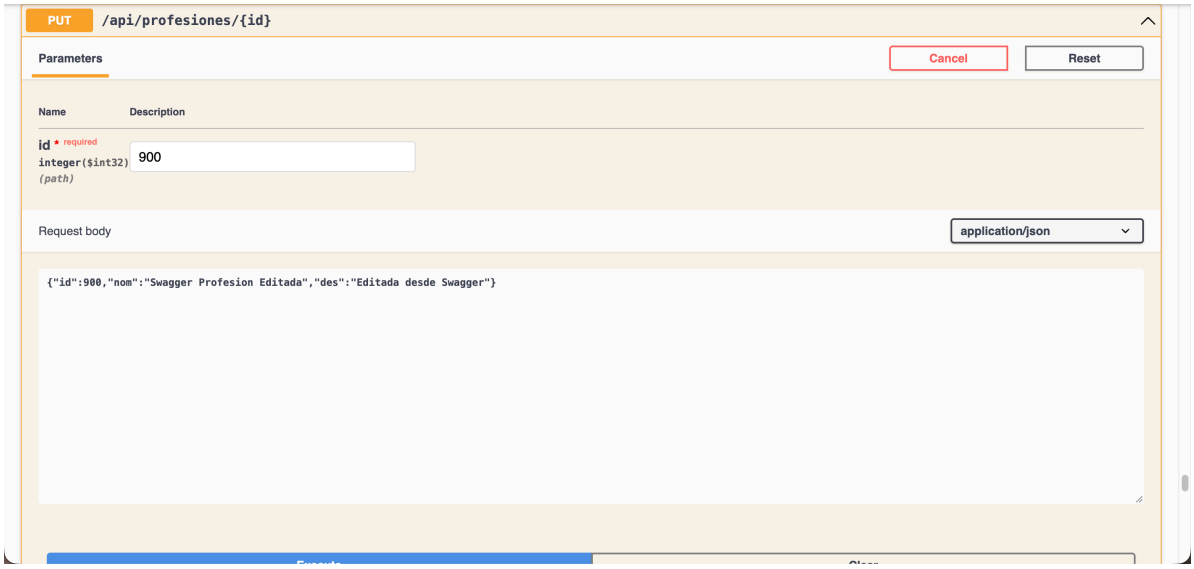


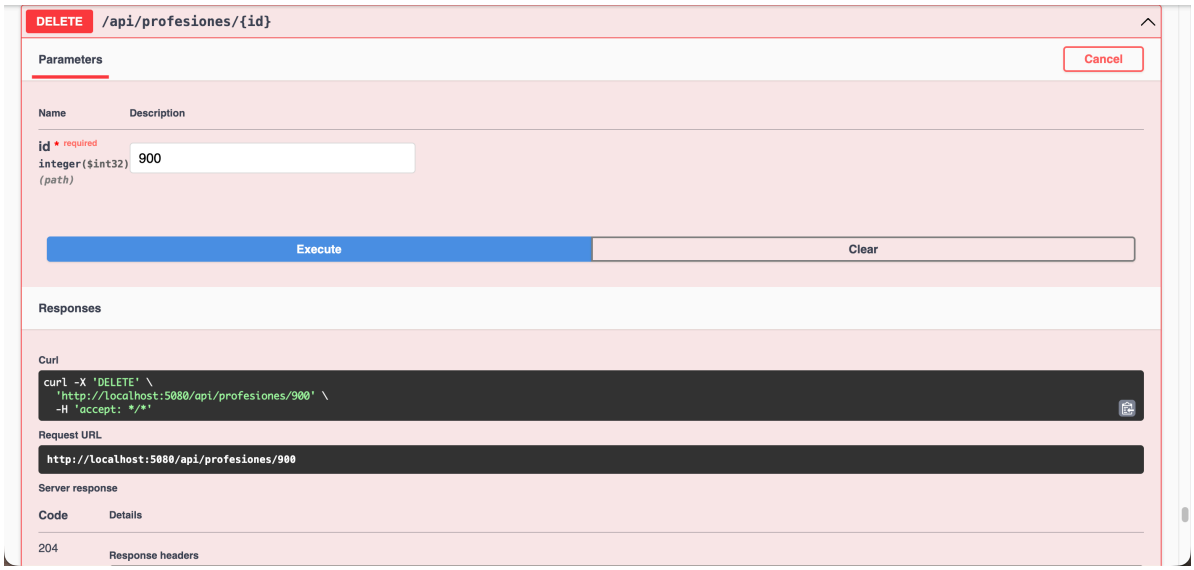
- Profesiones:

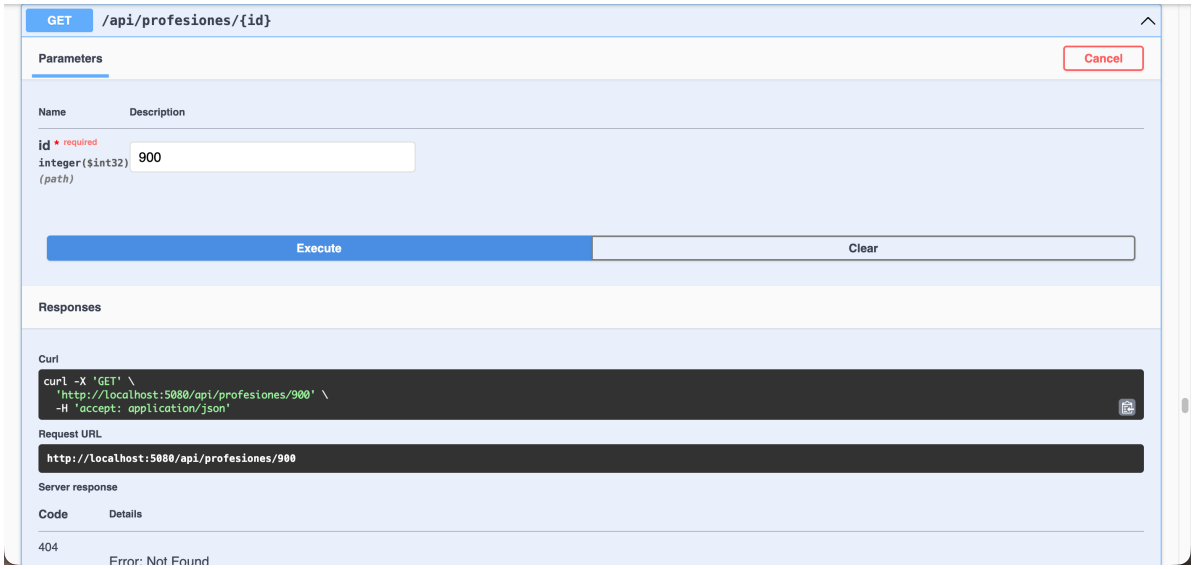



POST
/api/profesiones
Parameters
Cancel
Reset
No parameters
Request body
application/json
{"id":900,"nom":"Swagger Profesion","des":"Prueba desde Swagger"}
Execute
Clear
Responses

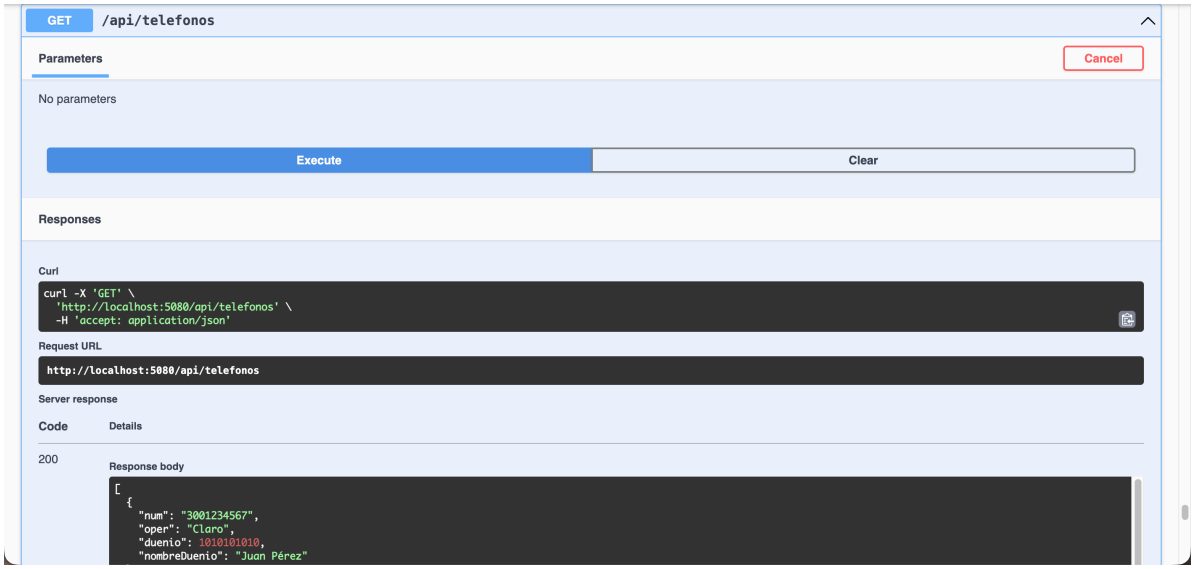

GET
/api/profesiones/{id}
Parameters
Cancel
Name
Description
id * required
integer($int32)
(path)
900
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://localhost:5080/api/profesiones/900' \
-H 'accept: application/json'
Request URL
http://localhost:5080/api/profesiones/900
Server response
Code
Details
200
Response body

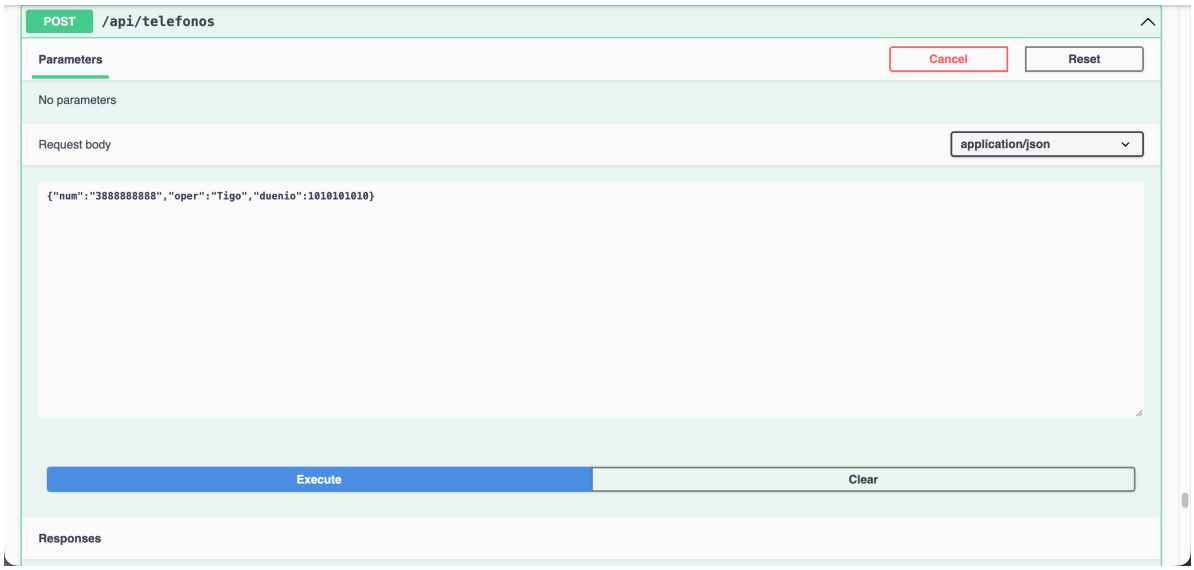

PUT
/api/profesiones/{id}
Parameters
Cancel
Reset
Name
Description
id * required
integer($int32)
(path)
900
Request body
application/json
{"id":900,"nom":"Swagger Profesion Editada","des":"Editada desde Swagger"}

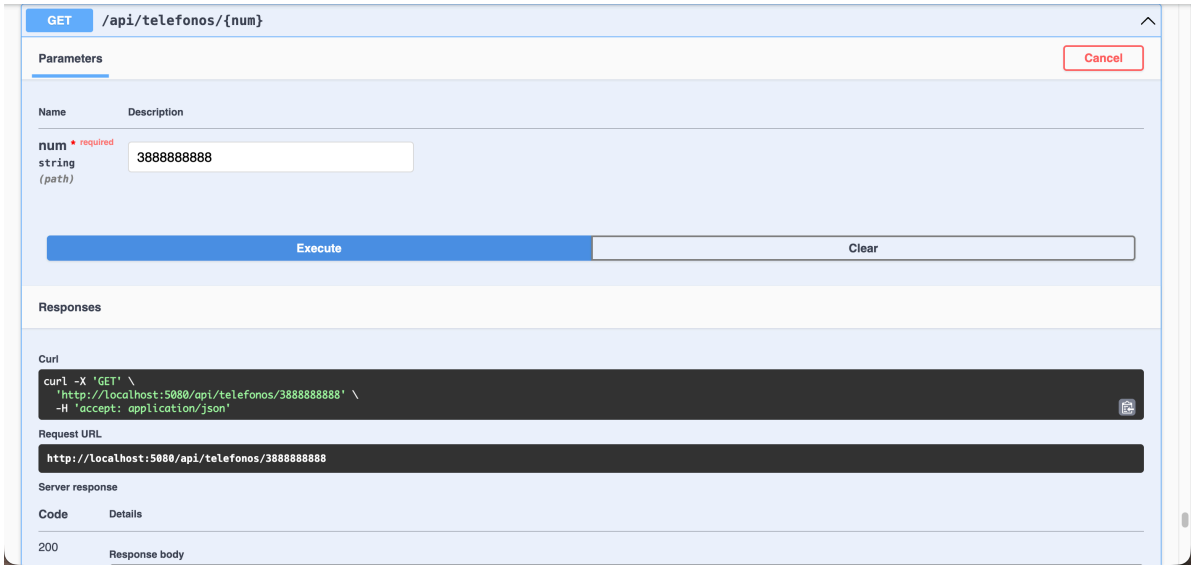

DELETE
/api/profesiones/{id}
Parameters
Cancel
Name
Description
id * required
integer($int32)
(path)
900
Execute
Clear
Responses
Curl
curl -X 'DELETE' \
'http://localhost:5080/api/profesiones/900' \
-H 'accept: */*'
Request URL
http://localhost:5080/api/profesiones/900
Server response
Code
Details
204
Response headers

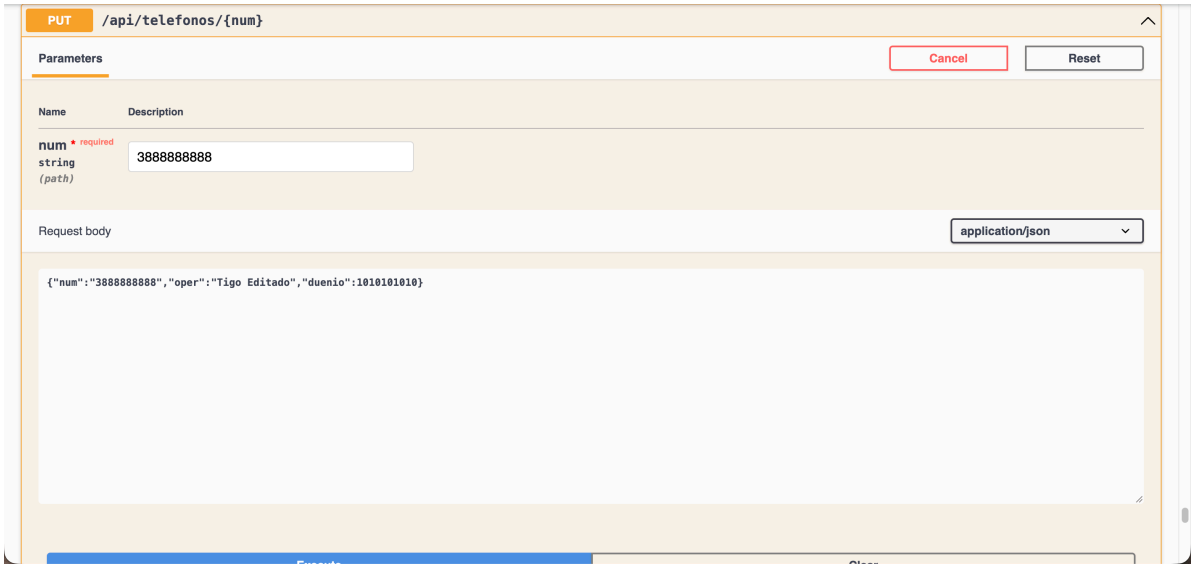

GET
/api/profesiones/{id}
Parameters
Cancel
Name
Description
id * required
integer($int32)
(path)
900
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://localhost:5080/api/profesiones/900' \
-H 'accept: application/json'
Request URL
http://localhost:5080/api/profesiones/900
Server response
Code
Details
404
Error: Not Found

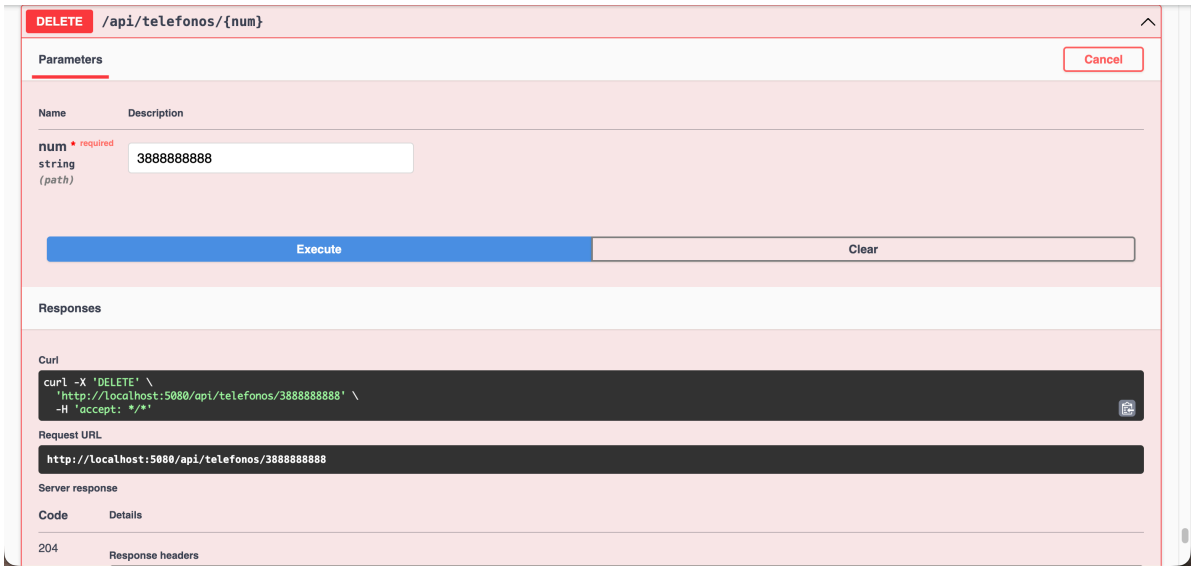
- Teléfonos:

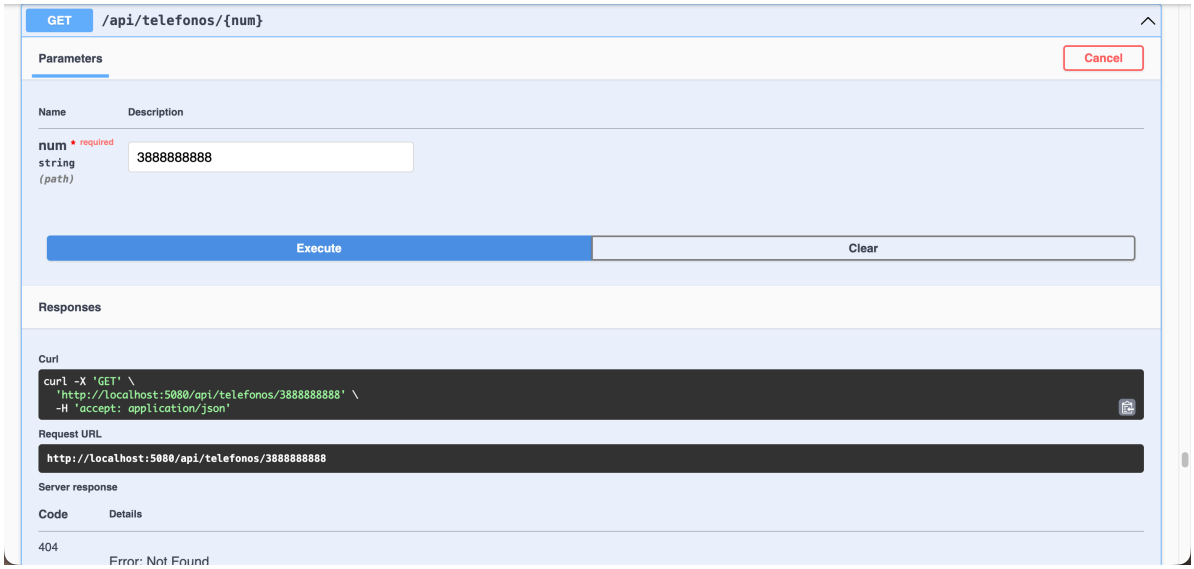

GET
/api/telefonos
Parameters
Cancel
No parameters
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://localhost:5080/api/telefonos' \
-H 'accept: application/json'
Request URL
http://localhost:5080/api/telefonos
Server response
Code
Details
200
Response body
[
{
"num": "3001234567",
"oper": "Claro",
"duenio": 1010101010,
"nombreDuenio": "Juan Pérez"


POST
/api/telefonos
Parameters
Cancel
Reset
No parameters
Request body
application/json
{"num":"3888888888","oper":"Tigo","duenio":1010101010}
Execute
Clear
Responses

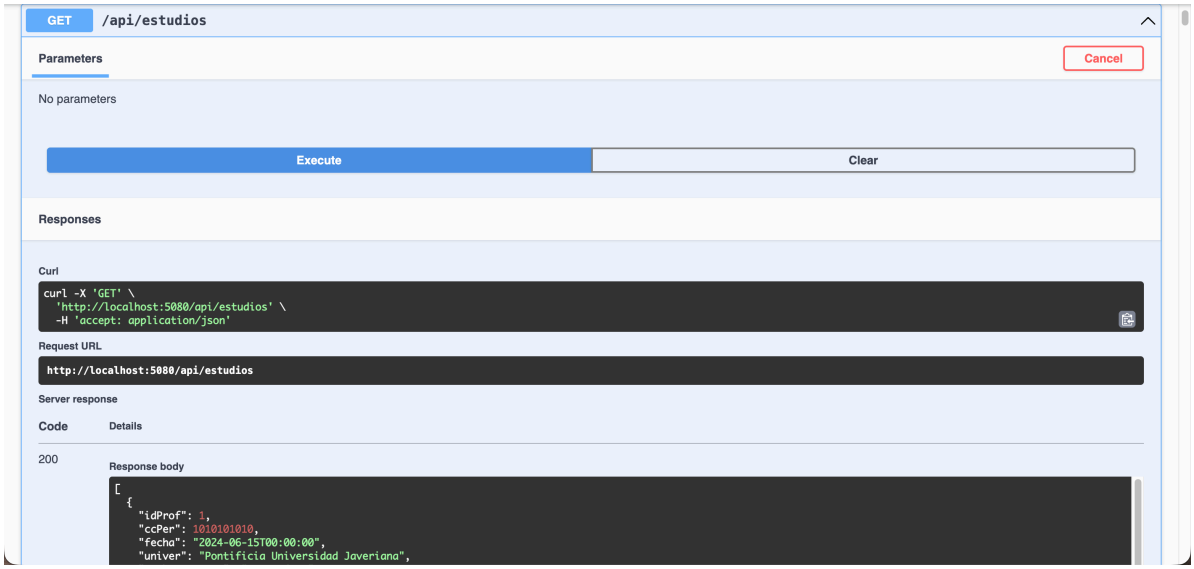

GET
/api/telefonos/{num}
Parameters
Cancel
Name
Description
num
required
string
(path)
3888888888
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://localhost:5080/api/telefonos/3888888888' \
-H 'accept: application/json'
Request URL
http://localhost:5080/api/telefonos/3888888888
Server response
Code
Details
200
Response body

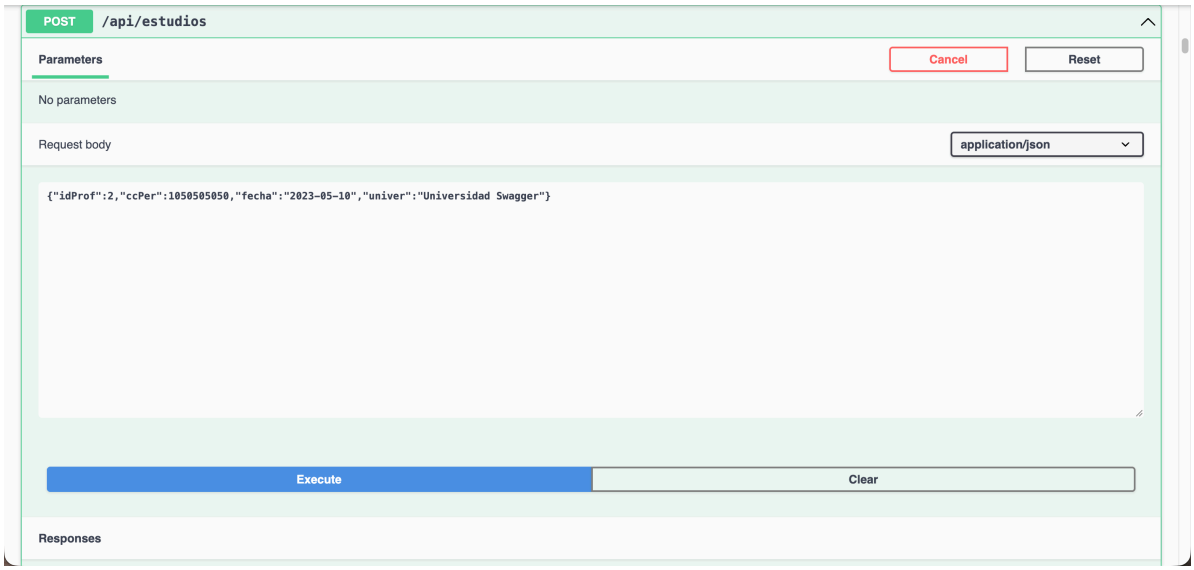

PUT
/api/telefonos/{num}
Parameters
Cancel
Reset
Name
Description
num
required
string
(path)
3888888888
Request body
application/json
{"num":"3888888888","oper":"Tigo Editado","duenio":1010101010}

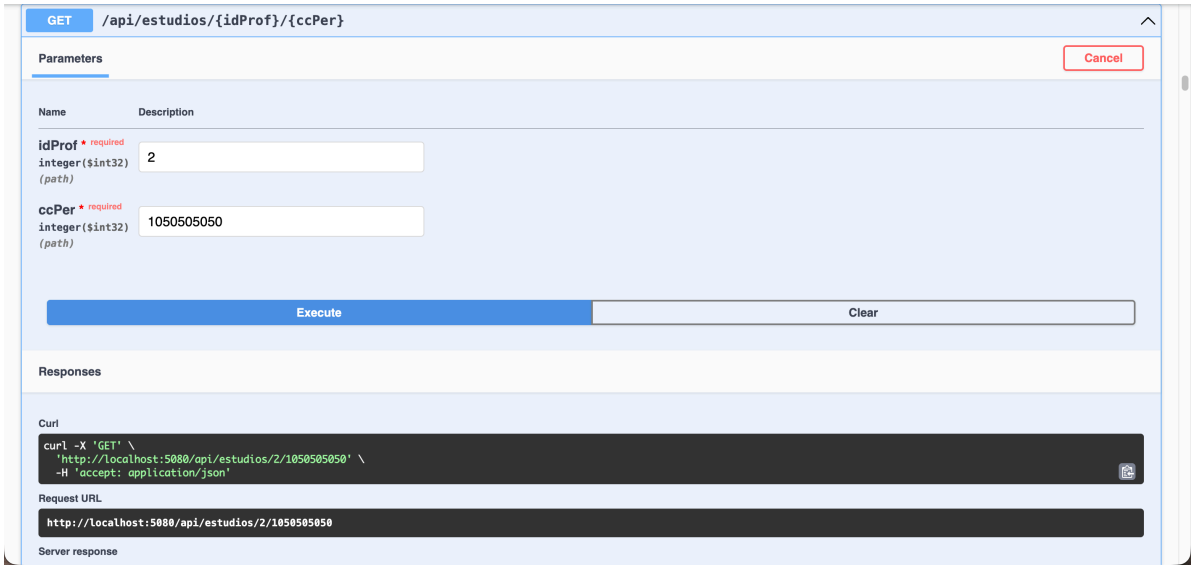

DELETE /api/telefonos/{num}
Parameters
Cancel
Name
Description
num * required
string
(path)
3888888888
Execute
Clear
Responses
Curl
curl -X 'DELETE' \
'http://localhost:5080/api/telefonos/3888888888' \
-H 'accept: */*'
Request URL
http://localhost:5080/api/telefonos/3888888888
Server response
Code
Details
204
Response headers

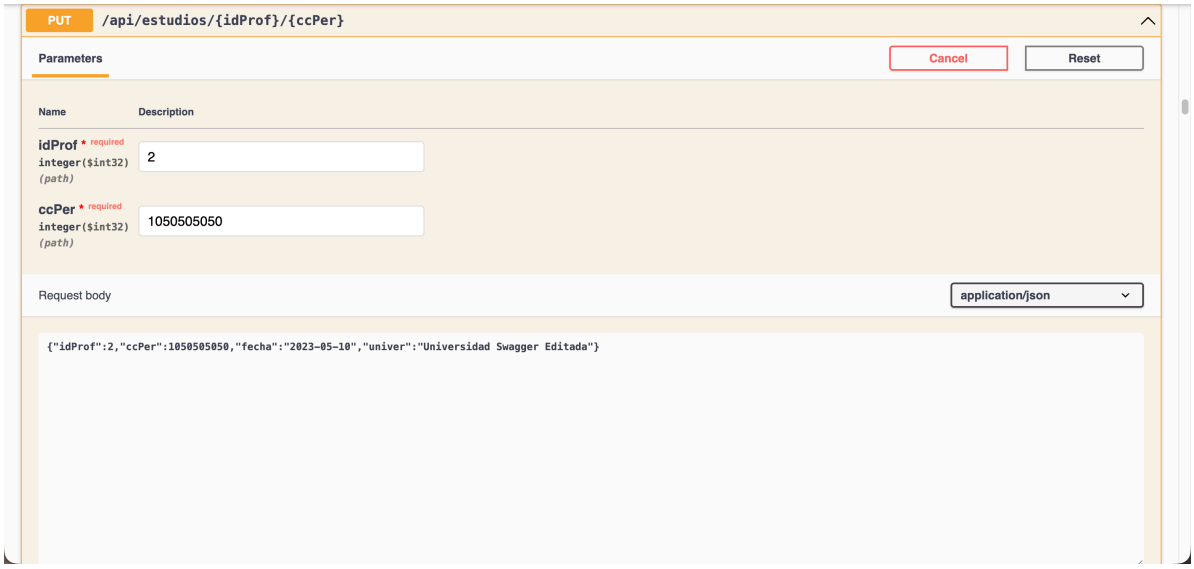

GET /api/telefonos/{num}
Parameters
Cancel
Name
Description
num * required
string
(path)
3888888888
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://localhost:5080/api/telefonos/3888888888' \
-H 'accept: application/json'
Request URL
http://localhost:5080/api/telefonos/3888888888
Server response
Code
Details
404
Error: Not Found

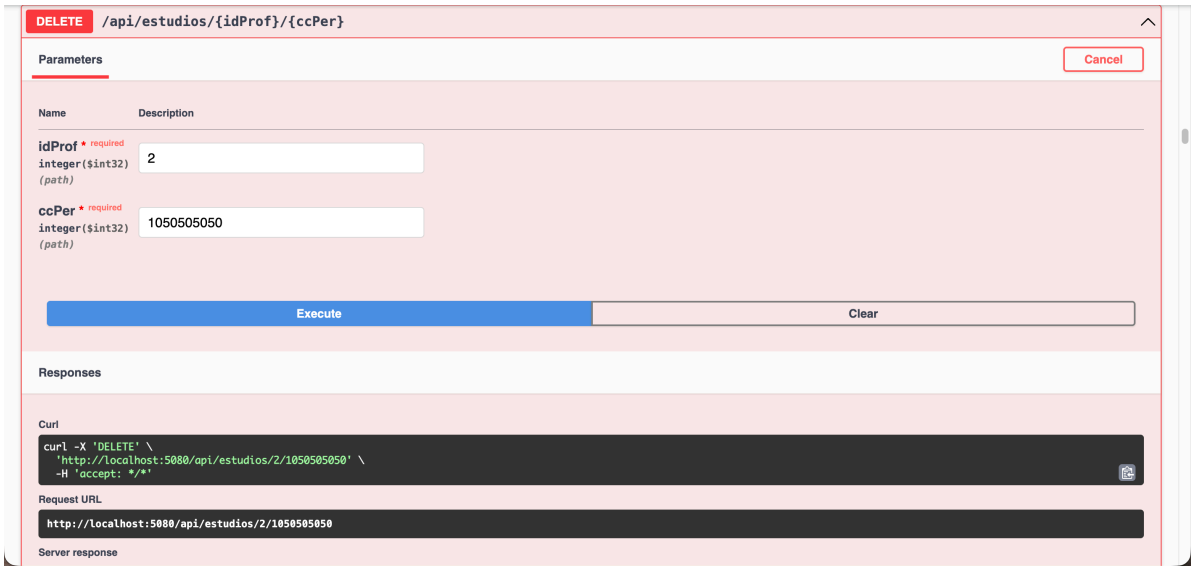
- Estudios:

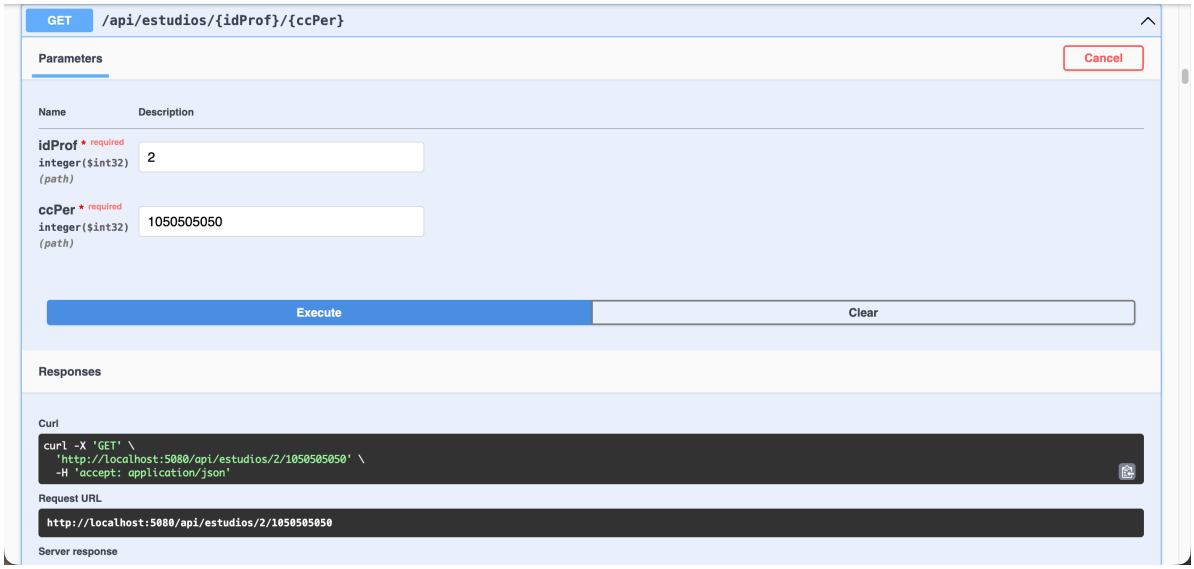

GET /api/estudios
Parameters
Cancel
No parameters
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://localhost:5080/api/estudios' \
-H 'accept: application/json'
Request URL
http://localhost:5080/api/estudios
Server response
Code
Details
200
Response body
[
{
"idProf": 1,
"ccPer": 1010101010,
"fecha": "2024-06-15T00:00:00",
"univer": "Pontificia Universidad Javeriana",


POST
/api/estudios
Parameters
Cancel
Reset
No parameters
Request body
application/json
{"idProf":2,"ccPer":1050505050,"fecha":"2023-05-10","univer":"Universidad Swagger"}
Execute
Clear
Responses


GET
/api/estudios/{idProf}/{ccPer}
Parameters
Cancel
Name
Description
idProf * required
integer($int32)
(path)
2
ccPer * required
integer($int32)
(path)
1050505050
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://localhost:5080/api/estudios/2/1050505050' \
-H 'accept: application/json'
Request URL
http://localhost:5080/api/estudios/2/1050505050
Server response

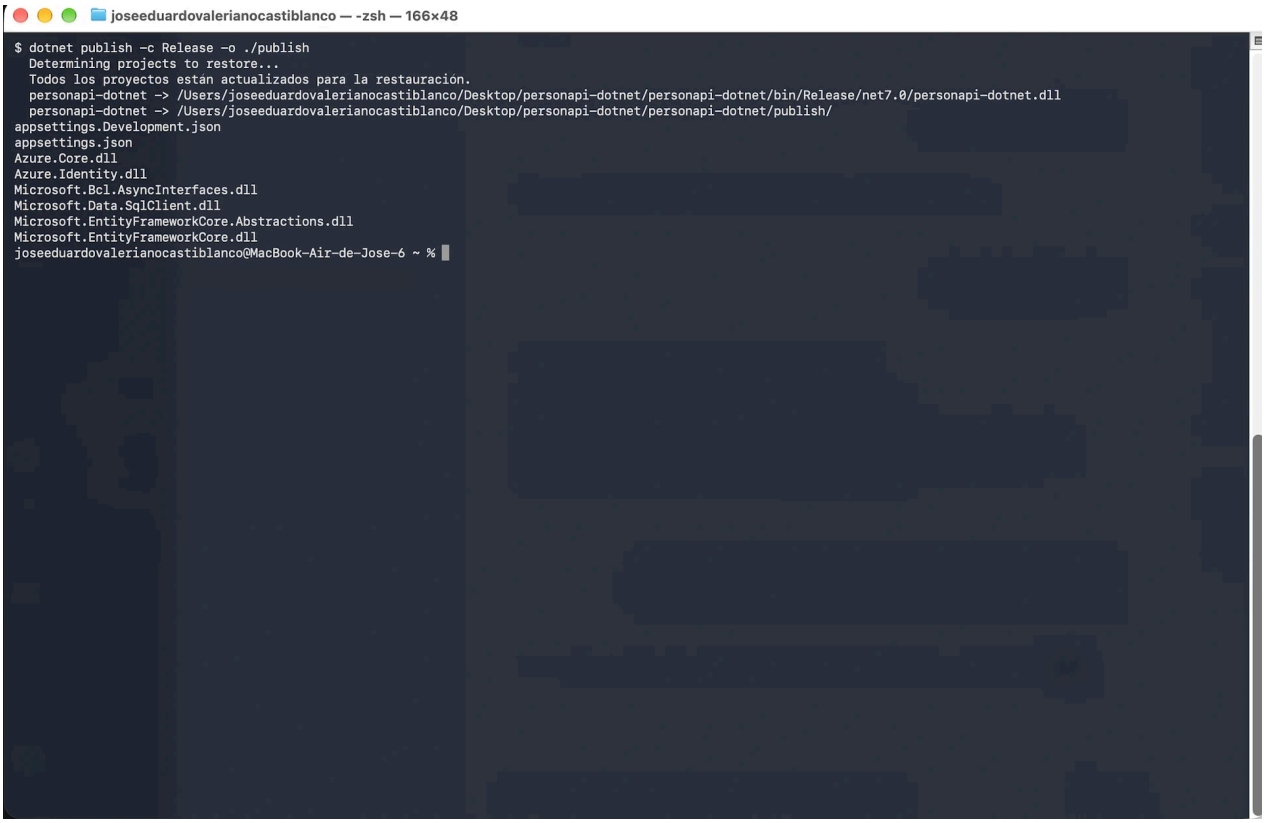

PUT
/api/estudios/{idProf}/{ccPer}
Parameters
Cancel
Reset
Name
Description
idProf * required
integer($int32)
(path)
2
ccPer * required
integer($int32)
(path)
1050505050
Request body
application/json
{"idProf":2,"ccPer":1050505050,"fecha":"2023-05-10","univer":"Universidad Swagger Editada"}



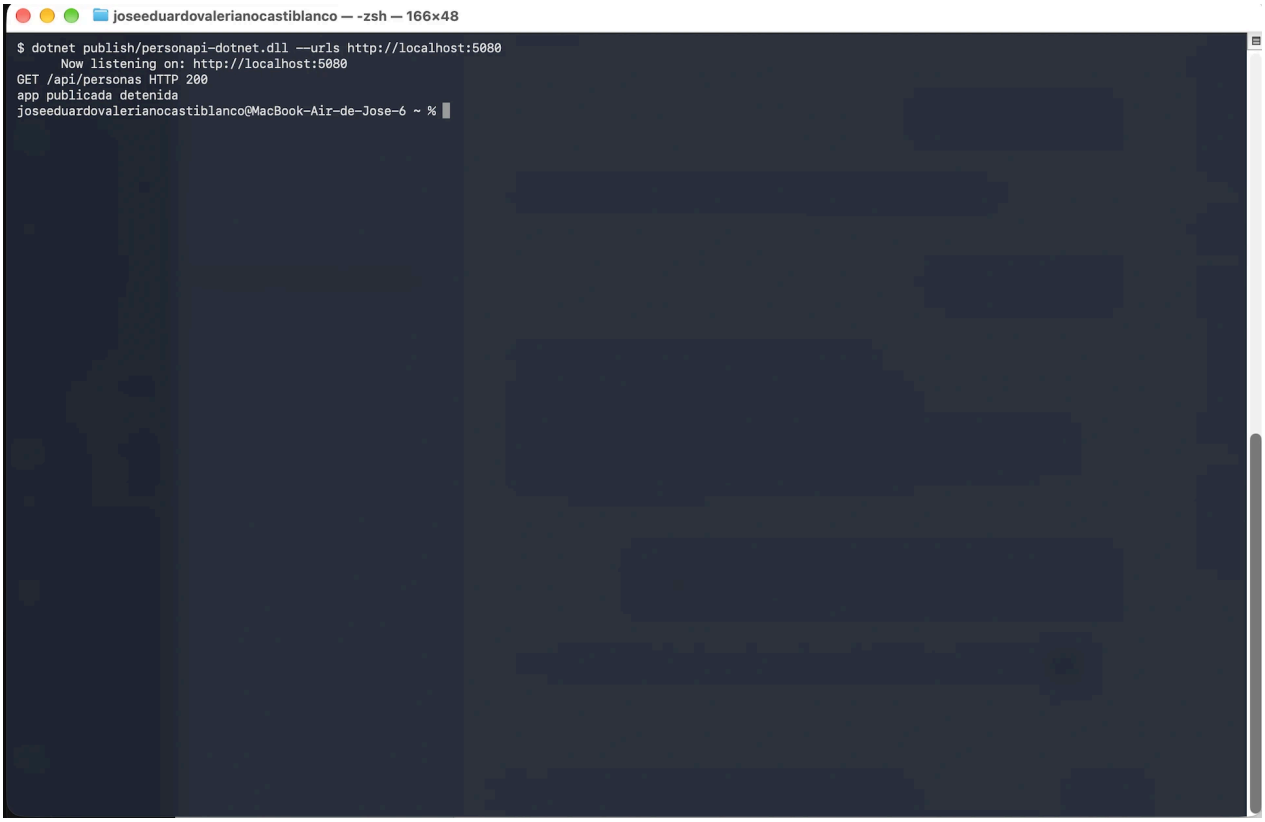


3.10 Publicación

Paso 33. Publicación Release. `dotnet publish -c Release -o ./publish`. La carpeta `publish/` está en `.gitignore`.



Paso 34. Ejecución del publicado. La aplicación publicada responde en el puerto 5080.



3.11 Control de versiones y entrega

Paso 35. Commits y tag. Un commit por etapa (monolito, capturas, documento). El repositorio es público en GitHub. La entrega se marca con el tag anotado `v1.0.0` y el mensaje "Entrega Laboratorio 1".

4. Conclusiones y lecciones aprendidas

1. **La separación en capas facilitó probar cada parte por separado.** Con las interfaces DAO comprobamos el mismo CRUD desde las vistas MVC, la API REST y Swagger, sin tocar los repositorios.
2. **Database First es práctico cuando la base ya existe.** El scaffold generó entidades y contexto en un paso, pero la base queda como fuente de verdad: cualquier cambio en las tablas exige volver a generar las clases.
3. **Montar SQL Server en un Mac tiene particularidades.** La imagen oficial es `amd64`, así que en Apple Silicon hay que activar Rosetta en Docker Desktop. También hay que esperar a que el motor esté listo antes de conectarse, lo que se confirma en los logs del contenedor.
4. **Las claves foráneas sin `ON DELETE CASCADE` obligan a decidir dónde vive la regla de negocio.** Dejar que el repositorio elimine primero los dependientes evitó errores de integridad, pero esa regla hay que recordarla si cambia el motor de base de datos.

Errores que hubo que corregir durante el laboratorio:

- **Permisos de macOS.** La captura de pantalla y el acceso de VS Code a la carpeta del proyecto estaban bloqueados (Grabación de pantalla y Acceso total al disco). Hubo que concederlos y reiniciar las aplicaciones.
- **Instalación del SDK.** El instalador de .NET 7 no quedó completo en el primer intento y hubo que reabrirlo.
- **Docker sin Rosetta.** Sin Rosetta, la imagen de SQL Server no corre en Apple Silicon; se activó en Virtual Machine Options.
- **Datos de prueba borrados.** Al probar DELETE de un estudio de la semilla se perdió un dato; se restauró volviendo a ejecutar `02_dml.sql`.
- **Automatización de navegadores.** Safari no permite ejecutar JavaScript desde AppleScript sin activar una opción de desarrollador, así que los formularios se probaron en Chrome.

---

5. Referencias

Microsoft. (2023). *ASP.NET Core MVC documentation*. https://learn.microsoft.com/aspnet/core/mvc/

Microsoft. (2023). *Entity Framework Core documentation*. https://learn.microsoft.com/ef/core/

Microsoft. (2023). *Scaffolding (reverse engineering) with EF Core*. https://learn.microsoft.com/ef/core/managing-schemas/scaffolding/

Microsoft. (2023). *Quickstart: Run SQL Server container images with Docker*. https://learn.microsoft.com/sql/linux/quickstart-install-connect-docker

Microsoft. (2023). *.NET 7 download*. https://dotnet.microsoft.com/download/dotnet/7.0

Swashbuckle.AspNetCore. *Swashbuckle.AspNetCore 6.5.0* [Paquete NuGet]. https://www.nuget.org/packages/Swashbuckle.AspNetCore/6.5.0

OpenAPI Initiative. (2021). *OpenAPI Specification 3.0.3*. https://spec.openapis.org/oas/v3.0.3

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm

Docker Inc. *Docker Desktop documentation*. https://docs.docker.com/desktop/

Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.